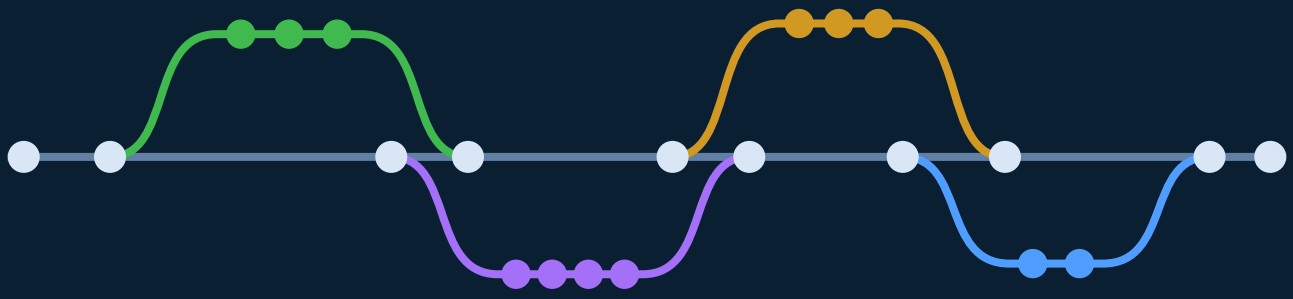
THE COMPLETE GUIDE · OCTOBER 2026 EDITION



GitHub for Teams

Everything you need to learn Git and GitHub and build a real project with your friends: from your first repository to branching, pull requests, code review, rebasing, recovery, GitHub Actions, security and releasing version 1.0.

INSIDE

I GitHub from zero

II Foundations for team work

III Team setup

IV Daily workflow

V History and recovery

VI Planning, automation and quality

VII Shipping

VIII More of GitHub

IX Playbook and reference

For complete beginners and for people who already push their own projects.
Every command explained. Written against Git 2.55, the GitHub CLI and GitHub.com, October 2026.

Contents

How to use this guide 4

Words you need to know 9

Part I: GitHub from zero 13

1. What Git and GitHub are . . . 14
2. Your first repository . . . 16
3. Everyday Git commands . . . 21
4. A tour of GitHub.com . . . 27
5. Writing on GitHub: Markdown . . . 31

Part II: Foundations for team work 34

6. How Git really works . . . 35
7. Set up your computer for team work . . . 41

Part III: Team setup 50

8. Create the team repository . . . 51
9. Protect main with rulesets . . . 60
10. Choose a branching strategy . . . 63

Part IV: Daily workflow 66

11. The team loop, step by step . . . 67
12. Write commits your team can read . . . 73
13. Merge, squash or rebase . . . 77
14. Interactive rebase . . . 81
15. Merge conflicts without panic . . . 87
16. Pull requests that get merged . . . 91
17. Code review as a team . . . 96
18. Forks and upstream remotes . . . 99

Part V: History and recovery 102

19. Undo anything . . . 103
20. Juggle work: stash, cherry-pick, worktrees . . . 108
21. Investigate history . . . 112

Part VI: Planning, automation and quality 117

22. Plan the work with Issues and Projects . . . 118
23. GitHub Actions: CI that guards main . . . 123
24. Deploy from main . . . 132
25. Automate code quality . . . 136
26. Security for small teams . . . 140

Part VII: Shipping 144

27. Releases, tags and versions . . . 145
28. Big files, big repos and shared environments . . . 150

Part VIII: More of GitHub **156**

29. Notifications, profiles and the social side . . . 157
30. Manage your repository . . . 160
31. Discussions, wikis, gists and packages . . . 164
32. Tools: GitHub Desktop, VS Code, mobile and Copilot . . . 167
33. Connect GitHub to other tools: API, webhooks and apps . . . 170

Part IX: Playbook and reference **173**

34. Team project playbook: idea to v1.0 . . . 174
35. Troubleshooting common errors . . . 179
36. Cheat sheet . . . 182
37. Practice drills . . . 186

START HERE

How to use this guide

One guide to learn Git and GitHub, from your first repository to shipping a project with friends.

This guide is for two kinds of readers. If you've never used Git or GitHub, it starts from zero: what they are, how to create an account and your first repository, and the handful of commands you'll use every day. If you already put your own projects on GitHub, it takes you further, into what changes when three or four friends work on the same code. Two people edit the same file on the same evening. Someone pushes a broken build at midnight. A branch drifts two weeks behind `main`, and nobody remembers who was supposed to fix the login bug. Every part of Git and GitHub that deals with those problems is here, together with a complete plan for finishing a project as a team.

Everything is written so that you can follow it on your own:

- Every feature starts with an **"At a glance"** card that answers five questions: *What is it? Why use it? How does it work? When should I use it?* and an *Example* with a short, realistic scenario.
- Every command block is followed by a **"What each line does"** box that explains each line in plain words.
- Harder ideas get an **"In plain English"** box with an everyday comparison.
- Where it helps, you'll see what the output should look like, so you know the command worked.
- Every technical word is explained in **Words you need to know**, the section right after this one.

The examples follow three friends, Alice, Bob and Chen, building a project together in an organization called `team-pixel` (written `my-org` in commands you'll copy).

How it's organized

Part	Covers	Read it when
I	What Git and GitHub are, your first repository, everyday commands, a tour of the website, Markdown	You're new to Git or GitHub. Start here
II	How Git stores work, setting up your computer, SSH keys, signed commits	Once, before a team project starts
III	Creating the organization and repository, protection rules, branching strategy	The person who sets up the repo reads it closely. Everyone else skims it
IV	The daily loop of branch, pull request, review and merge; commits; rebase; conflicts; forks	Everyone, early. You'll come back to it often
V	Undoing mistakes, the reflog, stash, worktrees, searching history, bisect	When something goes wrong (it will)
VI	Issues and project boards, GitHub Actions, deploying, automatic code checks, security	While setting up, then whenever you need it
VII	Version numbers, tags, releases, large files, shared dev environments	Before your first release
VIII	Notifications, profiles, repository management, discussions, wikis, gists, packages, GitHub Desktop, VS Code, Copilot, the API, webhooks and apps	Whenever you want to use more of GitHub
IX	Project playbook, troubleshooting, cheat sheet, practice drills	Keep it open while you work

Where to start

If you are...	Read
Completely new to Git and GitHub	Part I from start to finish (Chapters 1 to 5), doing every step on your own computer. Then Chapters 7, 11 and 12
Comfortable pushing your own projects	Chapters 6 and 7, then Parts III and IV
Starting a team project this week	Chapter 7 (everyone), Chapters 8 to 10 (whoever sets up the repository), Chapter 11 (everyone), then follow the playbook in Chapter 34
Stuck on an error	Chapter 35, then Chapter 19
Looking for one command	The cheat sheet in Chapter 36

How to read the commands in this guide

Where to type them

- **Windows:** use **Git Bash**, which is installed together with Git for Windows. Open your project folder in File Explorer, right-click an empty spot and choose **Open Git Bash here**. Or, in VS Code, open **View** → **Terminal**, click the small arrow next to the **+** button and choose **Git Bash**.
- **macOS:** use the **Terminal** app, or the terminal inside VS Code.
- **Linux:** any terminal.

Most commands must be run *inside your project folder*. Three commands help you find your way:

```
pwd                     # print the folder you're currently in
ls                      # list the files in this folder
cd ~/projects/my-app    # change into a folder
```

WHAT EACH LINE DOES

`pwd`	Short for "print working directory". Shows the full path of the folder the terminal is in right now.
`ls`	Lists files and folders here. `ls -a` also shows hidden ones, such as the `.git` folder.
`cd ~/projects/my-app`	Moves the terminal into that folder. `cd ..` goes up one level. `~` means your home folder.

The parts of a command

```
git  commit  -m  "fix: correct the cart total"
|    |       |   └── argument: the value the option needs
|    |       └────── option (also called a flag): changes how the command behaves
|    └────────────── subcommand: what you want the program to do
└─────────────────── program: git, gh, npm, ssh ...
```

Symbols you'll see

Symbol	What it means
`#`	Start of a comment. Everything after it on that line is a note for you; the shell ignores it. You don't need to type it
`<branch>`	A placeholder. Replace the whole thing, angle brackets included, with your own value, for example `feat/42-login`
`--name` and `-n`	Options. Long options start with two dashes, short options with one. Many options have both forms
`\` at the end of a line	The command continues on the next line. You can also type it all as one long line
`&&`	"And then": run the next command only if the previous one succeeded
`|` (pipe)	Feed the output of the command on the left into the command on the right
`>` and `>>`	Save output into a file. `>` replaces the file's contents; `>>` adds to the end
`$(...)`	Run the command inside the brackets first, and put its output here
`~`	Your home folder: `C:\Users\you` on Windows, `/Users/you` on macOS
`.` and `..`	The current folder, and the folder one level up
`*`	Wildcard. `*.png` means "every file ending in .png"
`"quotes"`	Keep text that contains spaces together as one value

When a command opens an editor

Some commands, like `git commit` without `-m` or `git rebase -i`, open a text editor and wait for you. Once you set it up as in Chapter 7 (or choose it while installing Git, Chapter 2), that editor is VS Code: write your text, save with **Ctrl+S** (**Cmd+S** on a Mac), then close the tab. Git continues as soon as the tab is closed.

> **Stuck in a strange black screen with ~ symbols?**
>
> That's Vim, Git's default editor until you change it (Chapter 7). Press **Esc**, then type `:wq` and press **Enter** to save and exit. To exit without saving, press **Esc** and type `:q!` instead.

When in doubt, run git status

`git status` never changes anything. It tells you which branch you're on, which files have changed, what's staged, and often suggests the exact command to run next. Run it whenever you're unsure what state you're in.

What output looks like

When it helps to show what a command prints, the output appears as comment lines (starting with `#`) inside the command block, or in a separate grey block. The exact numbers and names on your screen will be different.

Conventions

- `my-org/my-app` stands for your organization and repository. `alice`, `bob` and `chen` stand for your friends' GitHub usernames.
- Hashes like `4e1d2c7` are examples. Yours will be different.
- Commands are written for Git Bash, macOS and Linux. Plain `git` and `gh` commands also work in PowerShell; lines that use Unix tools such as `grep`, `awk` or `xargs` need Git Bash.

Indigo **At a glance** cards introduce each feature with those five rows. The colored boxes mean the same thing everywhere:

> **In plain English**
> Teal boxes explain an idea with an everyday comparison.

> **Tip**
> Green boxes are shortcuts and habits that save time.

> **Watch out**
> Amber boxes are traps that people fall into regularly.

> **Danger: you can lose work**
> Red boxes mark commands that can delete work Git can't give back.

> **Team rule**
> Purple boxes are rules worth copying into your team's CONTRIBUTING file.

Versions this guide assumes

Git 2.40 or newer (2.55 was current when this was written; check yours with `git --version`), the GitHub CLI `gh`, and GitHub.com as of October 2026. GitHub moves buttons around from time to time. If a setting isn't where this guide says, use the search box at the top of the settings page, or look it up on docs.github.com.

> **Free plan or paid plan?**
> On private repositories, some team features only work if the account that owns the repo pays for GitHub: branch protection and rulesets, required reviewers, code owners, and GitHub Pages. On public repositories they're free. Chapter 8 explains your options, including GitHub Pro for students at no cost.

START HERE

Words you need to know

The vocabulary of Git and GitHub, in plain English.

Git is a program on your computer that records the history of a project folder. **GitHub** is a website that stores copies of Git repositories online and adds tools for working together: pull requests, reviews, issues, automation. Git works without GitHub; GitHub is built on Git. Come back to this list whenever a word in the guide is unfamiliar.

Git basics

Term	Meaning
Repository (repo)	A project folder whose history Git tracks. The history lives in a hidden `.git` folder inside it
Commit	A saved snapshot of the whole project at one moment, with a message saying what changed, who made it and when
Hash (SHA)	The unique ID of a commit: a 40-character code such as `4e1d2c7a9b...`. The first 7 characters are usually enough to identify it
Working tree	The files you actually see and edit in your project folder
Staging area (index)	A waiting area for changes you want in your next commit. `git add` puts changes there; `git commit` saves whatever is waiting
Branch	A separate line of work with its own name. Technically it's a label pointing at the newest commit on that line. It lets you work without disturbing `main`
main	The default branch: the version of the project everyone agrees is current and working
HEAD	Git's "you are here" marker. It usually points at the branch you're working on
Switch / checkout	Change which branch (or commit) your folder shows. `git switch` is the newer, clearer command
Merge	Combine the work of two branches into one
Merge conflict	Both branches changed the same lines, so Git can't decide which version to keep and asks you to choose
Rebase	Move your branch so it starts from a newer point, by replaying your commits on top of the latest `main`
Stash	A temporary shelf where you can put unfinished changes and take them back later
Tag	A permanent name for one specific commit, usually a version number such as `v1.0.0`
Diff	A view of what changed between two versions: added lines marked `+`, removed lines marked `-`
`.gitignore`	A file listing things Git should not track, such as `node_modules/` or `.env`
Reflog	Git's private diary of everywhere HEAD has been on your computer. Used to rescue "lost" commits

Working with others

Term	Meaning
Remote	A copy of the repository somewhere else, usually on GitHub
origin	The usual name for the remote you cloned from
upstream	Two meanings. (1) The original repository you forked from. (2) The GitHub branch your local branch is linked to
Clone	Download a full copy of a repository, including all of its history
Fetch	Download new commits from GitHub without changing your files or branches
Pull	Fetch, then merge (or rebase) the new commits into the branch you're on
Push	Upload your commits to GitHub
Remote-tracking branch	Names like `origin/main`: your local record of where a branch was on GitHub the last time you fetched
Force push	Replace the branch on GitHub with your version, even if that throws commits away. Use `--force-with-lease`, the safer form
Fork	Your own copy, on GitHub, of someone else's repository
Collaborator	A person given access to a repository
Organization	A shared GitHub account that belongs to a group instead of one person

GitHub features

Term	Meaning
Pull request (PR)	A request to merge one branch into another. It has a page for discussion, code review and automatic checks
Draft pull request	A PR marked "not ready for review yet"
Review / approval	A teammate reads the changes, comments, and then approves or asks for changes
Squash merge	Merging a pull request as one single combined commit
Issue	A task, bug report or idea tracked on GitHub
Label, milestone	A colored tag on an issue; a group of issues with a deadline
Project (board)	A board or table of issues and PRs, with columns such as Todo, In progress, Done
Ruleset / branch protection	Rules GitHub enforces, such as "nobody may push straight to main"
CODEOWNERS	A file that says who must review changes to which folders
Release	A published version of the project with notes and downloadable files

Automation and security

Term	Meaning
Terminal, shell, CLI	The text window where you type commands; the program that reads them; a tool you use by typing commands, like `git` or `gh`
CI (continuous integration)	Building and testing the project automatically on every push and pull request
GitHub Actions	GitHub's automation service. It runs your CI
Workflow	A YAML file in `.github/workflows/` that tells GitHub Actions what to run and when
YAML	A text format made of `key: value` lines, where indentation (spaces at the start of a line) shows structure
Runner	The virtual machine that runs a workflow
Status check	The pass or fail result a workflow shows on a pull request
Secret	A password or API key stored in GitHub, hidden from logs and not visible in the code
SSH key	A matching pair of files that proves who you are without a password. The private key stays on your computer; the public key goes to GitHub
Token (PAT)	A long password-like string that scripts and tools use to access GitHub
2FA	Two-factor authentication: a second proof of identity, such as an app code or passkey, on top of your password
Dependency	An outside library your project uses, such as React or Flask
Lock file	A file such as `package-lock.json` that records the exact versions of all dependencies
Dependabot	A GitHub bot that opens pull requests to update your dependencies
Hook	A script Git runs automatically at a certain moment, for example just before a commit
Linter, formatter	A linter checks code for likely mistakes; a formatter fixes layout (spacing, quotes) automatically

More of GitHub

Term	Meaning
README	The `README.md` file shown on a repository's front page: what the project is and how to use it
Markdown	The simple text formatting used in README files, issues and comments (`#` headings, `**bold**`, `-` lists)
Star	A bookmark on a repository, which also shows appreciation
Watch	Your notification setting for a repository
Notification	A message telling you something on GitHub involves you, such as a review request or a mention
Mention	Writing `@username` to notify someone
Profile	Your public page at `github.com/your-name`, with pinned repositories and a contribution graph
Discussion	A forum thread inside a repository, for questions and ideas rather than tasks
Wiki	Documentation pages attached to a repository
Gist	A small shared snippet of one or a few files, with its own link and history
Package	Published software, such as a Docker image or npm library, hosted on GitHub Packages
Template repository	A repository others can copy to start a new project with the same files
Archive	Make a repository read-only once a project is finished
Insights	A repository's statistics: contributors, activity, traffic
github.dev	A VS Code editor in the browser, opened by pressing `.` on a repository
GitHub Desktop	GitHub's app for using Git with buttons instead of commands
Copilot	GitHub's AI coding assistant
API	A way for programs to talk to GitHub, instead of a person using the website
Webhook	A message GitHub sends to another service when something happens, such as a new pull request
GitHub App	A service connected to your repositories with specific permissions, such as a hosting provider or a chat bot

PART I

GitHub from zero

What Git and GitHub are, your first repository, the everyday commands, the website, and Markdown.

---

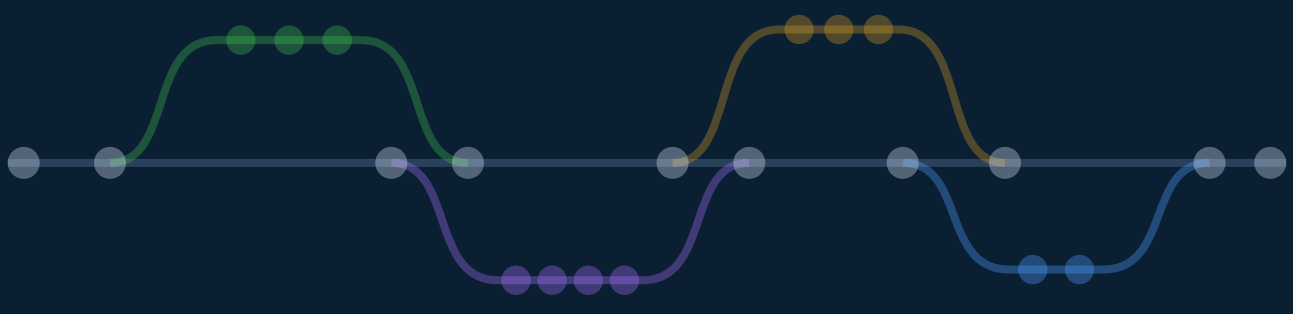
1 What Git and GitHub are

2 Your first repository

3 Everyday Git commands

4 A tour of GitHub.com

5 Writing on GitHub: Markdown



CHAPTER 1

What Git and GitHub are

The problem they solve, and how the two fit together.

Picture a group project without them. Files fly around in chat as `final.zip`, `final_v2.zip` and `final_v2_REAL_fixed.zip`. Two people edit the same file and one person's work silently disappears. Something breaks and nobody knows which change did it, or how to get yesterday's working version back. Git and GitHub were built to make all of that go away.

AT A GLANCE **Version control (Git)**	
What is it?	A program that records the history of a project folder as a series of snapshots called commits, so you can see every change, who made it and why, and go back to any earlier version.
Why use it?	You never lose work, you can experiment without fear, and several people can change the same project at the same time and combine their work safely.
How does it work?	Git keeps a hidden `.git` folder inside your project. Every time you commit, it saves a snapshot of your files there, together with your name, the time and a message. Branches let different lines of work grow side by side, and merging combines them.
When to use it?	For any project made of text files that changes over time: code, websites, documentation, even a thesis written in Markdown. As soon as a project matters to you, put it in Git.
Example	Alice breaks the login page while trying something new. Instead of trying to remember what changed, Alice runs one Git command and the folder is back to exactly how it was at the last commit, an hour ago.

AT A GLANCE **GitHub**	
What is it?	A website and service that stores Git repositories online and adds tools for working together: pull requests, code review, issues, project boards, automation and hosting.
Why use it?	Git on its own lives on one computer. GitHub gives the whole team one shared copy, a place to discuss and review changes, and robots that test and publish your work.
How does it work?	You "push" your commits from your computer to a repository on GitHub, and your friends "pull" them onto theirs. Everything else (pull requests, issues, Actions) is built around that shared repository.
When to use it?	Whenever more than one person works on a project, when you want a backup in the cloud, or when you want to show your work in a portfolio.
Example	Bob, Alice and Chen each work on their own laptops. Each evening they push to the team repository on GitHub, review each other's pull requests, and the website updates itself automatically from `main`.

> **In plain English**
>
> Git is like the "version history" of a shared document, but for a whole folder, and you decide when each version is saved. GitHub is like the shared drive where everyone's copies meet, with a review desk and a few helpful robots attached.

Git and GitHub side by side

	Git	GitHub
What it is	A program you install	A website and online service
Where it runs	On your computer	In the cloud, at github.com
Works offline?	Yes, almost everything	No
Main job	Recording history, branching, merging	Sharing, reviewing, planning, automating
You use it through	The terminal, VS Code, GitHub Desktop	Your browser, the `gh` command, the mobile app
Alternatives	(Git is the standard)	GitLab, Bitbucket, Gitea

The big picture: how a team uses them

1. Each person has a copy (a "clone") of the repository on their own computer.
2. They work on a separate branch, saving progress in small commits.
3. They push the branch to GitHub and open a pull request.
4. GitHub runs the automatic tests, and a teammate reviews the change.
5. The pull request is merged into `main`, the shared, working version.
6. Everyone pulls the new `main` and starts the next piece of work.

The rest of this part teaches the basics of each step. Parts II to IV then show how to do them well as a team.

Five words to learn first

- **Repository (repo):** a project folder tracked by Git, with its full history.
- **Commit:** one saved snapshot, with a message describing the change.
- **Branch:** a separate line of work, so you can change things without disturbing `main`.
- **Remote:** a copy of the repository somewhere else, usually on GitHub. The usual name for it is `origin`.
- **Pull request:** a request on GitHub to merge one branch into another, with review and tests.

Every other term is in **Words you need to know** at the front of this guide.

CHAPTER 2

Your first repository

An account, Git installed, and the full cycle from first commit to first push.

This chapter takes you from nothing to a repository on GitHub that you've changed from your own computer. Follow it once from top to bottom; it takes about 45 minutes.

1. Create a GitHub account

1. Go to github.com/signup. Enter your email, a password, a **username** and your country or region (or continue with a Google or Apple account and choose a username).
2. Choose the username carefully: it appears in every URL (`github.com/your-name`) and on everything you contribute. Short, professional and easy to spell is best.
3. Confirm your email address with the code GitHub sends you.
4. Turn on **two-factor authentication**: your profile picture → **Settings** → **Password and authentication** → **Enable two-factor authentication**. Use an authenticator app or a passkey. GitHub requires it for accounts that contribute code, and it protects your work if your password ever leaks.

2. Install Git

```
winget install --id Git.Git -e
```

WHAT EACH LINE DOES

`winget install --id Git.Git -e`	On Windows: installs Git for Windows, which also includes Git Bash and Git Credential Manager. `-e` means "exactly this package". Alternatively download the installer from git-scm.com and accept the defaults, choosing **Visual Studio Code** as Git's default editor if you use it.

On macOS, open Terminal and run `xcode-select --install` (or `brew install git` if you use Homebrew). On Ubuntu or Debian Linux, run `sudo apt install git`. Then check that it worked:

```
git --version
```

WHAT EACH LINE DOES

`git --version`	Prints the installed version, for example `git version 2.55.0`. If you get "command not found", close and reopen the terminal, or restart the computer.

A good editor makes everything easier. Visual Studio Code is free, runs on every system and has Git built in (Chapter 32).

3. Tell Git who you are

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

WHAT EACH LINE DOES

`user.name` / `user.email`	Written into every commit you make. Use the email of your GitHub account, so your commits are linked to your profile (Chapter 7 shows how to keep your real email private).
`init.defaultBranch main`	New repositories start with a branch called `main`.

4. Create a repository on GitHub

Click the **+** at the top right of GitHub → **New repository**, and fill in the form:

Field	What to choose
Owner	Your account (or an organization, Chapter 8)
Repository name	Short, lowercase, words joined with hyphens: `hello-github`
Description	One sentence about the project. Optional, but it shows on your profile
Public / Private	Public: anyone can see it (good for portfolios and free features). Private: only people you invite
Add a README file	Tick it. It's the project's front page, and it makes the repository ready to clone
Add .gitignore	Pick your language (for example Node or Python) so Git ignores files that don't belong in history
Choose a license	MIT is a common, permissive choice for code you want others to reuse (Chapter 30)

Click **Create repository**. You now have a repository on GitHub with one commit in it.

5. Clone it to your computer

On the repository page, click the green **Code** button, make sure **HTTPS** is selected, and copy the address. Then, in Git Bash or Terminal:

```
cd ~/projects
git clone https://github.com/your-name/hello-github.git
cd hello-github
```

WHAT EACH LINE DOES

`cd ~/projects`	Go to the folder where you keep your projects (create it first with `mkdir ~/projects` if needed).
`git clone https://github.com/...`	Downloads the repository, with its full history, into a new folder called `hello-github`, and remembers GitHub as the remote called `origin`.
`cd hello-github`	Go into the project folder. All Git commands for this project run from here.

AT A GLANCE **Clone**	
What is it?	Downloading a complete copy of a repository, including every commit, from GitHub to your computer.
Why use it?	You need a local copy to edit files, run the project and make commits.
How does it work?	Git downloads all the objects and branches, creates the folder, checks out the default branch, and records the address as the remote `origin`, so later `git pull` and `git push` know where to go.
When to use it?	Once per computer, at the start of working on a repository.
Example	Chen joins the team on Wednesday, runs `git clone` on the team repository, and has the entire project and its history in a minute.

6. Make a change, commit it, push it

Open `README.md` in your editor, add a line such as "My first repository on GitHub", and save the file. Then:

```
git status
git add README.md
git commit -m "docs: describe the project in the README"
git push
```

WHAT EACH LINE DOES

`git status`	Shows `README.md` in red under "Changes not staged for commit": Git noticed the change but hasn't been told to save it.
`git add README.md`	Stages the file: "include this change in the next commit". Run `git status` again and it turns green.
`git commit -m "..."`	Saves the staged change as a commit on your computer, with that message.
`git push`	Uploads the commit to GitHub. The first time, you have to sign in: see the note below.

This is what `git status` looks like before the `git add`:

```
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
```

Signing in the first time you push

On Windows, Git Credential Manager (installed with Git) opens a browser window; sign in to GitHub there and it remembers you. On macOS and Linux, Git asks for a username and password in the terminal, and GitHub doesn't accept your account password there. The easiest fix is to install the GitHub CLI (`brew install gh` on macOS; see cli.github.com for Linux), run `gh auth login`, choose **HTTPS**, and answer **Yes** to "Authenticate Git with your GitHub credentials?". Then run `git push` again. Chapter 7 explains `gh` and the alternative, SSH keys.

Refresh the repository page on GitHub: your new line is there, and the commit message appears next to the file.

The basic cycle

Pull, edit, add, commit, push. Those five steps are most of what anyone does with Git every day. Everything else in this guide makes them safer and smoother for a team.

7. Get changes from GitHub

Changes can also happen on GitHub: a teammate pushes, or you edit a file in the browser. To try it, open `README.md` on GitHub, click the pencil icon, add another line, and click **Commit changes**. Then, on your computer:

```
git pull
```

WHAT EACH LINE DOES

`git pull`	Downloads new commits from GitHub and adds them to your current branch. Your `README.md` now has the line you added on the website.

AT A GLANCE **Push and pull**	
What is it?	`git push` uploads your new commits to GitHub; `git pull` downloads other people's new commits and adds them to your branch.
Why use it?	They keep your computer and the shared repository in sync, which is how a team's work meets.
How does it work?	Push sends the commits GitHub doesn't have yet and moves the branch on GitHub forward. Pull first fetches the new commits, then merges (or rebases) them into your current branch. If GitHub has commits you don't, a push is refused until you pull.
When to use it?	Pull before you start working and before you push; push whenever you've made commits you want to back up or share, at least once a day.
Example	Bob pushes a new contact page at 6 p.m. When Alice sits down at 8 p.m., Alice first runs `git pull`, and the contact page appears in Alice's copy too.

8. Starting from a folder you already have

If you already have a project on your computer that isn't in Git yet:

```
cd ~/projects/my-project
git init
git add .
git commit -m "chore: first commit"
gh repo create my-project --private --source=. --push
```

WHAT EACH LINE DOES

`git init`	Turns the current folder into a Git repository by creating the hidden `.git` folder. Nothing is committed yet.
`git add .`	Stages every file in the folder (`.` means "here and below"). Create a `.gitignore` first (Chapter 3), so you don't commit things like `node_modules`.
`git commit -m "chore: first commit"`	Saves the first snapshot.
`gh repo create my-project --private --source=. --push`	Creates an empty repository on GitHub, links it as `origin`, and pushes your commit. (Install `gh` first; see Chapter 7.)

Without `gh`: create an *empty* repository on GitHub (don't tick README, .gitignore or license), then run `git remote add origin <address>` and `git push -u origin main`. If you tick those boxes, GitHub and your folder start with two unrelated histories (Chapter 35 explains the fix).

CHAPTER 3

Everyday Git commands

The dozen commands you'll use every day, explained with examples.

This chapter explains the commands from the basic cycle in more depth, plus the ones you'll need around them. Later chapters build on every one of them.

git status: where am I?

`git status` is the most useful command in Git, and it never changes anything. Run it before and after everything else.

```
On branch feat/contact-page
Your branch is ahead of 'origin/feat/contact-page' by 1 commit.

Changes to be committed:
        modified:   index.html

Changes not staged for commit:
        modified:   style.css

Untracked files:
        contact.html
```

HOW TO READ IT

`On branch feat/contact-page`	The branch you're working on.
`ahead of ... by 1 commit`	You have one commit that isn't on GitHub yet. Push it. ("behind" means GitHub has commits you don't; pull them.)
`Changes to be committed`	Staged changes (green on screen). They'll be in the next commit.
`Changes not staged for commit`	Changes Git sees but you haven't added (red).
`Untracked files`	New files Git has never tracked. They're ignored until you `git add` them.

git add: choose what goes into the next commit

```
git add index.html
git add src/
git add .
git add -p
```

WHAT EACH LINE DOES

`git add index.html`	Stage one file.
`git add src/`	Stage everything that changed inside the `src` folder.
`git add .`	Stage every change in the current folder and below, including new and deleted files.
`git add -p`	Go through your changes piece by piece and choose what to stage (Chapter 12).

git commit: save a snapshot

```
git commit -m "feat: add contact page"
git commit
```

WHAT EACH LINE DOES

`git commit -m "..."`	Commit with a short message written directly in the command.
`git commit`	Opens your editor so you can write a longer message: a short first line, a blank line, then an explanation (Chapter 12).

A good message completes the sentence "If applied, this commit will...": "add contact page", "fix crash when the cart is empty". Avoid messages like "update" or "stuff".

git log and git diff: look at history and changes

```
git log
git log --oneline -10
git diff
git diff --staged
```

WHAT EACH LINE DOES

`git log`	Lists commits, newest first: ID, author, date and message. Press `q` to leave the list, space for the next page.
`git log --oneline -10`	The last ten commits, one line each.
`git diff`	Shows the exact lines you changed but haven't staged: `-` lines were removed, `+` lines added.
`git diff --staged`	Shows what's staged, so what the next commit will contain.

Undo the simple things

```
git restore style.css
git restore --staged index.html
git rm old-page.html
git mv about.html about-us.html
```

WHAT EACH LINE DOES

`git restore style.css`	Throws away your unsaved changes to `style.css`, back to the last commit. This can't be undone.
`git restore --staged index.html`	Unstages the file; your edits stay in the file.
`git rm old-page.html`	Deletes the file and stages the deletion, ready to commit.
`git mv about.html about-us.html`	Renames the file and stages the rename.

Chapter 19 covers every other kind of undo, including undoing commits.

Branches in practice

```
git branch
git switch -c feat/contact-page
git switch main
git branch -d feat/contact-page
```

WHAT EACH LINE DOES

`git branch`	Lists your branches. The one marked with `*` is the one you're on.
`git switch -c feat/contact-page`	Creates a new branch starting from where you are and switches to it (`-c` means create).
`git switch main`	Switches back to `main`. Your files change to match that branch.
`git branch -d feat/contact-page`	Deletes a branch you've finished with (after it was merged).

AT A GLANCE **Branch (in practice)**	
What is it?	A separate line of work inside the same repository, with its own name.
Why use it?	You can build or try something without disturbing the working version on `main`, and several people can work at once without getting in each other's way.
How does it work?	Creating a branch is instant: Git just makes a new label pointing at the current commit. Commits you make on it move only that label. Switching branches changes your files to match (Chapter 6 shows the details).
When to use it?	For every feature, fix or experiment, even a small one. Never work directly on `main` in a team.
Example	Chen wants to try a new color scheme but isn't sure it'll look good. Chen does it on `experiment/colors`; if the team doesn't like it, the branch is deleted and `main` was never touched.

Merging a branch

When the work on a branch is done, you bring it into `main`. In a team you do this through a pull request on GitHub (below). On your own, you can merge locally:

```
git switch main
git pull
git merge feat/contact-page
git push
```

WHAT EACH LINE DOES

`git switch main` and `git pull`	Go to `main` and make sure it's up to date.
`git merge feat/contact-page`	Brings the branch's commits into `main`. If `main` hasn't changed since the branch started, Git simply moves `main` forward (a "fast-forward"); otherwise it creates a merge commit.
`git push`	Uploads the updated `main`.

AT A GLANCE **Merge**	
What is it?	Combining the changes from one branch into another.
Why use it?	It's how finished work on a branch becomes part of the main project.
How does it work?	Git finds the last commit both branches share, works out what each side changed since then, and combines both sets of changes. If they changed the same lines, Git stops and asks you to choose (a conflict, Chapter 15).
When to use it?	When a branch's work is finished and reviewed. In a team, let GitHub do it through a pull request.
Example	Alice finished the contact page on `feat/contact-page`. After Bob approves the pull request, Alice clicks **Merge**, and the page is part of `main`.

.gitignore: keep junk out of the repository

`.gitignore`

```
# Dependencies anyone can reinstall
node_modules/
.venv/

# Build output
dist/

# Secrets and local settings
.env

# Logs and system files
*.log
.DS_Store
```

HOW TO READ IT

`node_modules/`	A name ending in `/` matches a folder, wherever it is.
`*.log`	`*` matches anything, so this ignores every file ending in `.log`.
`.env`	An exact file name. `.env` files hold passwords and must never be committed.
`# ...`	Comment lines.

AT A GLANCE **.gitignore**	
What is it?	A file listing the files and folders Git should pretend don't exist.
Why use it?	Repositories stay small and clean, and secrets, downloaded libraries and machine-specific files never end up in history.
How does it work?	Before showing untracked files or adding new ones, Git checks each path against the patterns in `.gitignore` files. Matching files are skipped. Files that are already tracked aren't affected.
When to use it?	Create it before your first commit, and add to it whenever `git status` shows a file that doesn't belong in the repository.
Example	After running the app, Bob's `git status` shows 10,000 files in `node_modules/`. Adding `node_modules/` to `.gitignore` makes them disappear from the list, and they're never committed.

GitHub keeps ready-made `.gitignore` templates for almost every language at github.com/github/gitignore.

Your first branch and pull request, start to finish

This walkthrough uses everything above, plus GitHub's pull request button. Try it in your `hello-github` repository.

```
git switch main
git pull
git switch -c feat/about-page
git add about.html
git commit -m "feat: add about page"
git push -u origin feat/about-page
```

WHAT EACH LINE DOES

`git switch main` and `git pull`	Start from the newest `main`.
`git switch -c feat/about-page`	Create and switch to a branch for the new page. Create `about.html` in your editor.
`git add about.html` and `git commit -m "..."`	Stage and commit the new file.
`git push -u origin feat/about-page`	Upload the branch to GitHub. `-u` links your local branch with the GitHub one, so later a plain `git push` is enough.

Now open the repository on GitHub. A yellow banner says **feat/about-page had recent pushes**, with a **Compare & pull request** button:

1. Click it. Check that **base** is `main` and **compare** is `feat/about-page`.
2. Write a title ("Add about page") and a short description, then click **Create pull request**.
3. Look at the **Files changed** tab to see your changes as others will.
4. Click **Merge pull request**, then **Confirm merge**, then **Delete branch**.
5. Back in the terminal, run `git switch main`, `git pull` and `git branch -d feat/about-page`.

That's the core of team work on GitHub. Chapter 11 turns it into a routine for a whole team.

Which command when?

You want to	Run
See what's going on	`git status`
Get a project onto your computer	`git clone <address>`
Get your friends' latest work	`git pull`
Start new work	`git switch -c <branch-name>`
Save your progress	`git add <files>`, then `git commit -m "..."`
Share your work	`git push`
See what you changed	`git diff`
See the history	`git log --oneline`
Throw away changes to a file	`git restore <file>`
Go back to another branch	`git switch <branch-name>`

CHAPTER 4

A tour of GitHub.com

Every part of a repository page, and what you can do without a terminal.

Much of GitHub can be used entirely in the browser. This chapter walks through the repository page and the most useful things you can do there.

The repository page

At the top you'll see the owner and name (`my-org/my-app`), a badge showing Public or Private, and three buttons: **Watch**, **Fork** and **Star** (explained below). Under them is a row of tabs:

Tab	What it's for
Code	The files, branches, commits, README, releases and the green Code button
Issues	The task list: bugs, features and ideas (Chapter 22)
Pull requests	Proposed changes waiting for review and merging (Chapter 16)
Discussions	A forum for questions and ideas, if turned on (Chapter 31)
Actions	Automatic workflows and their logs (Chapter 23)
Projects	Boards and tables for planning (Chapter 22)
Wiki	Documentation pages, if turned on (Chapter 31)
Security	Security policy, Dependabot and scanning alerts (Chapter 26)
Insights	Statistics: contributors, traffic, commit activity (Chapter 30)
Settings	Repository settings. Only visible to people with Admin access

The Code tab

- **Branch selector** (top left of the file list): view the files on any branch or tag.
- **Go to file:** search for a file by name. Pressing `t` does the same.
- **Add file:** create a new file, or upload files from your computer.
- **The green Code button:** the addresses to clone with (HTTPS, SSH, GitHub CLI), **Open with GitHub Desktop**, **Download ZIP** (a copy of the files without history) and a **Codespaces** tab (Chapter 28).
- **The latest commit** line above the files, and a clock icon with the number of commits, which opens the full history.
- **The README** is shown below the file list. That's why every repository should have a good one (Chapter 5).
- **The About box** on the right: description, website link and topics (tags that help people find the repository), followed by Releases, Packages, Contributors and Languages.

Viewing a file

Click any file to open it. The bar above its contents has:

- **Code / Blame:** switch to Blame to see who last changed each line (Chapter 21).
- **Raw:** the plain file, useful for downloading or linking.
- **Copy** and **Download** icons.
- **The pencil:** edit the file in the browser.
- **History:** every commit that changed this file.

Click a line number to highlight that line; shift-click another to highlight a range. The page address then ends in something like `#L10-L20`, which you can paste into a chat or issue to point at those exact lines. Press `y` first to turn the address into a permanent link to this exact version.

Editing and uploading files in the browser

1. Open the file and click the pencil (or **Add file** → **Create new file**).
2. Make your change. The **Preview** tab shows how Markdown will look.
3. Click **Commit changes**. Write a commit message and choose either **Commit directly to the main branch** or **Create a new branch for this commit and start a pull request**. With a ruleset protecting `main` (Chapter 9), only the second option works, which is what you want anyway.

To add images or other files, use **Add file** → **Upload files** and drag them in. Files uploaded through the browser can be up to 25 MB each.

> **Small fixes from your phone**
>
> Browser editing is perfect for typo fixes in documentation, even from a phone: edit, choose "Create a new branch", and a pull request is ready for review in under a minute.

github.dev: a full editor in the browser

Press `.` (the full stop key) on any repository page, or change `github.com` to `github.dev` in the address. A web version of VS Code opens with the whole repository.

AT A GLANCE **github.dev**	
What is it?	A lightweight VS Code editor that runs entirely in your browser, opened straight from a repository.
Why use it?	You can edit many files at once, search the whole project and commit, without installing anything or cloning.
How does it work?	The editor loads the files directly from GitHub. Its Source Control panel stages and commits your changes back to GitHub, on the current branch or a new one. It has no terminal, so you can't run or test code there; for that, use a codespace (Chapter 28).
When to use it?	For documentation changes, quick edits across several files, and reviewing code comfortably, especially on a computer that isn't yours.
Example	At the library, Bob notices five pages with outdated links. Bob presses `.`, fixes all five in github.dev, and commits them on a new branch with a pull request.

Star, Watch and Fork

AT A GLANCE **Stars, watching and forks**	
What is it?	Three buttons at the top of every repository. **Star** bookmarks it and shows appreciation. **Watch** chooses which notifications you get from it. **Fork** creates your own copy of it.
Why use it?	Stars help you find repositories again and help good projects get noticed. Watching keeps you informed about projects you care about. Forking lets you change a project you can't push to.
How does it work?	Your starred repositories are listed on your profile (and you can sort them into lists). Watch has levels: **Participating and @mentions** (only threads you're involved in), **All Activity** (everything), **Ignore** and **Custom** (for example only releases). The button shows your current level. Forking is explained in Chapter 18.
When to use it?	Star tools and examples you want to remember; watch your team's repository with "All Activity" or "Custom"; fork when you want to contribute to someone else's project.
Example	Chen stars a useful date library to find it later, sets the team repository to Custom so pull requests and releases show up in notifications, and forks an open-source theme to fix a bug in it.

History, comparisons and branches

- **Commits:** the clock icon on the Code tab lists every commit on the current branch. Click one to see exactly what it changed.
- **Compare:** go to `github.com/my-org/my-app/compare` and pick two branches, tags or commits to see every difference between them, for example `main...feat/about-page`.
- **Branches:** the branch count next to the branch selector opens a page listing your branches, active ones and stale ones (no commits in three months). You can delete branches there, and restore a branch deleted from a pull request.

Keyboard shortcuts and search

Key	What it does
`?`	Show all keyboard shortcuts for the current page
`/` or `s`	Jump to the search box
`t`	Find a file by name in this repository
`.`	Open the repository in github.dev
`y`	Turn the address into a permanent link to this exact version
`g` then `c`	Go to the Code tab
`g` then `i`	Go to Issues
`g` then `p`	Go to Pull requests
`Ctrl+K` (`Cmd+K` on Mac)	Open the command palette: jump to repositories, files and actions by typing

The search box at the top searches all of GitHub, or narrows down with filters:

Search	Finds
`repo:my-org/my-app login`	"login" in one repository's code
`org:my-org TAX_RATE`	Code in every repository of the organization
`language:python path:tests/`	Python code inside `tests` folders
`is:issue is:open label:bug`	Open issues labelled bug
`is:pr review-requested:@me`	Pull requests waiting for your review
`stars:>1000 topic:react`	Popular repositories about React

Your home page (github.com when you're signed in) shows a feed of activity from repositories and people you follow, and the bell icon opens your notifications (Chapter 29).

CHAPTER 5

Writing on GitHub: Markdown

READMEs, issues, pull requests and comments all use the same simple formatting.

AT A GLANCE **Markdown**	
What is it?	A simple way to format text with ordinary characters: `#` for headings, `**` for bold, `-` for lists. GitHub turns it into nicely formatted text.
Why use it?	It's quick to write, easy to read even unformatted, and works everywhere on GitHub: README files, issues, pull requests, comments, wikis and discussions.
How does it work?	Files ending in `.md` and every comment box on GitHub are converted from Markdown to formatted text when shown. GitHub uses an extended version called GitHub Flavored Markdown, which adds tables, task lists, mentions and more.
When to use it?	Every time you write anything on GitHub, and for documentation files in your repository.
Example	Alice writes the project's README with a heading, a screenshot, a numbered list of setup steps and a code block of commands. On GitHub it appears as a polished front page for the project.

The basics

You type	You get
`# Heading`, `## Smaller heading`	Headings, from biggest (`#`) to smallest (`######`)
`**bold**`, `*italic*`, `~~strikethrough~~`	**bold**, *italic*, crossed-out text
Text wrapped in backticks, like `git status`	Code inside a sentence, in a monospaced font
`[link text](https://example.com)`	A link
`![description](screenshot.png)`	An image. The description is shown if the image can't load and read by screen readers
`- item` or `* item`	A bullet list. Indent with two spaces for a sub-list
`1. item`	A numbered list
`- [ ] task` and `- [x] done`	A checklist. The boxes are clickable in issues, pull requests and comments
`> quoted text`	A quotation
`---`	A horizontal line
An empty line	A new paragraph. A single line break is ignored

Code blocks

Put code between two lines of three backticks. Write the language after the first three backticks to get colors:

````
```js
function greet(name) {
  return `Hello, ${name}!`;
}
```
````

Tables

```
| Command      | What it does          |
| ------------ | --------------------- |
| `git status` | Shows what changed    |
| `git push`   | Uploads your commits  |
```

The second line, made of dashes, separates the header from the rows. The columns don't need to line up; it just looks tidier.

GitHub-only extras

You type	What happens
`@alice`	Mentions a person, who gets a notification
`@my-org/backend`	Mentions a whole team
`#42`	Links to issue or pull request 42 in this repository
`my-org/api#7`	Links to issue 7 in another repository
A commit ID like `4e1d2c7`	Turns into a link to that commit
`:tada:` `:bug:` `:rocket:`	Emoji. Type `:` and a word to search
`> [!NOTE]` on the first line of a quote	A colored note box. Also `[!TIP]`, `[!IMPORTANT]`, `[!WARNING]` and `[!CAUTION]`
`<details><summary>Title</summary> ... </details>`	A collapsible section, good for long logs
A code block with the language `mermaid`	A diagram (flowcharts, sequences) drawn from text
`$E = mc^2$`	A math formula

An alert box and a diagram look like this in Markdown:

````
> [!WARNING]
> Run `npm install` again after pulling, the dependencies changed.

```mermaid
flowchart LR
  A[Branch] --> B[Pull request] --> C[Review] --> D[Merge]
```
````

You can also paste or drag images and short videos straight into any comment box on GitHub; it uploads them and inserts the Markdown for you.

Write a README people actually read

The README is the first thing anyone sees. A simple structure that works for almost any project:

```
# Project name

One or two sentences: what it is and who it's for.

![Screenshot of the app](docs/screenshot.png)

## Getting started

1. `git clone https://github.com/my-org/my-app.git`
2. `cd my-app && npm install`
3. `cp .env.example .env`
4. `npm run dev`, then open http://localhost:5173

## Running the tests

`npm test`

## Project structure

- `src/` application code
- `tests/` automated tests
- `docs/` documentation and screenshots

## Team

Made by @alice, @bob and @chen. See CONTRIBUTING.md to help.
```

Images and links can use paths relative to the README, like `docs/screenshot.png`. GitHub resolves them inside the repository, on whichever branch is being viewed.

Your profile README

Create a public repository with exactly the same name as your username (for example `alice/alice`) and add a `README.md`. GitHub shows that README at the top of your profile page. Use it to introduce yourself, list what you're learning and link your best projects. Chapter 29 covers the rest of your profile.

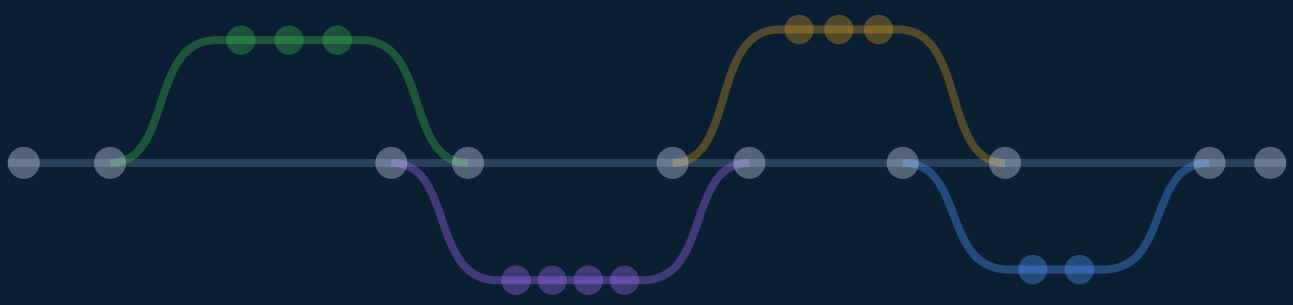
PART II

Foundations for team work

The model behind the commands, and a computer set up for team work.

6 How Git really works

7 Set up your computer for team work



CHAPTER 6

How Git really works

Commits, trees, refs and HEAD: the model that makes advanced commands predictable.

Most Git confusion comes from one wrong picture: thinking of a branch as a folder, or as a copy of the code. A branch is a tiny file that holds one commit ID. Once you see Git as a chain of snapshots with movable labels stuck on it, commands like rebase, reset and reflog stop being scary, because you can predict what they'll do.

Commits are snapshots, not diffs

AT A GLANCE **Commit**	
What is it?	A saved snapshot of every file in your project at one moment, together with a message, the author, the time, and a link to the commit that came before it.
Why use it?	Commits are your project's history and its undo button, and they're the unit you share with your team. Branches, merges, reverts and pull requests all work by pointing at commits or combining them.
How does it work?	When you commit, Git stores the contents of each changed file once, writes a "tree" that lists all the files, and then writes a commit object that points at that tree and at the parent commit. The commit's ID is a hash calculated from all of that, so a commit can never change without getting a new ID.
When to use it?	Every time you reach a small step that works: a function written, a bug fixed, a typo corrected. Several commits an hour is normal.
Example	Alice adds a "Forgot password?" link, checks that the page still loads, and commits `feat: add forgot password link` before starting on the email part.

> **In plain English**
>
> Think of a photo album. Every commit is a photo of your entire project at one moment, with a caption (the commit message) and a note saying which photo came before it. Git never edits a photo; it only adds new ones.

Inside the hidden `.git` folder, Git keeps four kinds of objects. Each one is named by a hash, a code calculated from its contents:

Object	What it holds
blob	The contents of one file. No name, no permissions, just the bytes
tree	A folder listing: file names pointing to blobs, and subfolder names pointing to other trees
commit	One root tree (the full snapshot), the parent commit or commits, the author, the date and the message
annotated tag	A named pointer to a commit, with its own message, author and optional signature

Every commit records the whole project as it was at that moment. When you ask for a diff, Git compares two snapshots and works out the differences on the spot. Files that didn't change are shared between commits, so storing full snapshots costs very little space.

A commit's hash is calculated from everything in it, including its parent's hash. Change anything in an old commit and its hash changes, which changes the next commit's parent, and so on up to the newest commit. That's why "rewriting history" always creates new commits instead of editing old ones. It's also why rewriting commits that your friends have already downloaded causes trouble: their copies still point at the old hashes.

You can look inside any of this. Run these in any repository:

```
git cat-file -t HEAD
git cat-file -p HEAD
git cat-file -p "HEAD^{tree}"
git rev-parse HEAD
```

WHAT EACH LINE DOES

`git cat-file -t HEAD`	Prints the type of the object HEAD points to. You'll see `commit`.
`git cat-file -p HEAD`	Prints that commit in raw form: the `tree` hash, the `parent` hash, `author`, `committer`, then the message.
`git cat-file -p "HEAD^{tree}"`	Prints the root folder listing of that commit: one line per file or subfolder, each with its own hash.
`git rev-parse HEAD`	Prints the full 40-character hash of the commit you're on.

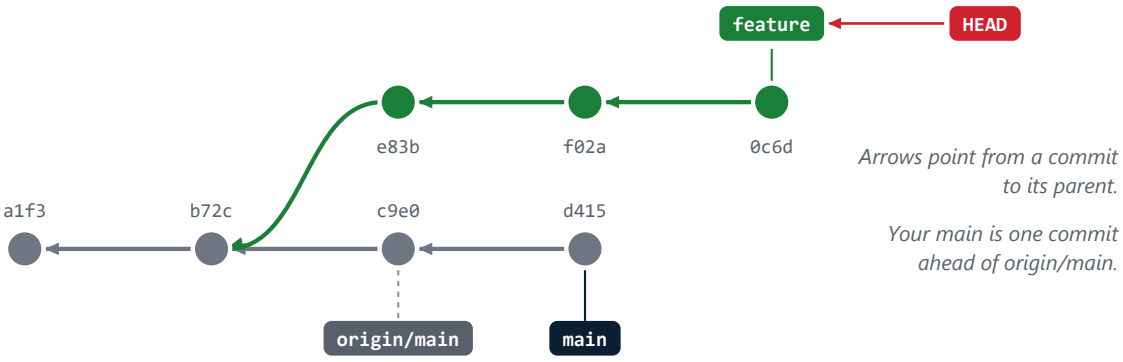
The output of the second command looks like this:

```
tree 9c1f2e7d3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d
parent 2c1b0a98e7d6c5b4a3f2e1d0c9b8a7f6e5d4c3b2
author Alice Khan <1234567+alice@users.noreply.github.com> 1760430000 +0600
committer Alice Khan <1234567+alice@users.noreply.github.com> 1760430000 +0600

feat: add login form markup
```

Branches, HEAD and remote-tracking branches

AT A GLANCE **Branch, HEAD and origin/main**	
What is it?	A branch is a named pointer to one commit. HEAD is a pointer to the branch you're working on. `origin/main` is your computer's remembered copy of where `main` was on GitHub.
Why use it?	Branches let several people, or several features, move forward separately without touching `main` until the work is ready. `origin/main` lets Git compare your work with GitHub's without being online all the time.
How does it work?	A branch is a tiny file inside `.git/refs/heads/` that contains a commit ID. Committing creates a new commit whose parent is the current one and rewrites that file with the new ID. Switching branches changes HEAD and updates your files to match. `git fetch` updates the `origin/` pointers.
When to use it?	Create a branch for every new piece of work, however small. Look at `origin/main` when you want to know what GitHub had the last time you fetched.
Example	Bob is halfway through a new navbar on `feat/12-navbar` when someone reports a typo. Bob creates `fix/15-typo` from `main`, fixes it, and the unfinished navbar is never shipped by accident.



Commits point back to their parents. Branches, origin/main and HEAD are labels that point at commits.

- **A branch is a pointer to one commit.** When you commit on a branch, Git creates the new commit and moves the branch label forward to it. Nothing else changes.
- **HEAD says where you are.** Normally it points at a branch name. When it points straight at a commit instead, you're in "detached HEAD" state, and new commits won't belong to any branch (Chapter 35 explains how to get out of it).
- `origin/main` **is a remote-tracking branch.** It's your computer's record of where `main` was on GitHub the last time your Git talked to GitHub. It moves only when you run `fetch`, `pull` or `push`. It is not live.
- **The history is a graph.** Each commit points to its parent. A merge commit has two parents. Commands like `log`, `rebase` and `merge` are all ways of walking or reshaping this graph.

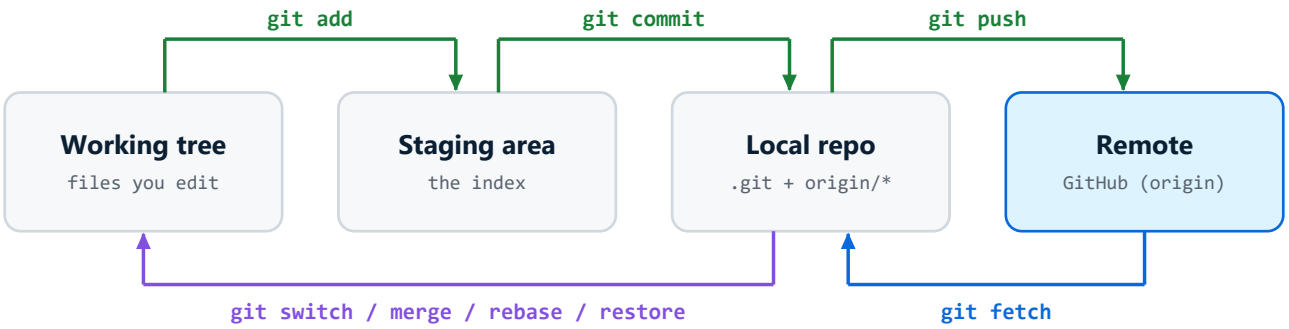
In the diagram, `main` is one commit ahead of `origin/main`: someone committed on `main` locally and hasn't pushed yet. The branch `feature` split off from commit `b72c` and has three commits of its own. HEAD points at `feature`, so that's the branch you're working on.

> **In plain English**
>
> Branches are sticky notes on photos in the album. Making a commit adds a new photo and moves your sticky note onto it. Deleting a branch peels off the sticky note; the photos stay.

That third bullet explains a classic surprise. `git status` says "Your branch is up to date with 'origin/main'" even though a friend pushed five minutes ago. Git compared your branch against its stale record of `origin/main`. Run `git fetch` first and the message changes.

The four places your work lives

AT A GLANCE **Staging area (the index)**	
What is it?	A preparation area between your files and the repository, where you collect exactly what your next commit will contain.
Why use it?	You rarely want to commit everything you changed at once. The staging area lets you turn an afternoon of mixed edits into several clean commits, each about one thing.
How does it work?	`git add` copies the current version of a file (or only chosen parts of it, with `-p`) into a file called the index inside `.git`. `git commit` turns whatever is in the index into a snapshot. Anything you didn't add stays as it was in the previous commit.
When to use it?	Whenever you've changed several things and only some of them belong in the next commit.
Example	Chen fixed a bug in `cart.js` and also tidied `header.css`. Staging only `cart.js` gives the commit `fix: cart total`; staging `header.css` afterwards gives a second commit, `style: tidy header`.



git pull = git fetch, then git merge (or git rebase)

Most Git commands move data between two of these four places.

WHAT MOVES YOUR WORK BETWEEN THEM

`git add`	Copies changes from your working tree into the staging area, so they'll be part of the next commit.
`git commit`	Turns whatever is in the staging area into a new commit in your local repository.
`git push`	Uploads your new commits to GitHub and moves the branch there.
`git fetch`	Downloads new commits from GitHub and updates `origin/*`. It never touches your files or your branches, so it's always safe.
`git switch`, `git merge`, `git rebase`, `git restore`	Change the files in your working tree to match commits in your local repository.
`git pull`	Shortcut for `git fetch` followed by `git merge` (or `git rebase`, with the setting from Chapter 7).

Naming commits: revision syntax

Any command that needs a commit accepts a hash, a branch name, or one of these shortcuts. You'll see them all through the guide.

Syntax	Means	Example
`HEAD`	The commit you're on	`git show HEAD`
`HEAD~3`	Three commits back from HEAD	`git reset --soft HEAD~3`
`HEAD^2`	The second parent of a merge commit	`git show HEAD^2`
`main..feature`	Commits on feature that aren't on main	`git log main..feature`
`main...feature`	In `git diff`: what feature changed since it split from main	`git diff main...feature`
`@{u}`	The GitHub branch the current branch is linked to, such as `origin/feature`	`git log @{u}..`
`@{-1}` or `-`	The branch you were on before this one	`git switch -`
`main@{2.days.ago}`	Where your main pointed two days ago (from the reflog)	`git diff main@{2.days.ago}`
`:/fix login`	Newest commit whose message contains "fix login"	`git show ":/fix login"`
`<sha>:<path>`	A file as it was in a given commit	`git show HEAD~5:src/app.js`

Short hashes are fine

You never need all 40 characters. The first 7 to 10 are enough, and `git log --oneline` prints exactly that.

Watch the graph while you learn

```
git log --oneline --graph --decorate --all
```

WHAT EACH OPTION DOES

`git log`	Shows commit history, newest first.
`--oneline`	One line per commit: short hash and the first line of the message.
`--graph`	Draws the branch structure on the left with `*`, `|` and `/` characters.
`--decorate`	Shows which branch names, tags and HEAD point at each commit.
`--all`	Includes every branch, not just the one you're on.

Typing that every time is tedious, so make a shortcut (an alias): `git config --global alias.lg "log --oneline --graph --decorate --all"`. From then on, `git lg` does the same thing. Run it before and after every rebase, merge or reset while you're learning. It shows exactly what the command did to the graph. VS Code's Source Control Graph and the GitLens extension show the same thing visually.

CHAPTER 7

Set up your computer for team work

Identity, SSH, signed commits, a tuned Git config and the GitHub CLI.

Everyone on the team should do this chapter once, on every computer they use. It takes about 30 minutes and prevents a whole category of problems later.

> **What --global means**
>
> `git config --global` saves a setting for every repository on your computer (in a file called `~/.gitconfig`). Without `--global`, the setting applies only to the repository you're inside. You can see all your settings with `git config --list --show-origin`.

Identity

If you followed Chapter 2, you've already set your name and email. Run the second command again only if you want to switch to the private no-reply address described below.

```
git config --global user.name "Your Name"
git config --global user.email "1234567+yourname@users.noreply.github.com"
```

WHAT EACH LINE DOES

`git config --global user.name "Your Name"`	Sets the name written into every commit you make. Use the name your friends know you by.
`git config --global user.email "..."`	Sets the email written into every commit. GitHub uses it to link commits to your account and show your avatar.

Your email is stored in every commit, and anyone can read it in a public repository. GitHub gives you a private "no-reply" address: go to **Settings** → **Emails**, tick **Keep my email addresses private**, and copy the address it shows (it looks like `1234567+yourname@users.noreply.github.com`). On the same page, tick **Block command line pushes that expose my email**. Whatever you choose, the email must belong to your GitHub account, or your commits show a grey avatar and don't count on your profile.

If you use a different identity for school or work, Git can switch automatically based on the folder a repository is in. Add this to the file `~/.gitconfig`:

`~/.gitconfig`

```
[includeIf "gitdir:~/work/"]
    path = ~/.gitconfig-work
```

WHAT EACH LINE DOES

`[includeIf "gitdir:~/work/"]`	"If the repository is anywhere inside the `work` folder in my home folder..."
`path = ~/.gitconfig-work`	"...also read the settings in this file." Put a different `user.name` and `user.email` in `~/.gitconfig-work`.

Install the GitHub CLI

AT A GLANCE **GitHub CLI (gh)**	
What is it?	GitHub's official command-line program. It does from the terminal most of what you'd otherwise click through on the website.
Why use it?	Creating pull requests, requesting reviews, watching tests and merging takes one command each instead of several pages of clicking. Commands can also be copied into notes and scripts.
How does it work?	`gh auth login` stores a login token on your computer. Each `gh` command then sends a request to GitHub's API, the same interface the website itself uses, and prints the answer.
When to use it?	Whenever you'd otherwise switch to the browser for routine work: opening PRs, checking tests, reviewing, creating issues.
Example	Right after pushing, Alice types `gh pr create --fill` and then `gh pr checks --watch`, and sees the tests pass without leaving the terminal.

This guide uses `gh` a lot, because one command is often faster than six clicks.

```
winget install --id GitHub.cli
```

WHAT EACH LINE DOES

`winget install --id GitHub.cli`	Installs `gh` on Windows using Windows' built-in package manager. Close and reopen your terminal afterwards so it finds the new program.

On macOS, run `brew install gh`; on Linux, follow the steps on cli.github.com. Then connect it to your GitHub account:

```
gh auth login
gh auth status
```

WHAT EACH LINE DOES

`gh auth login`	Starts a short question-and-answer setup. Choose **GitHub.com**, then **SSH** as the protocol, then **Login with a web browser**. It shows a one-time code; paste it into the browser page that opens.
`gh auth status`	Checks that it worked. You should see "Logged in to github.com account yourname".

SSH keys

AT A GLANCE **SSH key**	
What is it?	A matching pair of files, a private key and a public key, that proves your identity to GitHub.
Why use it?	You never type a password or token when you push, and it's safer than a password because the secret part never leaves your computer.
How does it work?	GitHub stores your public key. When you connect, GitHub sends a random challenge that only the matching private key can sign. Your computer signs it, GitHub checks the signature with the public key, and lets you in. The private key itself is never sent anywhere.
When to use it?	Set it up once on every computer you push from.
Example	Bob gets a new laptop, creates a key with `ssh-keygen`, uploads it with `gh ssh-key add`, and pushes straight away without any password prompt.

> **In plain English**
>
> An SSH key is a lock-and-key pair. The public key (the lock) is uploaded to GitHub. The private key (the key) never leaves your computer. When you push, your computer proves it holds the matching key, so you never type a password.

```
ssh-keygen -t ed25519 -C "you@example.com"
gh ssh-key add ~/.ssh/id_ed25519.pub --title "My laptop"
ssh -T git@github.com
```

WHAT EACH LINE DOES

`ssh-keygen -t ed25519 -C "you@example.com"`	Creates a new key pair. `-t ed25519` picks a modern, secure key type; `-C` adds a label so you can recognize it later. Press **Enter** to accept the default location, then type a passphrase (a password that protects the private key if your laptop is stolen).
`gh ssh-key add ~/.ssh/id_ed25519.pub --title "My laptop"`	Uploads the *public* half (the file ending in `.pub`) to your GitHub account. You can also paste it under **Settings** → **SSH and GPG keys** on the website.
`ssh -T git@github.com`	Tests the connection. The first time, type `yes` to trust GitHub's server. Success looks like the line below.

```
Hi yourname! You've successfully authenticated, but GitHub does not provide shell access.
```

If you cloned a repository with an `https://` address before, switch it to SSH:

```
git remote set-url origin git@github.com:my-org/my-app.git
git remote -v
```

WHAT EACH LINE DOES

`git remote set-url origin git@github.com:...`	Changes the address Git uses for the remote called `origin` to the SSH form.
`git remote -v`	Lists your remotes and their addresses, so you can check the change.

HTTPS also works fine. Git for Windows includes Git Credential Manager, which signs you in through the browser the first time you push and remembers you afterwards. Pick one method per computer and stick with it.

Windows: stop typing your passphrase

If you set a passphrase, Windows can remember it for you after you unlock the key once. Windows has its own SSH "agent" (a background program that holds unlocked keys), but Git for Windows ships a separate copy of SSH that doesn't use it. Start the Windows agent, then tell Git to use Windows' SSH:

```
# PowerShell opened as Administrator:
Get-Service ssh-agent | Set-Service -StartupType Automatic
Start-Service ssh-agent

# Normal PowerShell:
ssh-add $env:USERPROFILE\.ssh\id_ed25519
git config --global core.sshCommand "C:/Windows/System32/OpenSSH/ssh.exe"
```

WHAT EACH LINE DOES

`Get-Service ssh-agent | Set-Service -StartupType Automatic`	Makes the SSH agent service start automatically every time Windows starts. Needs an Administrator PowerShell (right-click PowerShell → **Run as administrator**).
`Start-Service ssh-agent`	Starts it right now.
`ssh-add $env:USERPROFILE\.ssh\id_ed25519`	Unlocks your private key once (it asks for the passphrase) and hands it to the agent.
`git config --global core.sshCommand "..."`	Tells Git to use Windows' own SSH program, which talks to that agent.

Sign your commits

AT A GLANCE **Signed commits**	
What is it?	Commits that carry a cryptographic signature made with your private key.
Why use it?	Anyone can type your name into `user.name` and push commits that look like yours. A signature proves a commit really came from you, and a ruleset can refuse commits without one.
How does it work?	When you commit, Git asks SSH to sign the commit's contents with your private key and stores the signature inside the commit. GitHub checks the signature against the signing keys on your account and shows "Verified" if they match. Changing anything in the commit breaks the signature.
When to use it?	Once everyone has their keys set up. Especially worth it for public repositories, or when your ruleset requires signed commits.
Example	A commit appears that claims to be from Chen, but it isn't signed. With vigilant mode on, GitHub marks it "Unverified", and the team knows it didn't come from Chen's laptop.

You can sign with the SSH key you already have:

```
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
git config --global tag.gpgsign true
gh ssh-key add ~/.ssh/id_ed25519.pub --type signing --title "My laptop (signing)"
```

WHAT EACH LINE DOES

`gpg.format ssh`	Sign with an SSH key instead of the older GPG system.
`user.signingkey ~/.ssh/id_ed25519.pub`	Which key to sign with.
`commit.gpgsign true`	Sign every commit automatically.
`tag.gpgsign true`	Sign every annotated tag automatically (tags are covered in Chapter 27).
`gh ssh-key add ... --type signing`	Uploads the same public key again, this time as a *signing* key, so GitHub can verify your signatures. If `gh` says it's missing a permission, run `gh auth refresh -s admin:ssh_signing_key` and try again.

Under **Settings** → **SSH and GPG keys** you can also turn on **vigilant mode**. It marks unsigned commits that claim to be yours as "Unverified", so nobody can quietly pretend to be you.

To check signatures on your own computer, Git needs a list of keys you trust:

```
echo "$(git config user.email) $(cat ~/.ssh/id_ed25519.pub)" >> ~/.ssh/allowed_signers
git config --global gpg.ssh.allowedSignersFile ~/.ssh/allowed_signers
git log --show-signature -1
```

WHAT EACH LINE DOES

`echo "..." >> ~/.ssh/allowed_signers`	Adds a line with your email and your public key to a file of trusted signers. `$(...)` inserts the output of the commands inside it.
`git config --global gpg.ssh.allowedSignersFile ...`	Tells Git where that file is.
`git log --show-signature -1`	Shows your last commit with its signature check. You should see `Good "git" signature for you@...`.

A Git config tuned for teams

AT A GLANCE Git configuration

What is it?	Settings that change how Git behaves, such as your name, your editor, and what `git pull` does.
Why use it?	Several of Git's defaults date from before teams worked the way they do today. A few settings remove a lot of everyday friction and make risky operations safer.
How does it work?	Git reads settings from three places: the whole computer (system), your user account (`--global`, the file `~/.gitconfig`) and the current repository (`.git/config`). Later ones win, so a repository can override your global setting.
When to use it?	When setting up a new computer, and whenever one of Git's defaults gets in your way.
Example	Alice keeps getting "Merge branch 'main' of github.com..." commits after every `git pull`. Setting `pull.rebase true` makes them disappear.

Run these once. Each one changes a default that causes friction when several people share a repository. The table below explains every line.

```
git config --global init.defaultBranch main
git config --global pull.rebase true
git config --global rebase.autoStash true
git config --global rebase.autoSquash true
git config --global rebase.updateRefs true
git config --global branch.autoSetupMerge simple
git config --global push.autoSetupRemote true
git config --global push.useForceIfIncludes true
git config --global push.followTags true
git config --global fetch.prune true
git config --global merge.conflictStyle zdiff3
git config --global rerere.enabled true
git config --global diff.algorithm histogram
git config --global diff.colorMoved zebra
git config --global branch.sort -committerdate
git config --global tag.sort version:refname
git config --global core.editor "code --wait"
```

Setting	What it does for you
`init.defaultBranch main`	New repositories start with a branch called `main` instead of `master`
`pull.rebase true`	`git pull` puts your local commits on top of what it downloaded, instead of adding a "Merge branch 'main' of github.com..." commit every time
`rebase.autoStash`	Puts uncommitted changes aside before a rebase and brings them back afterwards, so the rebase doesn't refuse to start
`rebase.autoSquash`	`fixup!` commits are folded into the commits they fix automatically during `git rebase -i` (Chapter 14)
`rebase.updateRefs`	When you rebase a stack of branches, the branches in the middle of the stack move too (Chapter 14)
`branch.autoSetupMerge simple`	A new branch started from `origin/main` won't get linked to `origin/main`, so your first push creates `origin/<your-branch>` instead of failing
`push.autoSetupRemote`	The first `git push` of a new branch creates it on GitHub and links the two. No more `--set-upstream origin ...`
`push.useForceIfIncludes`	Makes `--force-with-lease` safer (Chapter 13)
`push.followTags`	Pushes version tags together with the commits they point at
`fetch.prune`	When a branch is deleted on GitHub, your `origin/` copy of it disappears too, instead of piling up
`merge.conflictStyle zdiff3`	Conflict markers also show the original text, so you can see what each side changed (Chapter 15)
`rerere.enabled`	Remembers how you resolved a conflict and reuses that answer if the same conflict comes back
`diff.algorithm histogram`	Usually gives cleaner, easier-to-read diffs
`diff.colorMoved zebra`	Lines that were moved show in their own color instead of as "deleted here, added there"
`branch.sort -committerdate`	`git branch` lists your most recently used branches first
`tag.sort version:refname`	`git tag` sorts `v1.10.0` after `v1.9.0`, the way humans expect
`core.editor "code --wait"`	Commit messages and rebase lists open in VS Code. `--wait` makes Git wait until you close the tab

> **pull.rebase or pull.ff only?**
>
> Some teams prefer `git config --global pull.ff only`. Then `git pull` refuses to do anything clever when your branch and GitHub's have both changed, and you decide yourself whether to rebase or merge. Both are fine. The default behavior (adding a merge commit on every pull) is the one to avoid.

Aliases worth having

An alias is a short name for a longer command. After these, typing `git st` runs `git status -sb`, and so on, in every repository.

```
git config --global alias.lg "log --oneline --graph --decorate --all"
git config --global alias.st "status -sb"
git config --global alias.last "log -1 --stat"
git config --global alias.unstage "restore --staged"
git config --global alias.amend "commit --amend --no-edit"
git config --global alias.undo "reset --soft HEAD~1"
git config --global alias.pushf "push --force-with-lease"
git config --global alias.fixup "commit --fixup"
```

WHAT EACH ALIAS GIVES YOU

`git lg`	The whole commit graph, all branches, one line per commit.
`git st`	A short status: branch name, how far ahead or behind GitHub you are, and changed files.
`git last`	Your last commit and the list of files it changed.
`git unstage <file>`	Takes a file out of the staging area but keeps your edits.
`git amend`	Adds staged changes to the last commit without changing its message.
`git undo`	Undoes the last commit but keeps all its changes staged, ready to commit again.
`git pushf`	The safe force push (Chapter 13).
`git fixup <sha>`	Creates a commit that will be folded into commit `<sha>` later (Chapter 14).

Line endings: one file that saves a week of pain

AT A GLANCE **.gitattributes**	
What is it?	A file of rules that tells Git how to treat files matching a pattern: which line endings to use, which files are binary, and how to show their diffs.
Why use it?	Friends on Windows and on macOS produce different invisible line endings. Without shared rules, diffs fill up with fake changes and merges produce fake conflicts.
How does it work?	When you stage a file, Git converts its line endings to LF before storing it. When it writes a file into your folder, it uses the ending the rule asks for. Files marked `binary` are never converted. The file is committed, so every teammate's Git follows the same rules.
When to use it?	At the start of every project where people use different operating systems.
Example	Bob saves `app.js` on Windows, and on Alice's Mac every line shows as changed. After adding `.gitattributes` and renormalizing once, diffs show only real changes.

Windows ends each line of text with two invisible characters (called CRLF); macOS and Linux use one (LF). When friends on different systems edit the same files, Git can see every line as changed, which makes diffs

unreadable and causes fake conflicts. Fix it once, for the whole team, with a file called `.gitattributes` in the repository's top folder. It stores LF everywhere, which every modern editor on Windows handles fine:

.gitattributes

```
# Store text files with LF and check them out with LF on every OS
* text=auto eol=lf

# Windows-only scripts keep CRLF
*.bat  text eol=crlf
*.cmd  text eol=crlf
*.ps1  text eol=crlf

# Binary files: never convert, never diff as text
*.png  binary
*.jpg  binary
*.pdf  binary
*.zip  binary

# Better function names in diff headers and in git log -L
*.py    diff=python
*.java  diff=java
*.cs    diff=csharp
*.css   diff=css
*.html  diff=html
*.md    diff=markdown
```

HOW TO READ THIS FILE

`* text=auto eol=lf`	Each line is a file pattern followed by settings. `*` matches every file. `text=auto` lets Git detect which files are text; `eol=lf` stores and checks them out with LF line endings.
`*.bat text eol=crlf`	Windows batch and PowerShell scripts need CRLF to run correctly, so they keep it.
`*.png binary`	Images, PDFs and zips are binary: never convert them, and don't try to show text diffs for them.
`*.py diff=python`	Tells Git these are Python files, so diffs show the name of the function each change is in.

If the repository already has files in it, convert them once and commit the result:

```
git add --renormalize .
git commit -m "chore: normalize line endings"
```

WHAT EACH LINE DOES

`git add --renormalize .`	Re-checks every tracked file (the `.` means "this folder and everything in it") against the new rules and stages any line-ending fixes.
`git commit -m "..."`	Saves the fixes as one commit. `-m` gives the commit message directly instead of opening an editor.

PART III

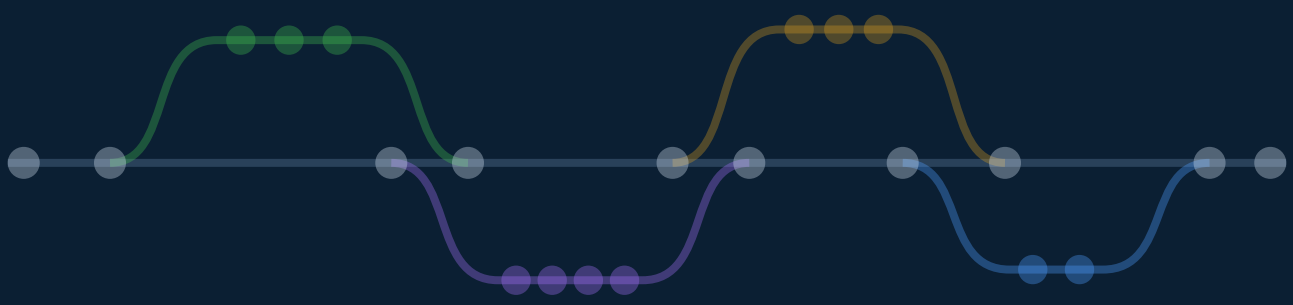
Team setup

One person builds the repository. Everyone agrees on the rules.

8 Create the team repository

9 Protect main with rulesets

10 Choose a branching strategy



CHAPTER 8

Create the team repository

Organization or personal repo, roles, plan limits, and the files every team repo needs.

One person, usually the most experienced, does the steps in this chapter. Everyone else should still read it, so they know why the repository looks the way it does.

Organization or personal repository?

AT A GLANCE **GitHub organization**	
What is it?	A shared GitHub account that belongs to a group. It has its own repositories, teams, settings and member list.
Why use it?	The project doesn't depend on one person's account, several people can be owners, and permissions can be given to whole teams at once.
How does it work?	Everyone keeps logging in with their own account. The organization gives each member, or each team, a role on each repository. Owners manage members, billing and organization-wide settings.
When to use it?	For three or more people, or for any project you might keep, show in a portfolio, or hand over to someone else.
Example	Four friends start a hackathon app in the free organization `team-pixel`. When one of them leaves the project months later, the repository at `github.com/team-pixel/app` stays exactly where it is.

A repository on GitHub belongs either to one person's account or to an organization (a shared account for a group).

	Personal repo with collaborators	Free organization
Who owns it	One person. If they lose interest or delete their account, the project goes with them	The organization. Several people can be owners
Permissions	Collaborators can push and merge. Only the owner changes settings	Per person or per team: Read, Triage, Write, Maintain, Admin
Teams and mentions	No	Yes, for example `@my-org/backend` notifies the whole backend team
Good for	Two people, short projects	Three or more people, or anything you might keep working on

For a group of friends, create a free organization:

1. On GitHub, click your profile picture (top right) → **Your organizations** → **New organization**.
2. Choose the **Free** plan.
3. Pick a name (it becomes part of every URL, like `github.com/my-org/my-app`) and enter a contact email.
4. On the next screen, invite your friends by their GitHub usernames. They get an email and must click **Join**.

It costs nothing. If you already have the project in your personal account, you can move it in later: open the repository → **Settings** → **General**, scroll to **Danger Zone** → **Transfer ownership**, and choose the organization. GitHub redirects the old URL to the new one.

Roles

Each person gets one role per repository. Roles stack: each one can do everything the role above it can, plus more.

Role	Can do
Read	Clone the repository, open issues, comment
Triage	Read, plus manage issues and pull requests (labels, assignees, closing) without pushing code
Write	Triage, plus push branches and merge pull requests. Most of the team should have this
Maintain	Write, plus manage some repository settings (topics, Pages, merge options)
Admin	Everything, including security settings, rulesets and deleting the repository

Give two people Admin, so the project doesn't stall when one of them is busy or away, and give everyone else Write. In an organization, do it with a team so you set permissions once: **Organization page** → **Teams** → **New team**, call it `core`, add your friends to it, then open the repository → **Settings** → **Collaborators and teams** → **Add teams**, choose `core` and give it the **Write** role.

Check your plan before you depend on a feature

GitHub's Free plan gives public repositories the full feature set. Private repositories on Free miss several of the features this guide relies on:

Feature	Public, Free	Private, Free	Private, Pro or Team
Rulesets and branch protection	Yes	No	Yes
Required reviewers, CODEOWNERS enforcement	Yes	No	Yes
GitHub Pages	Yes	No	Yes (the site itself is public)
Wikis	Yes	No	Yes
GitHub Actions minutes	Free on standard runners	2,000 per month	3,000 per month

Your options:

1. **Make the repository public.** Easiest, and good for your portfolio. Passwords and keys never belong in a repository anyway (Chapter 26).
2. **Use GitHub Pro.** If the owner is a student, the GitHub Student Developer Pack includes Pro at no cost. Pro covers private repositories owned by that *personal* account, with friends added as collaborators. It doesn't cover organizations.
3. **Pay for GitHub Team** if you want a private repository owned by an organization, with the rules enforced.
4. **Follow the rules by agreement.** Everything in this guide still works as a team habit. GitHub just won't block someone who breaks a rule.

> **Limits change**
>
> Every page on docs.github.com starts with a line saying which plans the feature works on. Check there if something in your settings is greyed out.

Create the repository from the terminal

```
gh repo create my-org/my-app --public --clone \
  --description "Our team project" \
  --add-readme --license mit --gitignore Node
cd my-app
```

WHAT EACH LINE DOES

`gh repo create my-org/my-app`	Creates a repository called `my-app` inside the organization `my-org`.
`--public`	Anyone can see it. Use `--private` instead if you need to (and read the plan table above).
`--clone`	Also downloads it to your computer right away, into a new folder called `my-app`.
`\`	The command continues on the next line.
`--description "Our team project"`	The short text shown under the repository name on GitHub.
`--add-readme`	Starts the repository with a `README.md` file.
`--license mit`	Adds an MIT license, which lets others reuse your code. Without any license, nobody may legally reuse it.
`--gitignore Node`	Adds a ready-made `.gitignore` for Node.js projects. Other choices include `Python`, `Java`, `VisualStudio`, `Unity` and `Android`.
`cd my-app`	Moves the terminal into the new folder.

You can do the same on the website with the **+** button (top right) → **New repository**, then clone it with `git clone`.

To add a collaborator to a *personal* repository from the terminal (in an organization, use teams as described above):

```
gh api -X PUT repos/your-name/my-app/collaborators/alice -f permission=push
```

WHAT EACH LINE DOES

`gh api`	Sends a request straight to GitHub's API, for things that don't have their own `gh` command.
`-X PUT repos/your-name/my-app/collaborators/alice`	"Add `alice` as a collaborator on `your-name/my-app`".
`-f permission=push`	With Write access (`push` is the API's name for Write). Alice still has to accept the invitation, which arrives by email and on GitHub.

The files every team repository should have

```
my-app/
├── .github/
│   ├── workflows/
│   │   └── ci.yml                # automatic tests (Chapter 23)
│   ├── ISSUE_TEMPLATE/
│   │   ├── bug_report.yml        # form for bug reports
│   │   ├── feature_request.yml   # form for feature ideas
│   │   └── config.yml
│   ├── CODEOWNERS                # who reviews which folders
│   ├── dependabot.yml            # automatic dependency updates (Chapter 26)
│   ├── pull_request_template.md  # pre-filled PR description
│   └── release.yml               # release notes layout (Chapter 27)
├── .editorconfig                 # same indentation in every editor
├── .gitattributes                # line endings (Chapter 7)
├── .gitignore                    # files Git should ignore
├── .env.example                  # list of settings, without real secrets
├── CONTRIBUTING.md               # how the team works (template in Chapter 34)
├── LICENSE
└── README.md                     # what the project is and how to run it
```

Files inside `.github/` are special: GitHub reads them to set up templates, automation and review rules. The rest of this chapter explains each one. You don't need all of them on day one; the README, `.gitignore`, `.gitattributes` and the PR template are the most important.

README

A team README answers the questions a new teammate (or you, in six months) will have on day one:

- What the project is, in one paragraph, with a screenshot.
- How to run it locally, as exact commands, tested on a fresh clone.
- How to run the tests.
- Which settings it needs (point to `.env.example`).
- A short map of the folders.
- Who's on the team, and a link to CONTRIBUTING.md.

Once Chapter 23 is done, add a badge at the top that shows whether the automatic tests are passing. This line of Markdown displays it:

```
![CI](https://github.com/my-org/my-app/actions/workflows/ci.yml/badge.svg)
```

.gitignore

`.gitignore` lists files and folders Git should pretend don't exist. Ignore:

- dependencies, which anyone can reinstall: `node_modules/`, `.venv/`
- build output, which anyone can rebuild: `dist/`, `build/`
- secrets: `.env`, `*.pem`, `*.key`
- logs: `*.log`

Editor and system clutter such as `.DS_Store`, `Thumbs.db` and `.idea/` is better placed in each person's own global ignore file, so the shared file doesn't grow a line for every editor:

```
git config --global core.excludesFile ~/.gitignore_global
```

WHAT EACH LINE DOES

`git config --global core.excludesFile ~/.gitignore_global`	Tells Git to also read `~/.gitignore_global` in every repository. Create that file and list your editor's junk files in it, one per line.

Two things trip people up:

- `.gitignore` doesn't affect files Git *already* tracks. To stop tracking one without deleting it from your disk, run `git rm --cached <file>` and commit.
- When you can't tell why a file is ignored, `git check-ignore -v <file>` prints the exact rule and the file it came from.

.editorconfig

Keeps indentation, final newlines and trailing spaces the same in everyone's editor, so nobody's changes are full of whitespace noise. VS Code needs the free EditorConfig extension; JetBrains IDEs support it out of the box.

.editorconfig

```
root = true

[*]
charset = utf-8
end_of_line = lf
insert_final_newline = true
trim_trailing_whitespace = true
indent_style = space
indent_size = 2

[*.py]
indent_size = 4

[*.md]
trim_trailing_whitespace = false
```

HOW TO READ THIS FILE

`root = true`	This is the top-level settings file; don't look in parent folders for more.
`[*]`	The settings below apply to every file.
`charset`, `end_of_line`	Save files as UTF-8 text with LF line endings.
`insert_final_newline`, `trim_trailing_whitespace`	End every file with a newline, and remove invisible spaces at the end of lines.
`indent_style = space`, `indent_size = 2`	Indent with two spaces, never tabs.
`[*.py]` with `indent_size = 4`	Python files use four spaces, the Python convention.
`[*.md]`	In Markdown, trailing spaces can mean a line break, so keep them.

CODEOWNERS

AT A GLANCE **CODEOWNERS**	
What is it?	A file that names the people or teams responsible for each part of the repository.
Why use it?	The right person automatically reviews every change to their area, and nobody has to remember who to ask.
How does it work?	When a pull request is opened, GitHub matches each changed file against the patterns in the file, with the last matching line winning. It requests a review from those owners. If the ruleset requires code owner review, the PR can't merge until one of them approves.
When to use it?	When different people know different parts of the code best, or when some files are risky to change (workflows, payment code).
Example	Chen built the backend. With the line `/backend/ @chen`, every pull request that touches the backend automatically asks Chen for a review.

`.github/CODEOWNERS`

```
# The last matching pattern wins.
*               @my-org/core
/frontend/      @alice @bob
/backend/       @chen
/docs/          @alice
/.github/       @alice @chen
```

HOW TO READ THIS FILE

`* @my-org/core`	Each line is a file pattern followed by owners. `*` means "any file", owned by the `core` team.
`/frontend/ @alice @bob`	Anything inside the `frontend` folder is owned by Alice and Bob. A later line overrides earlier ones for the same files.
`/.github/ @alice @chen`	Changes to automation and templates need Alice or Chen.

Turn on **Require review from Code Owners** in the ruleset (Chapter 9) to make their approval mandatory.

Pull request template

AT A GLANCE **Templates for pull requests and issues**	
What is it?	Files inside `.github/` that pre-fill new pull request descriptions, and turn the "new issue" page into a form.
Why use it?	Every PR and bug report arrives with the same information (what, why, how to test), so reviews and fixes go faster and nobody forgets the important parts.
How does it work?	GitHub looks for these files by name. When someone opens a PR, it copies `pull_request_template.md` into the description box. When someone opens an issue, it shows the forms from `ISSUE_TEMPLATE/` and builds the issue text from the answers.
When to use it?	As soon as more than two people open pull requests and issues.
Example	Bug reports used to say only "login broken". With the bug form, every report now arrives with the steps to reproduce it and the browser that was used.

`.github/pull_request_template.md`

```
## What does this change?
<!-- One or two sentences. -->

Closes #

## Why?

## How to test
1.
2.

## Screenshots
<!-- For UI changes: before and after -->

## Checklist
- [ ] I ran it locally and tried the change
- [ ] Tests added or updated
- [ ] README or docs updated if needed
- [ ] Small enough to review in about 15 minutes
```

HOW TO READ THIS FILE

`## What does this change?`	Lines starting with `##` become headings.
`<!-- ... -->`	A hidden comment: visible while writing, invisible once posted. Use it for instructions.
`Closes #`	The author adds the issue number, such as `Closes #42`, which closes that issue automatically when the PR merges (Chapter 16).
`- [ ] ...`	A checklist. On GitHub the boxes become clickable.

Issue forms

Issue forms turn the blank "new issue" box into a short form, so bug reports arrive with the information you need.

`.github/ISSUE_TEMPLATE/bug_report.yml`

```yaml
name: Bug report
description: Something is broken
title: "bug: "
labels: ["bug"]
body:
  - type: textarea
    id: what
    attributes:
      label: What happened, and what did you expect?
    validations:
      required: true
  - type: textarea
    id: steps
    attributes:
      label: Steps to reproduce
      value: |
        1.
        2.
        3.
    validations:
      required: true
  - type: input
    id: where
    attributes:
      label: Browser, OS or commit
```

HOW TO READ THIS FILE

`name`, `description`	What people see when they click **New issue** and choose a form.
`title: "bug: "`	The issue title starts pre-filled with this text.
`labels: ["bug"]`	The `bug` label is added automatically.
`body:`	The list of form fields. Each item starting with `-` is one field.
`type: textarea` / `type: input`	A large text box, or a single-line text box.
`id`	An internal name for the field; must be unique in the form.
`label`	The question shown above the field.
`value: |`	Text pre-filled in the box. The `|` lets the value span several lines.
`required: true`	The form can't be submitted while this field is empty.

`.github/ISSUE_TEMPLATE/config.yml`

```yaml
blank_issues_enabled: false
```

This one line forces everyone to use a form instead of a blank issue. Copy the bug form to `feature_request.yml` and change the questions, for example "What problem does this solve?" and "What would done look like?".

Repository settings to change on day one

Open the repository on GitHub and click **Settings**. Under **General**:

- **Features:** keep **Issues** on. Turn on **Projects** and **Discussions** if you'll use them (Chapter 22). Consider turning off the **Wiki** and keeping documentation in a `docs/` folder instead, where changes are reviewed like code.
- **Pull Requests:** untick **Allow merge commits** and **Allow rebase merging**, and keep **Allow squash merging** ticked (Chapter 13 explains why). Under it, set the default commit message to **Pull request title and description**.
- On the same page, tick **Always suggest updating pull request branches** (shows a button to bring a PR up to date), **Allow auto-merge** (lets a PR merge itself once it's approved and the checks pass) and **Automatically delete head branches** (deletes a PR's branch after merging, so old branches don't pile up).

Under **Settings → Actions → General**, set **Workflow permissions** to **Read repository contents and packages permissions**. This means automation can read your code but not change it, unless a workflow asks for more (Chapter 23).

Security settings (Dependabot, secret scanning, push protection) are covered in Chapter 26.

CHAPTER 9

Protect main with rulesets

Make the safe workflow the only workflow.

AT A GLANCE **Ruleset**	
What is it?	A set of rules that GitHub enforces on chosen branches or tags, such as "changes need a pull request" or "tests must pass".
Why use it?	It turns the team's agreements into something automatic. Nobody can break `main` by accident, at 2 a.m., or in a hurry.
How does it work?	Every time someone pushes to a protected branch or tries to merge into it, GitHub checks every active ruleset that targets it. If any rule isn't satisfied (no pull request, missing approval, failing check, force push), GitHub rejects the push with an error such as GH013, or keeps the merge button disabled.
When to use it?	As soon as other people depend on `main`. Ideally on the first day of the project.
Example	At midnight Bob tries to push a quick fix straight to `main`. GitHub rejects it, so Bob opens a PR, and the tests catch a typo that would have broken the site.

A rule everyone agrees on and nobody enforces lasts about a week. Rulesets make GitHub refuse pushes and merges that break your rules, so the safe path becomes the only path.

> **In plain English**
>
> A ruleset is a bouncer standing in front of `main`. Nobody gets in without a pull request, an approval and a green test run, however sure they are that their change is fine.

What to protect, and why

- **Nobody pushes straight to main.** Every change arrives through a pull request, so every change is seen by at least one other person.
- **Automatic tests must pass before merging.** Broken code never reaches `main`, so `main` always works.
- **History on main is never rewritten.** No force pushes, and nobody can delete the branch.
- **Review comments get resolved.** Nothing merges while someone still has an open question.

Create the ruleset

Open the repository → **Settings** → **Rules** → **Rulesets** → **New ruleset** → **New branch ruleset**.

1. **Ruleset name:** `protect-main`. **Enforcement status:** Active. (Disabled would save the rules without applying them.)
2. **Bypass list:** leave it empty. People on this list can ignore the rules; read the team rule at the end of this chapter before adding anyone.
3. **Target branches:** click **Add target** → **Include default branch**. This applies the rules to `main`.
4. Turn on the rules in the table below, then click **Create** at the bottom.

Rule	Setting and reason
Restrict deletions	On. Nobody can delete main
Block force pushes	On. History on main can't be rewritten
Require linear history	On, if you only allow squash merges (Chapter 8). Keeps main a straight line of commits
Require a pull request before merging	Required approvals: **1**. Then tick **Dismiss stale pull request approvals when new commits are pushed** (an approval doesn't cover code added after it), **Require conversation resolution before merging** (all comment threads must be marked resolved) and, if you use CODEOWNERS, **Require review from Code Owners**. Under allowed merge methods, keep only **Squash**
Require status checks to pass	Add your automatic test check (Chapter 23 sets up one called `ci-ok`). Tick **Require branches to be up to date before merging**
Require signed commits	Optional. Turn it on once everyone has signing set up (Chapter 7)

A status check only appears in the search box after its workflow has run at least once. So merge the test workflow from Chapter 23 first, then come back and add the check.

Require branches to be up to date means a pull request must contain the latest `main` before it can merge. It catches a sneaky case: two pull requests each pass their tests on their own, but break each other once both are merged. With a small team, the occasional extra click on **Update branch** is a cheap price.

Protect release tags too

Tags mark released versions (Chapter 27). Create a second ruleset to stop anyone moving them: **New ruleset** → **New tag ruleset**, name it `protect-tags`, set **Target tags** with **Add target** → **Include by pattern** and the pattern `v*` (every tag starting with "v"), then turn on **Restrict updates** and **Restrict deletions**. Once `v1.0.0` is published, nobody can quietly point it at different code.

Rulesets or classic branch protection?

You may see an older screen at **Settings** → **Branches** → **Add branch protection rule**. It does most of the same things. Rulesets are the newer system: several can apply to the same branch at once, anyone with read access can see which rules apply, and the bypass list is explicit. Use rulesets for new repositories.

Test it

Prove the rules work by trying to break them:

```
git switch main
git commit --allow-empty -m "test: direct push"
git push
git reset --hard origin/main
```

WHAT EACH LINE DOES

`git switch main`	Go to the `main` branch.
`git commit --allow-empty -m "test: direct push"`	Make a commit with no changes in it, just to have something to push.
`git push`	Try to push it straight to `main`. GitHub should refuse with the message below.
`git reset --hard origin/main`	Throw the test commit away, so your `main` matches GitHub's again.

```
remote: error: GH013: Repository rule violations found for refs/heads/main.
remote: - Changes must be made through a pull request.
```

If the push goes through instead, the ruleset isn't active or doesn't target `main`. Check its enforcement status and target.

> **Admins follow the rules too**
>
> Don't put yourself on the bypass list "just in case". The person with the most access is usually the one who pushes the 2 a.m. fix that breaks everything. If a real emergency needs a bypass, an admin can temporarily disable the ruleset and turn it back on afterwards, which at least leaves a visible trace.

CHAPTER 10

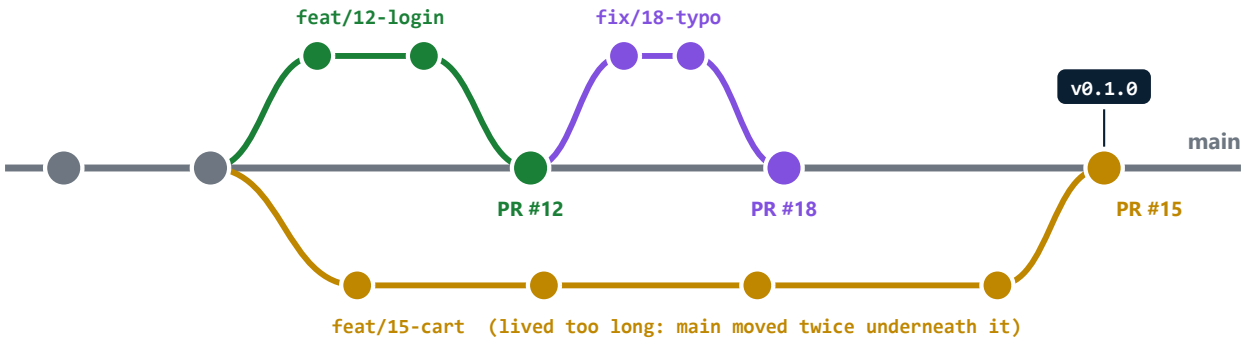
Choose a branching strategy

GitHub Flow for most teams, and when something heavier pays off.

A branching strategy is the team's agreement on which branches exist, where new branches start, and how work gets back into `main`. Without one, everyone invents their own, and merging becomes guesswork.

GitHub Flow

AT A GLANCE **GitHub Flow**	
What is it?	A simple branching strategy: one permanent branch (`main`) that always works, plus short-lived branches for each change.
Why use it?	It's easy to remember, keeps conflicts small because branches live only a few days, and means you can deploy `main` at any time.
How does it work?	Create a branch from `main`, commit, open a pull request, get a review and passing tests, squash-merge it into `main`, delete the branch, deploy. Then repeat for the next issue.
When to use it?	For most web apps, school projects and small teams, especially when you deploy often.
Example	Alice works on `feat/3-signup` and Bob on `feat/4-profile`. Each branch is merged through a PR within two days, and `main` stays ready to deploy the whole time.



GitHub Flow: short branches off main, merged back through pull requests. The amber branch lived longest, so main changed twice underneath it.

GitHub Flow has five rules:

1. `main` always works and could be shown to anyone at any moment.
2. Every change, however small, happens on a short-lived branch created from `main`.
3. Open a pull request early. The automatic tests run on it, and teammates can see what you're doing.
4. After a review and passing tests, squash-merge it into `main` and delete the branch.
5. Deploy (publish the app) from `main`.

It's simple enough that everyone remembers it, and it's what the rest of this guide assumes.

Other strategies

Strategy	How it works	Use it when
GitHub Flow	`main` plus short feature branches, merged through PRs	Web apps, student projects, most teams. Start here
Trunk-based	Everyone merges small changes into `main` at least daily. Unfinished features are hidden behind feature flags	The team has good tests and wants the fewest conflicts possible
Git Flow	Long-lived `develop`, plus `feature/*`, `release/*` and `hotfix/*` branches. `main` only holds releases	You maintain several released versions at once, like installable apps or libraries. Usually overkill for a friends' project
GitHub Flow plus release branches	Normal GitHub Flow, plus a `release/1.x` branch for patching an old version	You ship versions that users can't update instantly, like mobile or desktop apps

A **feature flag** is a simple on/off switch in your code, like `if (FEATURES.newCheckout) { ... }`. Unfinished work can be merged into `main` with the switch off, so it's safely hidden until it's ready.

Branch names

Use the pattern `<type>/<issue number>-<short description>`:

```
feat/42-login-form
fix/57-cart-total-rounding
docs/61-setup-instructions
refactor/70-split-api-client
chore/75-upgrade-node-24
```

WHAT THE PARTS MEAN

`feat`, `fix`, `docs`, `refactor`, `chore`	The kind of work: a new feature, a bug fix, documentation, restructuring code without changing behavior, or maintenance. The same types are used in commit messages (Chapter 12).
`42`	The number of the GitHub issue this branch works on, so anyone can look up why it exists.
`login-form`	A few words saying what it is. Lowercase, with hyphens instead of spaces.

How long a branch should live

- Aim for one to three days, and under about 400 changed lines.
- The longer a branch lives, the more `main` changes underneath it, and the worse the conflicts get when you finally merge (see the amber branch in the diagram).
- If a feature takes two weeks, merge it in pieces, with the unfinished parts behind a feature flag. Remove the flag when it's done.

One issue, one branch, one pull request

Every branch starts from an issue, and every pull request closes one. If you notice an unrelated bug while working, open a new issue for it instead of fixing it in the same PR. Small, single-purpose PRs get reviewed faster and are easy to undo.

PART IV

Daily workflow

The loop you'll run dozens of times: branch, commit, rebase, resolve, review, merge.

11 The team loop, step by step

12 Write commits your team can read

13 Merge, squash or rebase

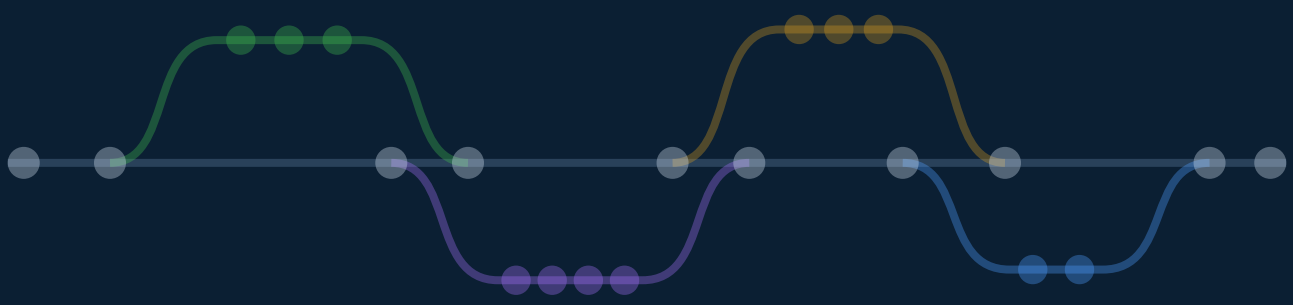
14 Interactive rebase

15 Merge conflicts without panic

16 Pull requests that get merged

17 Code review as a team

18 Forks and upstream remotes



CHAPTER 11

The team loop, step by step

From picking an issue to cleaning up after the merge.

This is the routine every team member follows for every piece of work. The first few times it feels like a lot of steps. After a week it's automatic, and it's what keeps four people from tripping over each other.



The daily loop. You do steps 1 to 4; steps 5 to 7 involve the automatic tests and a teammate.

1. Pick an issue and create the branch

Work starts from an issue on GitHub (Chapter 22 shows how to write them). Claim one so nobody else starts the same thing, then create a branch for it.

```
gh issue list --assignee @me
gh issue view 42
gh issue edit 42 --add-assignee @me
gh issue develop 42 --checkout --name feat/42-login-form
```

WHAT EACH LINE DOES

`gh issue list --assignee @me`	Lists the open issues assigned to you. `@me` means "my own account". Drop `--assignee @me` to see all open issues.
`gh issue view 42`	Shows issue number 42 in the terminal: title, description and comments. Add `--web` to open it in the browser.
`gh issue edit 42 --add-assignee @me`	Assigns the issue to you, so the team sees you're working on it.
`gh issue develop 42 --checkout --name feat/42-login-form`	Creates a branch called `feat/42-login-form` on GitHub, starting from the latest `main`, links it to issue 42 (it appears under "Development" in the issue's sidebar), downloads it and switches you to it.

You can also do it with plain Git:

```
git switch main
git pull
git switch -c feat/42-login-form
```

WHAT EACH LINE DOES

`git switch main`	Go to your `main` branch.
`git pull`	Download what your friends merged since last time, so your branch starts from the newest code.
`git switch -c feat/42-login-form`	Create a new branch with that name (`-c` means create) and switch to it. Your files don't change yet; the new branch starts identical to `main`.

2. Commit in small steps

Write code as usual. Whenever you reach a point that works, even if the whole feature isn't done, save it as a commit:

```
git status
git add -p
git commit
```

WHAT EACH LINE DOES

`git status`	Shows which files you've changed. Red files have changes that aren't staged yet.
`git add -p`	Goes through your changes one block at a time and asks whether to include each one (Chapter 12 explains the keys). This way you see everything before it's committed.
`git commit`	Opens your editor so you can write a proper commit message (Chapter 12). Save and close the tab to finish.

Small commits are easy to review, easy to rearrange later (Chapter 14) and easy to undo (Chapter 19).

3. Push early and open a draft pull request

AT A GLANCE **Draft pull request**	
What is it?	A pull request marked "not ready for review yet".
Why use it?	Your team can see what you're working on and the automatic tests run on every push, but nobody is asked to review unfinished work.
How does it work?	A draft can't be merged. GitHub doesn't send review requests to code owners while a PR is a draft. When you run `gh pr ready` (or click **Ready for review**), it becomes a normal PR and the review requests go out.
When to use it?	As soon as you've pushed the first commit on a branch, long before the work is finished.
Example	Chen opens a draft PR for the payment page on day one. Alice sees it touches the same file Alice is editing, and they agree who goes first before any conflict happens.

```
git push
gh pr create --draft --fill --assignee @me
```

WHAT EACH LINE DOES

`git push`	Uploads your branch to GitHub. The first time, `push.autoSetupRemote` from Chapter 7 creates `origin/feat/42-login-form` and links it to your branch.
`gh pr create --draft --fill --assignee @me`	Opens a pull request for this branch. `--draft` marks it "not ready for review yet"; `--fill` uses your commit messages as the title and description; `--assignee @me` marks you as responsible for it.

You can edit the title and description later with `gh pr edit` or on the website. If your plan doesn't offer drafts on a private repo, open a normal pull request and start its title with "WIP:" (work in progress).

4. Stay current with main

AT A GLANCE **Rebasing onto main**	
What is it?	Moving your branch so that it starts from the newest `main`, by replaying your commits on top of it.
Why use it?	Your branch then includes your friends' latest work. Conflicts appear early and in small pieces, and your pull request shows only your own changes.
How does it work?	Git finds the commit where your branch split off from `main`, sets your commits aside, moves your branch to the tip of `origin/main`, and re-applies your commits one by one. Each re-applied commit is new, with a new ID, which is why the push afterwards must be a forced one.
When to use it?	At least once a day while your branch is open, and right before you ask for review, as long as only you push to the branch.
Example	While Bob builds the profile page, Alice merges a new navbar into `main`. Bob rebases, gets the new navbar in the branch, and fixes one small conflict now instead of a big one next week.

While you work, your friends merge their pull requests into `main`. Bring their work into your branch at least once a day, and always before asking for a review:

```
git fetch
git rebase origin/main
git push --force-with-lease
```

WHAT EACH LINE DOES

`git fetch`	Downloads the latest commits from GitHub, including the new `main`, without changing your files.
`git rebase origin/main`	Lifts your branch's commits off, moves the starting point of your branch up to the newest `main`, and puts your commits back on top. If your changes and theirs touch the same lines, Git stops and asks you to resolve a conflict (Chapter 15).
`git push --force-with-lease`	Uploads the updated branch. A normal push is refused because the rebase gave your commits new IDs; `--force-with-lease` replaces the old version safely (Chapter 13).

> **In plain English**
>
> Rebasing is like rewriting your to-do list on a fresh page after the plan changed. Same items, same order, but now they start from where the team is today instead of where it was last week.

If a teammate also pushes to your branch, merge instead, so you don't rewrite commits they already have:

```
git fetch
git merge origin/main
git push
```

WHAT EACH LINE DOES

`git merge origin/main`	Combines the newest `main` into your branch by adding a merge commit. Nothing is rewritten, so a normal `git push` works and your teammate's copy stays valid.

5. Ask for review

```
gh pr ready
gh pr edit --add-reviewer alice,bob
gh pr checks --watch
```

WHAT EACH LINE DOES

`gh pr ready`	Changes the draft into a normal pull request, ready for review.
`gh pr edit --add-reviewer alice,bob`	Asks Alice and Bob to review. They get a notification.
`gh pr checks --watch`	Shows the automatic tests on this PR and keeps updating until they finish. Green means passed, red means failed.

Make sure the description says `Closes #42`, so merging the PR also closes the issue.

6. Respond to review

- Fix what reviewers ask for in new commits and push them. GitHub shows reviewers what changed since their last review.
- Reply to every comment, even with just "Done in 3f2a1c9". If you disagree, explain why politely.
- When you've addressed everything, ask for another look: click the circular-arrow icon next to the reviewer's name on the PR page, or run `gh pr edit --add-reviewer alice` again.

7. Merge

Once the PR is approved and the tests pass:

```
gh pr merge --squash --delete-branch
gh pr merge --auto --squash --delete-branch
```

WHAT EACH LINE DOES

`gh pr merge --squash --delete-branch`	Merges the PR now. `--squash` combines all its commits into one commit on `main`; `--delete-branch` deletes the branch on GitHub and on your computer afterwards.
`gh pr merge --auto --squash --delete-branch`	The same, but it waits: GitHub merges it automatically the moment approvals and tests are complete. Use one of the two lines, not both.

Before confirming, look at the squash commit message. It becomes the single line in `main`'s history for this whole piece of work, so it should read like `feat(auth): add login form (#42)`. GitHub adds the PR number for you.

8. Clean up

If you merged with `gh pr merge --delete-branch`, `gh` has already switched you to `main` and deleted the local branch; just run `git pull`. If you merged on the website:

```
git switch main
git pull
git branch -d feat/42-login-form
```

WHAT EACH LINE DOES

`git switch main`	Go back to `main`.
`git pull`	Download the newly merged work.
`git branch -d feat/42-login-form`	Delete your local copy of the finished branch. `-d` refuses if Git thinks the branch isn't merged.

After a squash merge, `git branch -d` usually refuses with "not fully merged". Git compares commits, and the squash commit on `main` is a brand-new commit, so Git can't tell your work is in it. If the PR page says "Merged", delete it with `-D` (capital D, which skips the check).

Over time, old branches pile up. This deletes every local branch whose GitHub copy has been deleted. Run it while you're on `main`:

```
git fetch --prune
git branch -vv | grep ': gone]' | awk '{print $1}'
git branch -vv | grep ': gone]' | awk '{print $1}' | xargs git branch -D
```

WHAT EACH LINE DOES

`git fetch --prune`	Downloads updates and forgets `origin/` copies of branches that were deleted on GitHub.
`git branch -vv`	Lists your branches with details. Branches whose GitHub copy is gone show `: gone]`.
`grep ': gone]'`	Keeps only the lines that contain `: gone]`.
`awk '{print $1}'`	Keeps only the first word of each line: the branch name. The second line stops here, so you can review the list first.
`xargs git branch -D`	Passes those names to `git branch -D`, which deletes them.

The golden rules

- Nobody commits directly to main.
- Start every branch from an up-to-date main.
- Push your branch at least once a day, even unfinished.
- Only rebase, amend or force-push branches that nobody else works on.
- Use `--force-with-lease`, never plain `--force`.
- Review within one day of being asked.

CHAPTER 12

Write commits your team can read

Atomic commits, messages that explain why, and staging exactly what you mean.

One logical change per commit

An "atomic" commit does exactly one thing, works on its own, and can be undone without dragging unrelated changes along. Don't mix "rename a variable in 30 files" with "fix the rounding bug" in one commit. The reviewer can't find the three-line fix among 200 renamed lines, and you can't undo one without the other.

Even if your team squash-merges, this matters: reviewers often read a pull request commit by commit, and `git bisect` (Chapter 21) works far better on small commits.

Message format

```
fix(cart): round line items before applying tax

Totals like 19.999 were shown as 20.00 but charged as 19.99,
because rounding happened after tax. Round each line item first,
then apply tax to the rounded subtotal.

Closes #57
Co-authored-by: Bob Lee <1234567+bob@users.noreply.github.com>
```

THE PARTS OF A COMMIT MESSAGE

`fix(cart): round line items...`	The **subject line**: at most 72 characters (50 is better), written as a command ("round", not "rounded"), no full stop. `fix` is the type and `cart` the area of the code (see Conventional Commits below).
(blank line)	Separates the subject from the body. Git and GitHub rely on it.
The middle paragraph	The **body**: why the change was needed and what behaves differently now. The diff already shows how. Wrap lines at about 72 characters.
`Closes #57`	Links and closes issue 57 when this reaches `main`.
`Co-authored-by: ...`	Credits a pair-programming partner. GitHub shows both avatars. Use an email from their GitHub account; their no-reply address works.

For small changes, the subject line alone is enough: `git commit -m "docs: fix typo in setup steps"`.

Conventional Commits

AT A GLANCE **Conventional Commits**	
What is it?	A shared format for commit messages: `type(scope): description`, for example `fix(cart): round totals before tax`.
Why use it?	People can scan the history and see at a glance what kind of change each commit is, and tools can write changelogs and choose the next version number automatically.
How does it work?	It's only a text convention; Git itself doesn't care. Tools read the type at the start of each message: `feat` means a new feature (minor version), `fix` a bug fix (patch version), and `!` or `BREAKING CHANGE:` a breaking change (major version).
When to use it?	For every commit message, or at least every pull request title if your team squash-merges.
Example	Before a release, Chen reads the history and sees three `feat:` and five `fix:` commits since v1.2.0, so the next version is a minor one: v1.3.0.

A widely used format for the subject line: `type(scope)!: description`. The scope (the part in brackets) is optional.

Type	Use for	Version bump
`feat`	A new feature users can see	minor
`fix`	A bug fix	patch
`perf`	A performance improvement	patch
`refactor`	Code change that neither fixes a bug nor adds a feature	none
`docs`	Documentation only	none
`test`	Adding or fixing tests	none
`style`	Formatting, whitespace, missing semicolons	none
`build`	Build system or dependencies	none
`ci`	Automatic test configuration	none
`chore`	Other maintenance	none
`revert`	Undoes an earlier commit	depends

A breaking change (one that forces users to change something on their side) gets a `!` after the type, like `feat(api)!: remove v1 endpoints`, or a `BREAKING CHANGE:` line at the end of the message. It means a major version bump. The "version bump" column is explained in Chapter 27.

With squash merges, only the pull request title needs to follow the format, because the PR title becomes the commit on `main`.

Stage exactly what you mean

AT A GLANCE **Interactive staging (git add -p)**	
What is it?	Staging only some of the changes in a file, block by block.
Why use it?	It lets you split mixed edits into separate, focused commits, and you review every change before it goes in.
How does it work?	Git splits each file's diff into "hunks" (groups of nearby changed lines), shows them one at a time, and copies only the hunks you accept into the staging area. The rest stays in your files, unstaged.
When to use it?	When a file contains changes that belong in different commits, or simply before every commit, to review what you're about to save.
Example	Alice's file contains a real fix and a leftover `console.log`. In `git add -p`, Alice presses `y` for the fix and `n` for the debug line.

```
git add -p
git add -p src/cart.js
git diff --staged
git commit -v
git add -N new-file.js
```

WHAT EACH LINE DOES

`git add -p`	"Patch" mode. Git shows each changed block (a "hunk") and asks what to do: `y` stage it, `n` skip it, `s` split it into smaller blocks, `e` edit it by hand, `q` quit, `?` help.
`git add -p src/cart.js`	The same, for one file only.
`git diff --staged`	Shows exactly what the next commit will contain.
`git commit -v`	Opens the commit message editor with the full diff shown below, so you can describe what you're actually committing.
`git add -N new-file.js`	Brand-new files don't appear in `git add -p` until Git knows about them. `-N` ("intent to add") tells Git about the file without staging its contents.

`git add -p` is the habit that makes atomic commits possible. Because you see every change before it goes in, it also catches leftover debug lines like `console.log("here")`.

Fix the last commit

AT A GLANCE **Amending a commit**	
What is it?	Replacing your most recent commit with a corrected version.
Why use it?	To fix a typo in the message or add a forgotten file without creating a "fix" commit.
How does it work?	Git builds a new commit from the old commit's changes plus whatever you've staged, gives it the same parent as the old one, and moves your branch to it. The old commit isn't deleted straight away; it stays reachable through the reflog for a while.
When to use it?	Right after committing, to fix the message or add a forgotten file, before anyone else has the commit.
Example	Bob commits and immediately notices `styles.css` wasn't included. `git add styles.css` and `git commit --amend --no-edit` fix it without an extra "forgot a file" commit.

```
git add forgotten-file.js
git commit --amend --no-edit
git commit --amend
```

WHAT EACH LINE DOES

`git add forgotten-file.js`	Stage the file you forgot.
`git commit --amend --no-edit`	Adds it to your last commit, keeping the same message.
`git commit --amend`	Opens the editor so you can change the last commit's message (and includes anything staged).

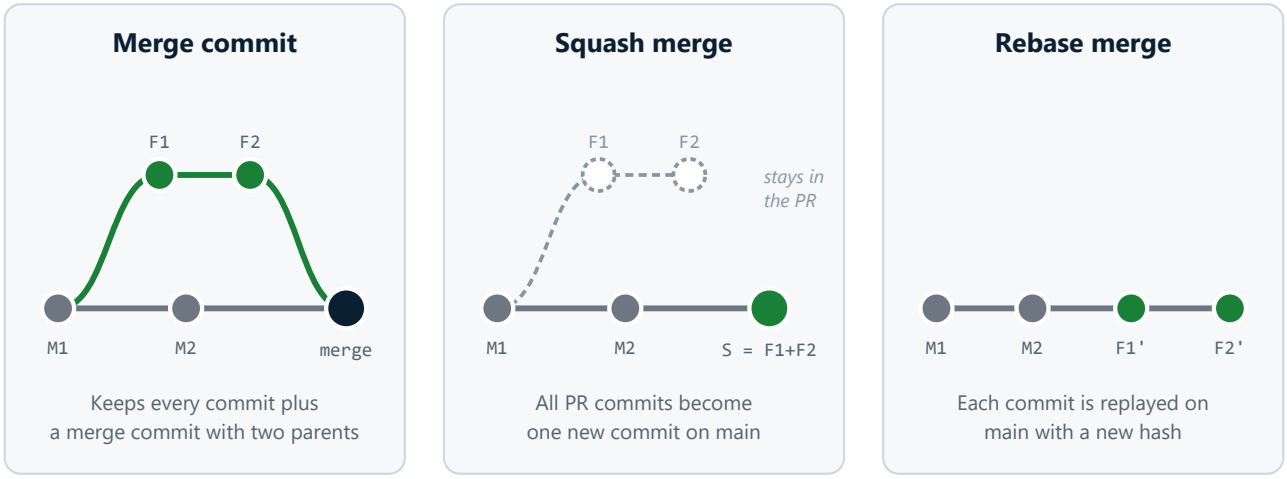
If you've already pushed the old one, you'll need `git push --force-with-lease`, so only amend commits on your own branch.

CHAPTER 13

Merge, squash or rebase

Three ways to combine work, and which one your team should pick.

When a pull request is finished, its commits have to get into `main` somehow. GitHub offers three buttons for this, and the choice changes what `main`'s history looks like.



The same pull request (commits F1 and F2) landed on main three different ways.

Merge commit

AT A GLANCE **Merge commit**	
What is it?	A commit with two parents that joins the history of two branches.
Why use it?	It keeps the exact history of the branch, every commit included, and records when and where it was combined.
How does it work?	Git finds the last commit both branches share (the merge base), combines the changes each side made since then, and saves the result as a new commit whose parents are the tips of both branches.
When to use it?	For long-lived branches such as release branches, or when you want to keep every commit and the exact shape of the branch.
Example	The team keeps `release/1.x` for patches. Merging it back into `main` with a merge commit shows clearly when each patch came in.

You get one with GitHub's **Create a merge commit** button, or with `git merge feature` on your computer.

- Keeps the exact history, including every "wip" and "fix typo" commit.
- The graph gets busy fast when several people merge every day.
- Good for long-lived branches like `release/1.x`, where you want the history preserved exactly.

Squash merge

AT A GLANCE **Squash merge**	
What is it?	Merging a pull request as one single new commit on `main`.
Why use it?	`main` gets one clean commit per feature or fix, which makes history easy to read and a whole feature easy to undo.
How does it work?	GitHub calculates the total change the PR makes compared with where it started, applies it on top of `main` as one new commit with a single parent, and uses the PR title as the message. The branch's own commits aren't added to `main`; they stay visible in the PR.
When to use it?	As the default for pull requests in a small team, especially when branch histories are messy.
Example	Chen's PR has 14 commits like "wip" and "try again". Squash-merging puts one clean commit, `feat: add payment page (#31)`, on `main`.

What that means in practice:

- `main` reads like a list of finished features and fixes, one commit per pull request.
- Undoing a feature means reverting one commit.
- The individual commits are gone from `main`, but they stay visible in the closed pull request.
- Git doesn't see the branch as merged, so `git branch -d` complains (use `-D` once the PR shows "Merged").
- Don't keep working on a branch after it was squash-merged. Start a fresh branch from `main`.

Rebase merge

AT A GLANCE **Rebase merge**	
What is it?	Merging a pull request by copying each of its commits onto the end of `main`.
Why use it?	History stays a straight line, and every individual commit is kept.
How does it work?	GitHub replays the PR's commits one by one on top of the latest `main`, as `git rebase` would, creating new commits with new IDs, and then moves `main` to the last one. No merge commit is created.
When to use it?	When every commit in the PR is clean and meaningful on its own, and you want to keep them all on `main`.
Example	Alice's PR has three tidy commits: a refactor, the feature and its tests. A rebase merge keeps all three on `main`, in a straight line.

It only makes sense if every commit is clean, which means tidying them with interactive rebase first (Chapter 14).

What your team should pick

Allow **squash merging only**, write pull request titles in Conventional Commits format, and turn on "require linear history" in the ruleset. It's the most forgiving setup: messy commits on a branch never reach `main`. If your team later gets disciplined about atomic commits and wants them on `main`, switch to rebase merges.

Bringing main into your branch: rebase or merge?

This is a different question from the one above. While your branch is still open, you also need to pull new work from `main` into it (step 4 of Chapter 11). You can do that two ways:

	`git rebase origin/main`	`git merge origin/main`
Result	Your commits replayed on top of the newest main, in a straight line	A merge commit added to your branch
Rewrites history?	Yes, so you push with `--force-with-lease`	No
Safe if others push to the branch?	No	Yes
Conflicts	Resolved commit by commit	Resolved once

The golden rule of rebasing

Don't rebase commits that other people may have built on. Rebasing your own unmerged feature branch is normal and fine. Rebasing `main`, or a branch a teammate is also committing to, is not: their copies still contain the old commits, and the next time they pull, both versions get tangled together.

Force-pushing safely

AT A GLANCE **--force-with-lease**	
What is it?	A push that is allowed to overwrite the branch on GitHub, but only if nobody else has changed it since you last looked.
Why use it?	After a rebase or amend, a normal push is refused. A plain `--force` could silently erase a teammate's commits; the lease check prevents that.
How does it work?	Your Git tells GitHub "I expect this branch to be at commit X", where X is your `origin/<branch>`. GitHub updates the branch only if it really is still at X. If someone pushed in the meantime, it isn't, and the push is rejected.
When to use it?	Every time you have to push after a rebase, an amend or an interactive rebase on your own branch. Never use plain `--force`.
Example	Bob rebases and pushes with `--force-with-lease`. Alice had pushed a small fix to Bob's branch in the meantime, so the push is refused; Bob fetches, sees Alice's commit, and keeps it.

After a rebase or an amend, GitHub's copy of your branch and yours have different commits, so a normal push is refused. You have to force it.

```
git push --force-with-lease
```

WHAT EACH LINE DOES

`git push --force`	(Don't use.) Replaces the branch on GitHub with yours, no questions asked, even if a teammate pushed something one minute ago. Their work is gone from GitHub.
`git push --force-with-lease`	Replaces the branch only if it's still exactly where your computer last saw it. If someone pushed in the meantime, the push is refused, and you run `git fetch` and look before trying again.

There's one gap. If something downloaded updates in the background (VS Code does this), your computer's record is already up to date, so the check passes even though you never looked at the new commits. The `push.useForceIfIncludes true` setting from Chapter 7 closes that gap: Git also checks that your branch really contains the latest commit from GitHub.

> **Never force-push to main**
>
> The ruleset from Chapter 9 blocks it, and it should. Rewriting `main` breaks every open branch and every teammate's copy.

CHAPTER 14

Interactive rebase

Rewrite your branch into a clean story before anyone reviews it.

AT A GLANCE **Interactive rebase**	
What is it?	A rebase where you edit the list of your commits before Git replays them: combine, reorder, reword, split or delete.
Why use it?	To turn a messy work-in-progress history into a short, clear series of commits before review. Clean commits are easier to review, revert and search with `git bisect`.
How does it work?	Git writes your commits into a to-do file and opens it in your editor. After you save and close it, Git moves your branch back to the base and works through the list line by line: `pick` re-applies a commit, `squash` and `fixup` merge it into the one before, `edit` pauses so you can change it, `drop` skips it. Every replayed commit gets a new ID.
When to use it?	Before asking for review (or before a rebase merge), on a branch that only you work on.
Example	Chen has eight commits: three real steps and five "fix typo" commits. One interactive rebase turns them into three clean commits for the reviewers.

Your first draft of commits usually looks like "wip", "fix", "actually fix", "typo". Interactive rebase turns that into something worth reviewing. It only changes your own branch.

Start one

```
git fetch
git rebase -i origin/main
git rebase -i HEAD~4
```

WHAT EACH LINE DOES

`git fetch`	Get the newest `main` from GitHub first.
`git rebase -i origin/main`	Opens an editable list of every commit on your branch that isn't on `main` yet. `-i` means interactive.
`git rebase -i HEAD~4`	Alternative: open a list of only your last four commits. Use one of the two.

Your editor opens a to-do list, with the oldest commit at the top:

```
pick 3f2a1c9 feat: add login form markup
pick 8b4d0e2 wip
pick c71e9aa fix typo in label
pick 19ad3f0 feat: validate email on submit
pick 5e0b7d1 add tests
```

Each line is one commit: a command, the short ID, and the message. Change the commands (and move lines up or down to reorder), then save and close the file:

```
pick 3f2a1c9 feat: add login form markup
fixup 8b4d0e2 wip
fixup c71e9aa fix typo in label
pick 19ad3f0 feat: validate email on submit
squash 5e0b7d1 add tests
```

Git now replays the commits following your instructions. "wip" and "fix typo" are folded into the form commit, and "add tests" is merged into the validation commit. Because of `squash`, Git opens the editor once more so you can write the combined message for those two. The result is two clean commits. Push with `git push --force-with-lease`.

Command	Short	What it does
`pick`	`p`	Keep the commit as it is
`reword`	`r`	Keep it, but stop to let you edit the message
`edit`	`e`	Stop after this commit so you can change it (amend it, split it)
`squash`	`s`	Combine with the commit above and merge both messages
`fixup`	`f`	Combine with the commit above and throw this message away
`fixup -C`		Combine with the commit above and use this commit's message instead
`drop`	`d`	Remove the commit. Deleting the line does the same
`exec`	`x`	Run a terminal command at this point in the list
`break`	`b`	Stop here. Continue with `git rebase --continue`

If something goes wrong in the middle, `git rebase --abort` puts everything back exactly as it was before you started.

The fixup workflow

AT A GLANCE **Fixup commits and autosquash**	
What is it?	A commit marked to be folded into an earlier commit later, named "fixup!" followed by the earlier commit's message.
Why use it?	You can fix a mistake in an older commit right away and let Git do the reordering, instead of moving lines around by hand.
How does it work?	`git commit --fixup <sha>` copies the target commit's subject line and adds "fixup! " in front. During `git rebase -i`, autosquash finds commits whose subject matches, moves each one directly under its target, and marks it `fixup`.
When to use it?	When review feedback, or your own discovery, concerns an earlier commit on your branch.
Example	A reviewer notices that Alice's second commit doesn't reject empty emails. Alice fixes it, commits with `git commit --fixup <id of commit 2>`, and autosquash folds the fix into that commit.

A reviewer finds a bug in an earlier commit on your branch. Instead of adding a "fix review comment" commit at the end, aim the fix at the commit it belongs to:

```
git log --oneline origin/main..
git add -p
git commit --fixup 19ad3f0
git rebase -i origin/main
```

WHAT EACH LINE DOES

`git log --oneline origin/main..`	Lists the commits on your branch that aren't on `main`, so you can find the one to fix, say `19ad3f0`.
`git add -p`	After editing the code, stage the fix.
`git commit --fixup 19ad3f0`	Creates a commit called "fixup! feat: validate email on submit", marked to be folded into `19ad3f0`.
`git rebase -i origin/main`	Opens the to-do list. With `rebase.autoSquash` on (Chapter 7), the fixup is already moved under its target and marked `fixup`. Just save and close.

Variants: `git commit --fixup=amend:<sha>` also replaces the target's message, and `--fixup=reword:<sha>` changes only its message.

Split a commit

```
git rebase -i origin/main
git reset HEAD~
git add -p
git commit -m "refactor: extract validateEmail()"
git add -p
git commit -m "fix: reject emails with trailing spaces"
git rebase --continue
```

WHAT EACH LINE DOES

`git rebase -i origin/main`	In the to-do list, change `pick` to `edit` on the commit you want to split. Git stops right after that commit.
`git reset HEAD~`	Undoes that commit but keeps all its changes in your files, unstaged.
`git add -p` then `git commit -m ...`	Stage the first part and commit it.
The second `git add -p` and `git commit`	Stage the rest and commit it as a second commit.
`git rebase --continue`	Tells Git you're done; it replays the remaining commits.

Test every commit

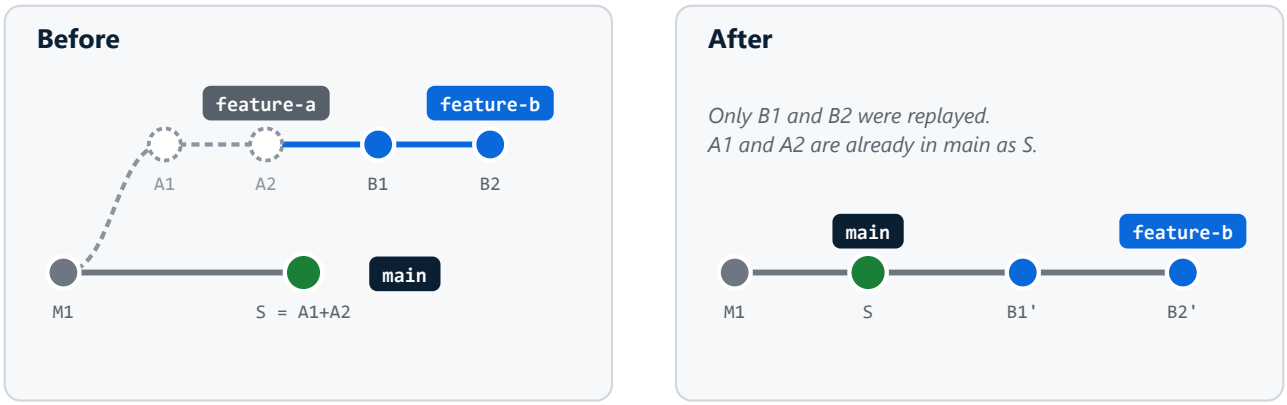
```
git rebase -i --exec "npm test" origin/main
```

WHAT EACH LINE DOES

`--exec "npm test"`	Runs your tests after each commit is replayed, and stops at the first commit where they fail, so you can fix it there. Replace `npm test` with your project's test command.

Move a branch to a new base with --onto

AT A GLANCE **git rebase --onto**	
What is it?	A rebase where you choose both the new starting point and exactly which commits to move.
Why use it?	To move a branch that was built on top of another feature branch, especially after that other branch was squash-merged and its original commits no longer match `main`.
How does it work?	`git rebase --onto <new-base> <old-base> <branch>` takes the commits that are in `<branch>` but not in `<old-base>`, and replays only those on top of `<new-base>`.
When to use it?	When your branch started from another branch that has since been squash-merged, or when you created a branch from the wrong place.
Example	Bob built `feat/b-ui` on top of Alice's `feat/a-api`. After Alice's PR is squash-merged, Bob runs `git rebase --onto origin/main feat/a-api feat/b-ui`, and Bob's PR shows only Bob's changes again.



git rebase --onto origin/main feature-a feature-b

A common team situation. You started `feature-b` on top of `feature-a` because you needed its code. Then `feature-a` was squash-merged into `main` as one new commit, S. Your branch still contains A1 and A2, and a normal rebase would try to apply them again on top of S, giving you conflicts with your own code. Tell Git to take only the commits that come after `feature-a`:

```
git fetch
git rebase --onto origin/main feature-a feature-b
git rebase --onto origin/main 1a2b3c4 feature-b
git push --force-with-lease
```

WHAT EACH LINE DOES

`git rebase --onto origin/main feature-a feature-b`	"Take the commits of `feature-b` that come after `feature-a`, and replay them onto `origin/main`." The three names are: new base, old base, branch to move.
`git rebase --onto origin/main 1a2b3c4 feature-b`	Use this form instead if the `feature-a` branch is already deleted: give the ID of its last commit (A2). Find it with `git log --oneline feature-b`.
`git push --force-with-lease`	Upload the moved branch.

Stacked branches and --update-refs

AT A GLANCE **--update-refs**	
What is it?	A rebase option that also moves any other branches pointing at commits inside the stack being rebased.
Why use it?	When branches are stacked on top of each other, rebasing the top one would otherwise leave the lower ones pointing at old commits.
How does it work?	Before replaying, Git notes which branches point at commits in the range. As it creates the new copies of those commits, it moves each of those branches to the matching new copy.
When to use it?	When you work with several branches stacked on top of each other.
Example	Chen has `feat/db`, `feat/api` and `feat/ui` stacked. After a change in `feat/db`, one rebase of `feat/ui` with `--update-refs` brings all three branches up to date.

If you keep a stack of branches (`feat-a`, then `feat-b` built on it, then `feat-c` built on that) and change something in `feat-a`, rebase the top of the stack with `--update-refs`. Every branch in the stack moves along with it. The `rebase.updateRefs` setting from Chapter 7 makes this automatic.

```
git switch feat-c
git rebase -i --update-refs origin/main
git push --force-with-lease origin feat-a feat-b feat-c
```

WHAT EACH LINE DOES

`git switch feat-c`	Go to the top branch of the stack.
`git rebase -i --update-refs origin/main`	Rebases the whole stack onto the newest `main`, moving `feat-a` and `feat-b` too.
`git push --force-with-lease origin feat-a feat-b feat-c`	Uploads all three branches at once.

Undo a rebase

```
git rebase --abort
git reset --hard ORIG_HEAD
git reflog
```

WHAT EACH LINE DOES

`git rebase --abort`	While a rebase is still in progress (stopped at a conflict or an `edit`): cancel it and go back to how things were.
`git reset --hard ORIG_HEAD`	Right after a rebase finished: jump back to the commit you were on before it. `ORIG_HEAD` is overwritten by the next reset, merge or rebase, so use it straight away.
`git reflog`	Later on: lists everywhere your branch has been, so you can find the commit from before the rebase (Chapter 19).

CHAPTER 15

Merge conflicts without panic

Why they happen, how to read them, and habits that prevent most of them.

Why conflicts happen

AT A GLANCE **Merge conflict**	
What is it?	A situation where Git can't combine two sets of changes by itself, because both changed the same lines in different ways.
Why does Git stop?	Guessing would silently break your code. Stopping and asking is the only safe choice, because only a person can know which version (or which mix of both) is right.
How does it work?	Git does a "three-way merge": it compares the common starting version (the base) with each side. Where only one side changed a section, Git takes that change. Where both changed the same section differently, it writes both versions into the file between markers and marks the file as unmerged until you resolve it.
When does it happen?	Most often after you rebase or merge `main` into your branch, or when two pull requests change the same lines of the same file.
Example	Alice changes a button's text to "Sign in" and Bob changes the same button to "Log in". Whoever merges second gets a conflict, and the two agree on the final wording.

Conflicts also happen when one side edited a file that the other side deleted. Everything else merges automatically. A conflict isn't an error and nothing is broken. It's Git asking for a decision that only a person who understands both changes can make.

> **In plain English**
>
> Two friends edit the same sentence in a shared essay, each in their own copy. When the copies are combined, someone has to read both versions and decide what the sentence should say. That's all a conflict is.

Read the markers

When a conflict happens, Git writes both versions into the file, surrounded by marker lines. With the `merge.conflictStyle zdiff3` setting from Chapter 7, it looks like this:

```
<<<<<<< HEAD
const TAX_RATE = 0.15;
||||||| base
const TAX_RATE = 0.1;
=======
const TAX_RATE = getTaxRate(country);
>>>>>>> feat/57-tax-by-country
```

HOW TO READ THE MARKERS

`<<<<<<< HEAD`	Start of the conflict. The lines below it are one side's version (which side depends on whether you're merging or rebasing; see the table further down).
`||||||| base`	The lines below it are the original text, before either side changed it.
`=======`	The lines below it are the other side's version.
`>>>>>>> feat/57-tax-by-country`	End of the conflict, labelled with the other side's name.

The middle "base" section is what makes `zdiff3` worth turning on. Here you can see that `main` changed the rate from 0.1 to 0.15, while the feature branch replaced the constant with a function. The right answer is probably to keep the function and make 0.15 its default value. Without the base section, you'd have to guess which side changed what.

To resolve it, edit the file so it contains exactly the code you want, and delete all four marker lines:

```
const TAX_RATE = getTaxRate(country);   // getTaxRate now defaults to 0.15
```

Resolve step by step

```
git status
git diff --name-only --diff-filter=U
git add src/cart.js
git rebase --continue
```

WHAT EACH LINE DOES

`git status`	Lists the conflicted files under "both modified" (or "Unmerged paths").
`git diff --name-only --diff-filter=U`	Prints just the names of the conflicted files. `U` stands for unmerged.
(edit each file)	Open each conflicted file, make it say what it should, and delete every marker line.
`git add src/cart.js`	Tells Git this file is resolved. Repeat for each file.
`git rebase --continue`	Carries on with the rebase. If you were merging, run `git merge --continue` instead; for a cherry-pick, `git cherry-pick --continue`.

Changed your mind or got lost? `git rebase --abort` or `git merge --abort` puts everything back to how it was before you started. Nothing is lost.

VS Code highlights conflicts with clickable **Accept Current**, **Accept Incoming** and **Accept Both** links above each one, and a **Resolve in Merge Editor** button that opens a three-pane view. To make VS Code the tool that `git mergetool` opens:

```
git config --global merge.tool vscode
git config --global mergetool.vscode.cmd 'code --wait --merge $REMOTE $LOCAL $BASE $MERGED'
git mergetool
```

WHAT EACH LINE DOES

`git config --global merge.tool vscode`	Names VS Code as your merge tool.
`git config --global mergetool.vscode.cmd '...'`	Tells Git how to start it: with the other side (`$REMOTE`), your side (`$LOCAL`), the original (`$BASE`) and the file to save the result in (`$MERGED`).
`git mergetool`	Opens each conflicted file in that tool, one after another.

Before you continue, run the app or the tests. A file without conflict markers can still be wrong: both sides might have added a function with the same name, or one side renamed something the other side started using.

"Ours" and "theirs" swap during a rebase

	During a merge	During a rebase
`--ours`, "Current", HEAD	Your branch	The branch you're rebasing onto, such as main
`--theirs`, "Incoming"	The branch being merged in	Your commit that's being replayed

A rebase works by starting from `main` and replaying your commits onto it one by one. From Git's point of view, `main` is "ours" and your commit is the visitor, "theirs". That's confusing the first time, so check the table whenever you choose a side. To take a whole file from one side:

```
git checkout --theirs src/cart.js
git checkout --ours src/cart.js
git add src/cart.js
```

WHAT EACH LINE DOES

`git checkout --theirs src/cart.js`	During a rebase: replace the conflicted file with *your* version.
`git checkout --ours src/cart.js`	During a rebase: replace it with *main's* version. (During a merge, the meanings are the other way round.)
`git add src/cart.js`	Mark the file as resolved.

Lock files and generated files

Lock files such as `package-lock.json`, `yarn.lock`, `pnpm-lock.yaml` or `poetry.lock` are written by programs, not people, and are almost impossible to merge by hand. Take `main`'s version and let the package manager rebuild it with your branch's dependencies:

```
git checkout --ours package-lock.json
npm install
git add package-lock.json
git rebase --continue
```

WHAT EACH LINE DOES

`git checkout --ours package-lock.json`	During a rebase, take main's version of the lock file. (During a merge, use `--theirs` to get main's version.)
`npm install`	Reads your `package.json` and updates the lock file so it also includes the packages your branch added.
`git add package-lock.json`	Mark it resolved.
`git rebase --continue`	Carry on.

Build output, minified files and compiled assets shouldn't be in Git at all. If they conflict, that's the sign to add them to `.gitignore`.

Let Git remember: rerere

AT A GLANCE **rerere (reuse recorded resolution)**	
What is it?	A Git feature that remembers how you resolved a conflict and reapplies that resolution if the same conflict appears again.
Why use it?	When you rebase a long-lived branch several times, you often hit identical conflicts. rerere saves you from solving them again.
How does it work?	When a conflict appears, Git saves the conflicted text. When you resolve it and continue, Git saves your result next to it in `.git/rr-cache`. The next time an identical conflict appears, Git applies the saved result automatically, and you only check and stage it.
When to use it?	When you rebase a long-lived branch again and again, or abort a merge and redo it.
Example	Bob's branch conflicts with `main` in `routes.js` every time Bob rebases. With rerere on, the conflict is resolved once and Git repeats that answer on every later rebase.

Turn it on with `rerere.enabled` (Chapter 7). When it reuses an answer, Git prints "Resolved 'src/cart.js' using previous resolution". Check the result and `git add` it. If a recorded answer was wrong, `git rerere forget <path>` deletes it.

Prevent most conflicts

- Keep pull requests small and merge them quickly. Conflicts grow with branch age.
- Rebase on `main` every day.
- Agree on a code formatter (Prettier, Black, Ruff, gofmt) and run it on save and in the automatic checks. Arguments about quotes and spacing cause a surprising share of conflicts.
- Before moving, renaming or reformatting a shared file, tell the team. Do it in its own small pull request and merge it fast.
- Break up files everyone edits, like one giant routes file or one huge CSS file.
- Add entries to lists in alphabetical order instead of always at the end. Two people adding to the end of the same list is a guaranteed conflict.

CHAPTER 16

Pull requests that get merged

Size, descriptions, linked issues, drafts, stacked PRs and gh shortcuts.

AT A GLANCE **Pull request**	
What is it?	A GitHub page that proposes merging one branch (the head) into another (the base). It shows the changes, a discussion, reviews and the results of automatic tests.
Why use it?	It gives the team one place to see, test, discuss and approve a change before it reaches `main`, and it permanently records why the change was made.
How does it work?	GitHub compares the head branch with the base branch from the point where they split, and shows the difference. Every push to the head branch updates the PR and re-runs the workflows. When it's merged, GitHub creates the merge, squash or rebased commits on the base branch.
When to use it?	For every change that goes into `main`, even a one-line fix.
Example	Alice fixes a typo on the homepage and still opens a PR. The tests run, Bob approves in a minute, and the history records why the text changed.

This chapter is about making pull requests that are quick to review and merge.

Keep them small

Aim for under 400 changed lines. A 2,000-line pull request gets "Looks good to me" from a reviewer who couldn't read it all. Split large work:

- by layer: the API first, then the screen that uses it;
- by refactor then feature: one PR moves code around without changing behavior, the next one changes behavior;
- or as a stack of dependent PRs (below).

A title and description reviewers can use

The title should follow your commit format, like `feat(auth): add password reset email`, because with squash merges it becomes the commit message on `main`. The description says what changed, why, how to test it, and shows screenshots for anything visual. The PR template from Chapter 8 prompts for all of this.

Link issues so they close themselves

AT A GLANCE **Closing keywords**	
What is it?	Words like `Closes #42` in a pull request description that link the PR to an issue.
Why use it?	The issue closes by itself when the work is merged, and anyone reading the issue can jump straight to the code that solved it.
How does it work?	GitHub reads the PR description for the keywords followed by an issue reference. It shows the link in both places, and when the PR is merged into the default branch it closes the linked issues.
When to use it?	In every pull request that finishes an issue.
Example	Bob's PR description says `Closes #18`. When the PR merges, issue 18 closes by itself and the project board moves it to Done.

Put a closing keyword and an issue number in the PR description, such as `Closes #42`.

- The keywords are close, closes, closed, fix, fixes, fixed, resolve, resolves and resolved.
- Each issue needs its own keyword: `Closes #42, closes #43`.
- An issue in another repository is written `Fixes my-org/api#7`.
- This only works when the PR is going into the default branch (`main`).
- To link an issue without closing it, write something like "Part of #40".

Reviewers, assignees and labels

- The **assignee** is the person responsible for getting the PR merged, usually its author.
- **Reviewers** are the people asked to review. CODEOWNERS adds some automatically.
- **Labels** and **milestones** help with planning (Chapter 22).

Try a teammate's PR on your computer

Reading the diff isn't always enough. For anything bigger than a small change, run it.

```
gh pr list
gh pr checkout 57
npm install && npm test
git switch -
```

WHAT EACH LINE DOES

`gh pr list`	Lists open pull requests with their numbers.
`gh pr checkout 57`	Downloads PR number 57 into a local branch and switches to it.
`npm install && npm test`	Install its dependencies, then (only if that worked) run the tests. Use your project's own commands.
`git switch -`	Go back to the branch you were on before.

Without `gh`, the same thing is `git fetch origin pull/57/head:pr-57` (download PR 57 into a branch called `pr-57`) followed by `git switch pr-57`.

Keep a PR up to date

The **Update branch** button on the PR page brings the latest `main` into the PR branch on GitHub, by merging or (from its drop-down) by rebasing. If you use the rebase option, your own copy of the branch is now out of date: run `git pull` before you continue working. With `pull.rebase` on (Chapter 7) it sorts itself out.

Stacked pull requests

AT A GLANCE **Stacked pull requests**	
What is it?	A chain of pull requests where each one is based on the branch of the previous one, instead of on `main`.
Why use it?	A big feature can be reviewed in small pieces without waiting for each piece to merge before starting the next.
How does it work?	Each PR only shows the difference between its own branch and its base branch, so reviewers see each layer separately. When the lowest PR merges and its branch is deleted, GitHub changes the next PR's base to `main`.
When to use it?	When a feature is too big for one pull request and the later parts depend on the earlier ones.
Example	Chen splits "user profiles" into PR 1 (database), PR 2 (API, based on PR 1) and PR 3 (page, based on PR 2), so reviewers deal with one small PR at a time.

Sometimes a feature is too big for one PR, and the second part depends on the first. Instead of waiting, you can stack them:

1. Create branch `feat/a-api` from `main` and open PR #1 into `main`.
2. Create branch `feat/b-ui` *from* `feat/a-api`, and open PR #2 with its **base** set to `feat/a-api`. Its diff then shows only the UI changes, so reviewers aren't shown the API changes twice.
3. Reviewers review #1 first. When #1 merges and its branch is deleted, GitHub automatically changes PR #2's base to `main`.
4. Because #1 was squash-merged, move #2 with `--onto` (Chapter 14) and force-push it.

```
git switch -c feat/b-ui feat/a-api
gh pr create --base feat/a-api --fill
git fetch
git rebase --onto origin/main feat/a-api feat/b-ui
git push --force-with-lease
```

WHAT EACH LINE DOES

`git switch -c feat/b-ui feat/a-api`	Create `feat/b-ui` starting from `feat/a-api` (not from `main`) and switch to it.
`gh pr create --base feat/a-api --fill`	Open PR #2 into `feat/a-api`, so its diff shows only what's new in `feat/b-ui`.
`git fetch`	Later, after PR #1 was squash-merged: download the new `main`.
`git rebase --onto origin/main feat/a-api feat/b-ui`	Replay only `feat/b-ui`'s own commits onto `main`, leaving out PR #1's commits, which `main` already contains.
`git push --force-with-lease`	Upload the moved branch.

Auto-merge and merge queues

AT A GLANCE **Auto-merge**	
What is it?	A pull request setting that tells GitHub to merge the PR by itself as soon as everything it needs is in place.
Why use it?	Nobody has to keep refreshing the page waiting for the last approval or the tests to finish.
How does it work?	Once auto-merge is enabled on a PR, GitHub watches it. When all required approvals are given and all required checks pass, it merges with the method you chose. If a new commit makes a check fail, it waits again.
When to use it?	When a pull request is approved, or about to be, and you're only waiting for the tests to finish.
Example	Alice turns on auto-merge and goes to dinner. Ten minutes later the tests pass and GitHub merges the PR by itself.

Turn it on with `gh pr merge --auto --squash` or the **Enable auto-merge** button on the PR. It needs **Allow auto-merge** in the settings and a ruleset with required checks (Chapters 8 and 9). Merge queues go a step further and test several PRs combined before merging them, which matters for repositories with dozens of merges a day. A team of three to five people doesn't need one.

gh commands for pull requests

Command	What it does
`gh pr create --fill`	Open a PR for the current branch using your commit messages
`gh pr create --draft --base main --title "..." --body-file pr.md`	Same, choosing everything yourself; the description is read from `pr.md`
`gh pr status`	Your PRs, and PRs waiting for your review
`gh pr list --search "review-requested:@me"`	Only the PRs waiting for your review
`gh pr view --web`	Open this branch's PR in the browser
`gh pr diff 57`	Read a PR's changes in the terminal
`gh pr checkout 57`	Download a PR to your computer
`gh pr checks --watch`	Follow the automatic tests until they finish
`gh pr review 57 --approve`	Approve. Also `--request-changes -b "..."` and `--comment -b "..."`
`gh pr comment 57 -b "..."`	Add a comment
`gh pr ready`	Turn a draft into a normal PR
`gh pr merge --squash --delete-branch`	Merge and clean up
`gh pr merge --auto --squash`	Merge automatically once everything is green

CHAPTER 17

Code review as a team

How to give and take review so it speeds you up instead of slowing you down.

AT A GLANCE **Code review**	
What is it?	A teammate reads a pull request's changes before they merge, then comments, approves, or asks for changes.
Why use it?	It catches bugs a single person misses, spreads knowledge of the code across the team, and keeps the code consistent.
How does it work?	Reviewers comment on lines in the PR's diff. Comments are grouped into a review that ends with a verdict. If the ruleset requires approvals, the merge button stays blocked until there are enough approvals and no unresolved "changes requested" review.
When to use it?	On every pull request before it merges. Reading other people's PRs is also one of the fastest ways to learn the codebase.
Example	Bob reviews Chen's login code and notices that passwords are printed to the server log. One comment prevents a security problem from reaching users.

Review catches bugs, but in a team of friends its bigger job is spreading knowledge. After a month, anyone should be able to work on any part of the code, and review is how that happens.

How to review on GitHub, step by step

1. Open the pull request and read the description and the linked issue, so you know what it's supposed to do.
2. Click the **Files changed** tab. Green lines were added, red lines were removed.
3. To comment on a line, hover over it and click the blue **+**. Drag across line numbers to comment on several lines.
4. Write your comment and click **Start a review** (not **Add single comment**), so all your comments go out together as one notification.
5. When you've gone through every file, click **Review changes** (top right), write a short summary, and choose **Comment**, **Approve** or **Request changes**. Then **Submit review**.

For the author

- Read your own diff on the **Files changed** tab before asking anyone. You'll catch a third of the comments yourself.
- Say what kind of feedback you want: "Mainly the database schema. The UI is a placeholder."
- Keep the PR small, and keep refactors in separate PRs.
- Answer every comment: fix it, or explain why not.

- Don't merge with unresolved conversations.
- If you go back and forth more than twice on one point, switch to a call, then write the decision in the PR.

For the reviewer

- Respond within one working day. A PR waiting three days blocks a person for three days.
- Look at things in this order: does it do what the issue asked; is it correct (unusual inputs, errors, empty and missing values); is it safe (input checks, secrets, permissions); is it tested; is it readable. Style comes last, and ideally a formatter handles it so nobody has to comment on it.
- For anything bigger than a small change, check it out and run it (`gh pr checkout 57`).
- Comment on the code, not the person: "This runs one query per item, so the page makes 100 queries" instead of "you wrote a slow loop".
- Say what's good, too. People repeat what gets noticed.

Label your comments

Starting each comment with a label tells the author what must change and what's optional:

Label	Means
`blocking:`	Must change before this merges
`suggestion:`	A better way, in my opinion. Author decides
`question:`	I don't understand this part. Explain, ideally in a code comment
`nit:`	Tiny and optional: naming, wording, ordering
`praise:`	This is good, keep doing it

Review tools on GitHub

- **Suggested changes:** put replacement code in a `suggestion` block in your comment. The author can apply it with one click (**Commit suggestion**), or batch several into one commit:

````
```suggestion
const total = items.reduce((sum, item) => sum + item.price, 0);
```
````

- Tick **Viewed** on each file as you finish it. If the author changes that file afterwards, the tick is removed so you know to look again.
- When indentation changed, hide whitespace-only changes: the gear icon on **Files changed**, or add `?w=1` to the end of the page address.
- Keyboard shortcuts: `?` lists them all, `t` opens a file finder, and `y` turns the current address into a permanent link to that exact version. Use permanent links when you point to code in issues, so the link keeps showing the right lines after the code changes.

Approve, comment or request changes

Choice	Means
Approve	Fine to merge, possibly after small nits
Comment	Feedback without a verdict
Request changes	Not ready. With required reviews turned on, this blocks merging until the reviewer approves or the review is dismissed

> **AI-written code**
>
> If someone on the team uses an AI coding assistant, the same rules apply. The author must understand and test every line before asking for review, keep the PR small, and say in the description which parts were generated. Review generated code at least as carefully as anything else. If your plan includes it, you can also request Copilot as a reviewer for a first pass. It doesn't replace a teammate's approval.

Reviewer checklist

- ☐ The change does what the linked issue asks, and nothing unrelated
- ☐ Unusual inputs handled: empty input, missing values, very large values, network errors
- ☐ No secrets, tokens or personal data in code, logs or test files
- ☐ User input is checked; database queries are parameterized (never built by gluing strings together)
- ☐ New behavior has tests, and the automatic checks are green
- ☐ Names say what things are; no dead code or leftover debug output
- ☐ README or docs updated if setup or behavior changed
- ☐ You ran it, if it's more than a trivial change

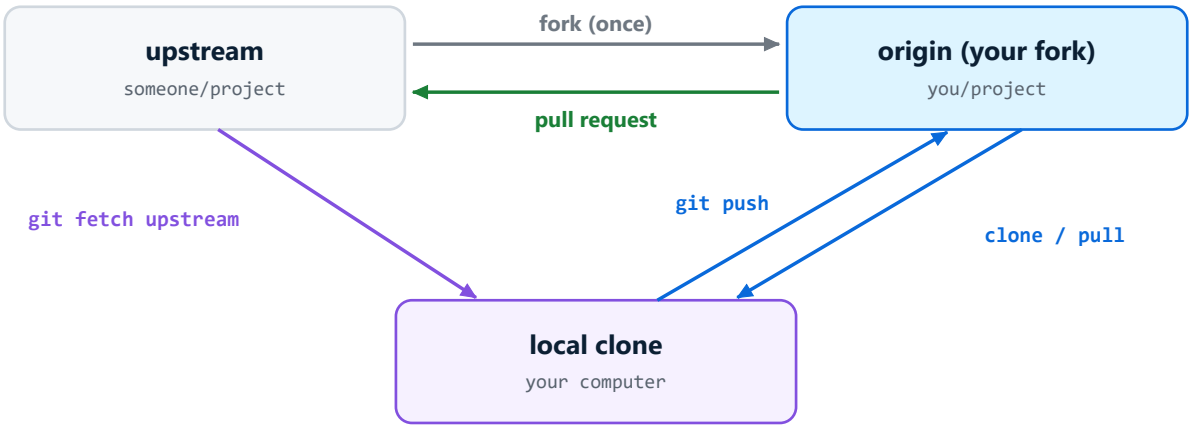
CHAPTER 18

Forks and upstream remotes

Contributing to repositories you can't push to.

AT A GLANCE **Fork**	
What is it?	Your own copy, on GitHub, of someone else's repository, which stays linked to the original.
Why use it?	You can contribute to projects where you don't have write access, or experiment freely without affecting the original.
How does it work?	GitHub copies the repository into your account and remembers which repository it came from. You push branches to your fork. A pull request from your fork asks the original's maintainers to bring your branch's commits into their repository.
When to use it?	When you don't have write access to a repository: contributing to open source, fixing something in a friend's personal project, or starting from a course template.
Example	Alice finds a typo in the documentation of a library the team uses. Alice forks it, fixes the typo, and opens a PR; the library's maintainers merge it.

In your team repository everyone has Write access and works with branches. A fork is for repositories where you *don't* have Write access: open-source projects, a friend's personal repository, a course template.



Three copies of the same repository. You push to your fork and open pull requests from it into upstream.

Set up

```
gh repo fork someone/project --clone
cd project
git remote -v
```

WHAT EACH LINE DOES

`gh repo fork someone/project --clone`	Creates your own copy (`you/project`) on GitHub and downloads it. It also sets up two remotes: `origin` (your fork) and `upstream` (the original).
`cd project`	Move into the downloaded folder.
`git remote -v`	Lists the remotes, so you can check. You should see the four lines below.

```
origin    git@github.com:you/project.git (fetch)
origin    git@github.com:you/project.git (push)
upstream  git@github.com:someone/project.git (fetch)
upstream  git@github.com:someone/project.git (push)
```

If you forked on the website instead, add the upstream remote yourself: `git remote add upstream git@github.com:someone/project.git`.

Keep your fork in sync

AT A GLANCE **The upstream remote**	
What is it?	A second remote, named `upstream`, that points at the original repository, next to `origin`, which points at your fork.
Why use it?	Your fork doesn't update itself when the original changes. `upstream` is how you get those changes.
How does it work?	`git fetch upstream` downloads the original's commits into `upstream/` pointers on your computer. You then merge them into your own branches and push them to your fork.
When to use it?	In every fork you use for more than one pull request.
Example	A week after forking, the original project has changed a lot. Before starting a new fix, Bob runs `git fetch upstream` and updates the fork's `main`.

The original keeps changing. Bring those changes into your fork regularly:

```
git fetch upstream
git switch main
git merge --ff-only upstream/main
git push origin main
```

WHAT EACH LINE DOES

`git fetch upstream`	Download the latest commits from the original repository.
`git switch main`	Go to your `main`.
`git merge --ff-only upstream/main`	Move your `main` forward to match the original. `--ff-only` refuses if your `main` has commits of its own, which would mean you committed to `main` by mistake (move them to a branch; see Chapter 19).
`git push origin main`	Update your fork on GitHub too.

The website shortcut: click **Sync fork** on your fork's GitHub page, then run `git pull` on your computer.

Contribute a change

```
git switch -c fix/typo-in-docs upstream/main
git push -u origin fix/typo-in-docs
gh pr create --repo someone/project --fill
```

WHAT EACH LINE DOES

`git switch -c fix/typo-in-docs upstream/main`	Create a new branch starting from the original's latest `main`, and switch to it. Make and commit your change here.
`git push -u origin fix/typo-in-docs`	Upload the branch to *your fork* (`origin`). `-u` links your local branch to it.
`gh pr create --repo someone/project --fill`	Open a pull request from your fork's branch into the original repository.

Leave **Allow edits by maintainers** ticked on the PR page, so the owners can push small fixes to your branch instead of asking you to. Read the project's CONTRIBUTING.md first. Many projects want an issue opened before a PR, a particular commit format, or signed commits.

> **Keep your fork's main clean**
>
> Never commit to `main` in your fork. Always branch. Otherwise every sync turns into a merge, and your PRs pick up commits that have nothing to do with them.

PART V

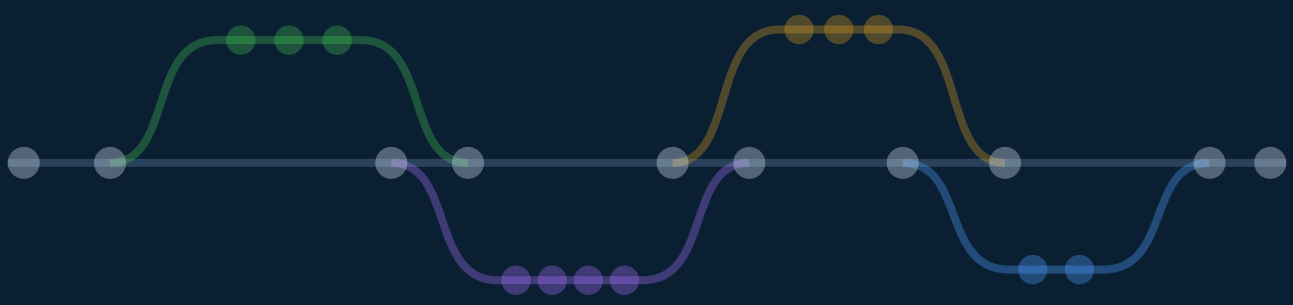
History and recovery

Undo almost anything, juggle several tasks at once, and find out where a bug came from.

19 Undo anything

20 Juggle work: stash, cherry-pick, worktrees

21 Investigate history



CHAPTER 19

Undo anything

A situation-by-situation guide to restore, reset, revert and the reflog.

Once something has been committed, Git almost never loses it. Uncommitted changes are the exception: `git restore`, `git reset --hard` and `git clean` can delete work that Git never saved. The safest habit is to commit often on your branch, even "wip" commits, and tidy up later with interactive rebase (Chapter 14).

Pick the right command

Find your situation in the left column. The sections after the table explain the commands in detail.

You want to	Run
Throw away unsaved edits to a file	`git restore <file>`
Unstage a file but keep the edits	`git restore --staged <file>`
Fix the last commit's message	`git commit --amend`
Add a forgotten file to the last commit	`git add <file>`, then `git commit --amend --no-edit`
Undo the last commit, keep its changes staged	`git reset --soft HEAD~1`
Undo the last commit, keep its changes as unstaged edits	`git reset HEAD~1`
Delete the last commit and its changes completely	`git reset --hard HEAD~1`
Undo a commit that's already on main	`git revert <sha>`
Undo a merged pull request	The **Revert** button on the merged PR, or `git revert <sha>` locally
Get an old version of one file back	`git restore --source=<sha> <file>`
Delete untracked files	`git clean -n` to preview, then `git clean -fd`
Bring back a deleted branch	Find its last commit in `git reflog`, then `git branch <name> <sha>`
Undo a rebase or reset you just did	`git reset --hard ORIG_HEAD`, or find the old commit in `git reflog`
Make your branch exactly match GitHub	`git fetch`, then `git reset --hard origin/<branch>`

WHAT THE LESS OBVIOUS COMMANDS MEAN

`git restore <file>`	Puts the file back to how it was in the last commit. Your unsaved edits to it are lost for good.
`git restore --source=<sha> <file>`	Puts the file back to how it was in commit `<sha>`. The change shows up as a normal edit you can commit.
`git clean -n` / `git clean -fd`	`-n` lists the untracked files (files Git has never seen) that would be deleted, without deleting anything. `-fd` really deletes them, folders included.
`git revert <sha>`	Creates a new commit that does the exact opposite of commit `<sha>`. History stays intact.

reset in three modes

AT A GLANCE **git reset**	
What is it?	A command that moves your current branch to a different commit, and optionally makes the staging area and your files match it.
Why use it?	To undo commits that exist only on your computer, to unstage files, or to start over from a known state.
How does it work?	Git rewrites the branch pointer to the target commit. With `--soft` it stops there. With `--mixed` it also replaces the staging area with the target's snapshot. With `--hard` it also overwrites the files in your folder.
When to use it?	To undo commits you haven't pushed, to unstage files, or to throw away an experiment completely.
Example	Chen commits twice on `main` before remembering to create a branch. `git branch feat/x` followed by `git reset --hard origin/main` moves both commits onto a new branch.

`git reset <commit>` moves your current branch back to that commit. The mode decides what happens to the work in between:

Mode	Branch moves	Staging area	Files on disk
`--soft`	Yes	Kept	Kept
`--mixed` (the default)	Yes	Reset to match	Kept
`--hard`	Yes	Reset to match	Reset to match. Uncommitted changes are gone

> **In plain English**
>
> Imagine taking the last few pages out of a notebook. `--soft` keeps the torn-out pages clipped to the front, ready to put back. `--mixed` keeps them loose on your desk. `--hard` throws them in the bin.

Use reset for commits that exist only on your computer, or on a branch only you use. For commits that are already on a shared branch, use revert instead.

reset --hard and clean -f

Both delete uncommitted work, and the reflog can't bring it back, because it was never committed. Run `git status` first. When in doubt, run `git stash -u` before you reset; a stash can be recovered.

revert: undo by adding a commit

AT A GLANCE **git revert**	
What is it?	A command that creates a new commit which undoes the changes of an earlier commit.
Why use it?	It's the safe way to undo something on a shared branch like `main`, because it adds history instead of rewriting it, so nobody's copy breaks.
How does it work?	Git calculates what the earlier commit changed, applies the exact opposite to your current files, and saves that as a new commit with a message saying what it reverts.
When to use it?	When a bad change is already on `main` or another shared branch.
Example	A merged PR breaks the checkout page. Alice clicks **Revert** on that PR, merges the revert PR, and the site works again within minutes, with the whole story still visible in history.

```
git revert 4e1d2c7
git revert -m 1 9a8b7c6
git revert --no-commit 4e1d2c7..8f3a0b1
git commit -m "revert: back out the new checkout flow"
```

WHAT EACH LINE DOES

`git revert 4e1d2c7`	Creates a new commit that undoes commit `4e1d2c7`. Git opens the editor with a ready-made message; save and close.
`git revert -m 1 9a8b7c6`	Undoes a *merge* commit. `-m 1` says "keep parent number 1", which is the `main` side.
`git revert --no-commit 4e1d2c7..8f3a0b1`	Undoes every commit after `4e1d2c7` up to and including `8f3a0b1`, but doesn't commit yet.
`git commit -m "..."`	Saves all of those undos as one commit.

With squash merges, each pull request is one ordinary commit, so `git revert <sha>` undoes the whole PR. You'd still do this on a branch and open a PR, because the ruleset protects `main`.

Common accidents

I committed to main instead of a branch

The ruleset will reject your push anyway. Move the commits to a branch before doing anything else. If you have uncommitted changes, commit or stash them first, because `reset --hard` would delete them:

```
git branch feat/my-work
git reset --hard origin/main
git switch feat/my-work
```

WHAT EACH LINE DOES

`git branch feat/my-work`	Creates a new branch pointing at your current commit, so your commits now also belong to `feat/my-work`. It doesn't switch to it.
`git reset --hard origin/main`	Moves your `main` back to match GitHub's `main`. Your commits are safe on the new branch.
`git switch feat/my-work`	Switch to the new branch and carry on. Push it and open a PR as usual.

I committed to the wrong branch

```
git log --oneline -3
git switch right-branch
git cherry-pick 7c3d9e1
git switch wrong-branch
git reset --hard HEAD~1
```

WHAT EACH LINE DOES

`git log --oneline -3`	Shows the last three commits. Note the ID of the misplaced one, say `7c3d9e1`.
`git switch right-branch`	Go to the branch where the commit should be.
`git cherry-pick 7c3d9e1`	Copy that commit onto this branch.
`git switch wrong-branch`	Go back to the wrong branch.
`git reset --hard HEAD~1`	Remove the commit from it. Only do this if you haven't pushed it yet.

I deleted a branch that had unmerged work

When you delete a branch, Git prints its last commit, like `Deleted branch feat/x (was a1b2c3d).` That ID is all you need:

```
git branch feat/x a1b2c3d
```

WHAT EACH LINE DOES

`git branch feat/x a1b2c3d`	Creates a branch called `feat/x` pointing at commit `a1b2c3d`. Everything is back.

If the message has scrolled away, look in `git reflog` for the last `commit:` entry you made on that branch, and use its ID.

I ran reset --hard and lost commits

The commits still exist. Find them in the reflog (next section) and point a branch at them, or `git reset --hard` back to them.

I pushed a bad commit to my own feature branch

Either add a fixing commit (or a `git revert`), or, because nobody else uses the branch, remove it: `git reset --hard HEAD~1`, then `git push --force-with-lease`.

The reflog: your safety net

AT A GLANCE **Reflog**	
What is it?	A private log, on your computer only, of every position HEAD and each branch has had.
Why use it?	It's how you get back commits after a bad reset, a rebase gone wrong, or a deleted branch.
How does it work?	Every time HEAD or a branch moves, Git appends a line (old commit, new commit, reason) to a file in `.git/logs`. `git reflog` shows that file. Commits that no branch points to are kept in the repository until Git's cleanup removes them after the expiry period, so the reflog can lead you back to them.
When to use it?	When commits seem to have vanished after a reset, a rebase or a deleted branch.
Example	Bob runs `git reset --hard HEAD~3` by mistake. `git reflog` shows where the branch was a moment ago, and `git reset --hard HEAD@{1}` brings all three commits back.

Even commits that no branch points to any more can be found there.

```
git reflog
git reset --hard HEAD@{3}
git reflog show feat/42-login
```

WHAT EACH LINE DOES

`git reflog`	Lists where HEAD has been, newest first (example output below).
`git reset --hard HEAD@{3}`	Jumps back to where HEAD was three moves ago. In the example, that's just before the rebase started.
`git reflog show feat/42-login`	The same list, but only for one branch.

```
4f6a2b1 HEAD@{0}: rebase (finish): returning to refs/heads/feat/42-login
9d8c7b6 HEAD@{1}: rebase (pick): feat: validate email
e5f4a3b HEAD@{2}: rebase (start): checkout origin/main
2c1b0a9 HEAD@{3}: commit: feat: add login form markup
```

Each line shows the commit ID, its position (`HEAD@{0}` is now, `HEAD@{1}` one move ago, and so on) and what happened. Read from the bottom up to follow the story: you made a commit, started a rebase, replayed a commit, and finished.

The reflog lives only on your computer; it isn't pushed. Entries expire after 90 days, or after 30 days for commits no branch can reach. And it only knows about commits, never about changes you didn't commit.

CHAPTER 20

Juggle work: stash, cherry-pick, worktrees

Switch tasks without losing your place.

You're halfway through a feature when a friend asks you to look at their PR, or an urgent bug appears. These three tools let you switch tasks without making a mess.

Stash

AT A GLANCE **Stash**	
What is it?	A stack of saved, uncommitted changes that you can put aside and bring back later.
Why use it?	To switch tasks, pull, or rebase without committing half-done work.
How does it work?	`git stash` saves your staged and unstaged changes (and untracked files with `-u`) as special hidden commits, records them in a list, and then resets your files to the last commit. `pop` re-applies the saved changes and removes them from the list.
When to use it?	When you need a clean working folder for a short interruption: switching branches, pulling, or reviewing someone's work.
Example	Chen is in the middle of an edit when Alice asks for an urgent review: `git stash`, `gh pr checkout 21`, review, `git switch -`, `git stash pop`.

> **In plain English**
>
> A stash is a drawer. You sweep your unfinished work off the desk into the drawer, deal with the interruption, then pull the work back out.

```
git stash push -m "half-done validation"
git stash push -u -m "with new files"
git stash push --staged -m "only staged"
git stash push -m "just cart" -- src/cart.js
git stash list
git stash show -p stash@{1}
git stash pop
git stash apply stash@{1}
git stash drop stash@{1}
git stash branch feat/from-stash stash@{0}
```

WHAT EACH LINE DOES

`git stash push -m "half-done validation"`	Saves your changes to tracked files in a stash with a label, and returns your files to the last commit.
`git stash push -u -m "..."`	Same, but also includes brand-new (untracked) files.
`git stash push --staged -m "..."`	Stashes only what's in the staging area.
`git stash push -m "..." -- src/cart.js`	Stashes only that file. The `--` separates options from file names.
`git stash list`	Lists your stashes. The newest is `stash@{0}`, the one before `stash@{1}`, and so on.
`git stash show -p stash@{1}`	Shows the changes inside a stash.
`git stash pop`	Puts the newest stash back into your files and removes it from the list.
`git stash apply stash@{1}`	Puts a stash back but keeps it in the list.
`git stash drop stash@{1}`	Deletes a stash.
`git stash branch feat/from-stash stash@{0}`	Creates a new branch at the commit where you stashed, and applies the stash there. Useful when `pop` gives a conflict.

Stashes are easy to forget, and they're never pushed to GitHub. If something will sit for more than a day, commit it on a branch instead.

Cherry-pick

AT A GLANCE **Cherry-pick**	
What is it?	Copying the changes of one specific commit onto the branch you're on.
Why use it?	To bring a single fix into another branch, like a release branch, without merging everything else from where it came from.
How does it work?	Git calculates what the chosen commit changed compared with its parent, applies those changes to your current files (which can cause a conflict), and creates a new commit with the same message but a new ID.
When to use it?	To copy one specific fix to another branch, such as a release branch, or to rescue a good commit from a branch you're abandoning.
Example	A security fix was merged into `main` but must also go out in version 1.0. `git cherry-pick -x <sha>` copies only that fix onto `release/1.0`.

```
git cherry-pick 7c3d9e1
git cherry-pick -x 7c3d9e1
git cherry-pick 4e1d2c7^..8f3a0b1
git cherry-pick --no-commit 7c3d9e1
git cherry-pick --continue
git cherry-pick --abort
```

WHAT EACH LINE DOES

`git cherry-pick 7c3d9e1`	Copies commit `7c3d9e1` onto the current branch.
`git cherry-pick -x 7c3d9e1`	Same, and adds "(cherry picked from commit ...)" to the message, so people can trace where it came from.
`git cherry-pick 4e1d2c7^..8f3a0b1`	Copies a range of commits, from `4e1d2c7` to `8f3a0b1`, both included. The `^` makes the first one included.
`git cherry-pick --no-commit 7c3d9e1`	Applies the changes to your files without committing, so you can adjust them first.
`git cherry-pick --continue`	After fixing a conflict and running `git add`, carry on.
`git cherry-pick --abort`	Cancel and go back to how things were.

Good uses: a fix that must go into `main` and into a release branch, or rescuing one good commit from a branch you're abandoning. Don't use it instead of merging branches. Copies of the same change with different IDs make history confusing and cause conflicts later.

Worktrees: two branches checked out at once

AT A GLANCE **Worktree**	
What is it?	An additional working folder attached to the same repository, with a different branch checked out.
Why use it?	You can work on two branches at the same time, for example an urgent fix and your feature, without stashing or cloning the repository again.
How does it work?	All worktrees share one set of commits and objects in the main `.git` folder. Each extra folder has its own HEAD, staging area and files, plus a small `.git` file that points back to the main repository. Git won't check out the same branch in two worktrees at once.
When to use it?	When you need two branches at the same time for longer than a quick stash: an urgent fix during a big feature, or reviewing while you build.
Example	Bob is halfway through a big refactor when the live site breaks. `git worktree add -b fix/crash ../app-hotfix origin/main` gives a clean second folder for the fix, and the refactor stays untouched.

You can fix an urgent bug without stashing anything, and review a friend's PR without touching your half-done work.

```
git worktree add -b fix/88-crash ../my-app-hotfix origin/main
cd ../my-app-hotfix
cd ../my-app
git worktree remove ../my-app-hotfix
git worktree add --detach ../my-app-review
cd ../my-app-review
gh pr checkout 57
git worktree list
git worktree prune
```

WHAT EACH LINE DOES

`git worktree add -b fix/88-crash ../my-app-hotfix origin/main`	Creates a new folder `my-app-hotfix` next to your project, with a new branch `fix/88-crash` starting from the latest `main`.
`cd ../my-app-hotfix`	Go into it. Fix the bug, commit, push and open a PR as usual. Your original folder is untouched.
`cd ../my-app`	Go back to your main folder when done.
`git worktree remove ../my-app-hotfix`	Delete the extra folder once its work is pushed.
`git worktree add --detach ../my-app-review`	Creates another folder for reviewing, not on any branch yet.
`cd ../my-app-review` then `gh pr checkout 57`	Go into it and load PR 57 there.
`git worktree list`	Lists all your worktree folders.
`git worktree prune`	Cleans up Git's records of worktree folders you deleted by hand.

Each worktree is a full copy of the files, so it needs its own `npm install` or virtual environment. One branch can only be checked out in one worktree at a time.

CHAPTER 21

Investigate history

log, diff, blame and bisect: what changed, who changed it, and when a bug arrived.

log filters you'll actually use

AT A GLANCE Searching history with git log	
What is it?	Options for `git log` that filter commits by file, author, date, message, or by the code they changed.
Why use it?	To answer questions like "when did this break?", "who added this setting?" or "what changed in this file last week?" in seconds.
How does it work?	`git log` walks backwards from a commit through its parents. Each filter decides which commits to show. The pickaxe (`-S`) is clever: for every commit it counts how often your text appears before and after, and shows only the commits where that number changed.
When to use it?	Whenever you need to know when, why or by whom something changed.
Example	The tax rate in the shop is suddenly wrong. `git log -S "TAX_RATE"` finds the one commit that changed it, and its PR explains why.

`git log` shows commit history. These options narrow it down to what you're looking for:

```
git log --oneline --graph --all
git log -p src/cart.js
git log --follow -p src/cart.js
git log --stat -5
git log --author="alice" --since="2 weeks ago"
git log -i --grep="login"
git log -S "TAX_RATE"
git log -G "fetch\(.*api"
git log -L :calculate_total:src/cart.py
git log -L 40,60:src/cart.js
git log main..feat/42-login
git log --first-parent main
git shortlog -sn
```

WHAT EACH LINE DOES

`git log --oneline --graph --all`	The whole commit graph, one line per commit.
`git log -p src/cart.js`	Every commit that changed this file, with the actual changes (`-p` means "show patch").
`git log --follow -p src/cart.js`	Same, but keeps going back past the point where the file was renamed.
`git log --stat -5`	The last five commits, with which files each changed and how many lines.
`git log --author="alice" --since="2 weeks ago"`	Only Alice's commits from the last two weeks.
`git log -i --grep="login"`	Commits whose message contains "login". `-i` ignores upper/lower case.
`git log -S "TAX_RATE"`	Commits that added or removed the text `TAX_RATE` somewhere in the code.
`git log -G "fetch\(.*api"`	Commits whose changes match a pattern (a regular expression).
`git log -L :calculate_total:src/cart.py`	The history of just one function.
`git log -L 40,60:src/cart.js`	The history of lines 40 to 60 of a file.
`git log main..feat/42-login`	Commits on the branch that aren't on `main` yet.
`git log --first-parent main`	Only the commits made or merged directly on `main`, one per merged PR.
`git shortlog -sn`	Number of commits per person, highest first.

`-S` (called the "pickaxe") is the fastest way to answer "when did this function disappear?" or "who added this setting?".

diff between anything

```
git diff
git diff --staged
git diff main...feat/42-login
git diff HEAD~3 -- src/
git diff --word-diff
git show 4e1d2c7 --stat
git grep -n "TODO"
```

WHAT EACH LINE DOES

`git diff`	Changes you've made but haven't staged.
`git diff --staged`	Changes you've staged for the next commit.
`git diff main...feat/42-login`	Everything the branch changed since it split from `main`: exactly what its PR shows.
`git diff HEAD~3 -- src/`	Changes over the last three commits, only inside the `src` folder.
`git diff --word-diff`	Shows changed words instead of whole lines. Good for text and documentation.
`git show 4e1d2c7 --stat`	One commit: its message and the files it changed.
`git grep -n "TODO"`	Searches all tracked files for "TODO" and shows line numbers. Faster than a normal search because it skips ignored files.

blame, the useful way

AT A GLANCE **git blame**	
What is it?	A line-by-line view of a file showing which commit, author and date last changed each line.
Why use it?	To find the commit, and through it the pull request, that explains why a line of code looks the way it does.
How does it work?	Git starts from the current file and walks back through history, assigning each line to the most recent commit that introduced it. `-C` also detects lines that were moved or copied from other files, and an ignore-revs file makes Git skip listed commits and keep looking further back.
When to use it?	When a line of code surprises you and you want to know the reason behind it before changing it.
Example	Alice finds a strange 300 ms delay in the code. Blame points to Chen's commit `fix: wait for animation before focusing input (#44)`, so Alice leaves it in place.

```
git blame -w -C -C -C src/cart.js
git blame -L 40,60 src/cart.js
```

WHAT EACH LINE DOES

`git blame src/cart.js`	For every line of the file: the commit, author and date of the last change to it.
`-w`	Ignore changes that only touched spaces or indentation.
`-C -C -C`	Follow code that was moved or copied from other files, so you see who originally wrote it.
`-L 40,60`	Only lines 40 to 60.

Once blame shows you the commit, `git show <sha>` and the pull request that commit came from tell you *why*. On GitHub, open a file and click **Blame**. The small icon next to each block shows the blame from just before that change, so you can keep walking back in time.

One big formatting commit ("ran Prettier on everything") makes blame point at that commit for every line. List such commits in a file, and Git skips them:

```
.git-blame-ignore-revs
```

```
# chore: run prettier on the whole codebase
3e5a1f0c9b8d7e6f5a4b3c2d1e0f9a8b7c6d5e4f
```

```
git config blame.ignoreRevsFile .git-blame-ignore-revs
```

WHAT EACH LINE DOES

`.git-blame-ignore-revs`	A file with one full 40-character commit ID per line. Lines starting with `#` are comments.
`git config blame.ignoreRevsFile .git-blame-ignore-revs`	Tells your Git to skip those commits in blame. Each teammate runs it once. GitHub's blame view reads the file automatically.

bisect: find the commit that broke it

AT A GLANCE **git bisect**	
What is it?	A binary search through history that finds the exact commit that introduced a bug.
Why use it?	It's far faster than checking commits one by one: about 10 tests for 1,000 commits.
How does it work?	Git keeps a range between a known good commit and a known bad commit and checks out the one in the middle. Your "good" or "bad" answer throws away half the range. This repeats until one commit is left. With `git bisect run`, Git uses your command's exit code as the answer.
When to use it?	When something used to work and now doesn't, and you don't know which change broke it.
Example	Search worked in v1.2 but not today, 60 commits later. `git bisect` finds the guilty commit in six steps.

Something worked last week and is broken today, and there are 200 commits in between. Here's how to find the guilty one.

> **In plain English**
>
> It's the number-guessing game. "Is it higher or lower than 500?" "Lower." "Than 250?" Each answer cuts the possibilities in half.

```
git bisect start
git bisect bad
git bisect good v1.2.0
git bisect good
git bisect reset
```

WHAT EACH LINE DOES

`git bisect start`	Begin a search.
`git bisect bad`	"The commit I'm on now is broken."
`git bisect good v1.2.0`	"This older version worked." Use a tag, a commit ID or a branch. Git now checks out a commit halfway between.
`git bisect good` (or `bad`, or `skip`)	Test the app, then tell Git whether this commit is good or bad (or `skip` if it can't be tested). Git jumps to the next halfway point. Repeat until Git prints "<sha> is the first bad commit".
`git bisect reset`	End the search and go back to where you started.

If a command can tell good from bad, Git can do every step for you:

```
git bisect start HEAD v1.2.0
git bisect run npm test
```

WHAT EACH LINE DOES

`git bisect start HEAD v1.2.0`	Start, with the current commit as bad and `v1.2.0` as good, in one line.
`git bisect run npm test`	Runs `npm test` on each halfway commit. If it succeeds (exit code 0), the commit counts as good; if it fails (exit codes 1 to 127, except 125), bad; exit code 125 means skip.

If the full test suite is slow, write a tiny script that checks only the bug and run that instead. Drill 4 in Chapter 37 lets you practice this.

Search on GitHub

The search box at the top of GitHub understands filters:

- Code: `repo:my-org/my-app TAX_RATE` searches one repository; add `path:src/` or `language:python` to narrow it.
- Pull requests and issues: `is:pr is:open review-requested:@me` (PRs waiting for your review), `is:issue is:open no:assignee label:bug` (unclaimed bugs), `is:pr is:merged author:alice merged:>2026-09-01` (Alice's PRs merged since September 1st).

PART VI

Planning, automation and quality

Plan the work, let robots check it, and keep secrets out of Git.

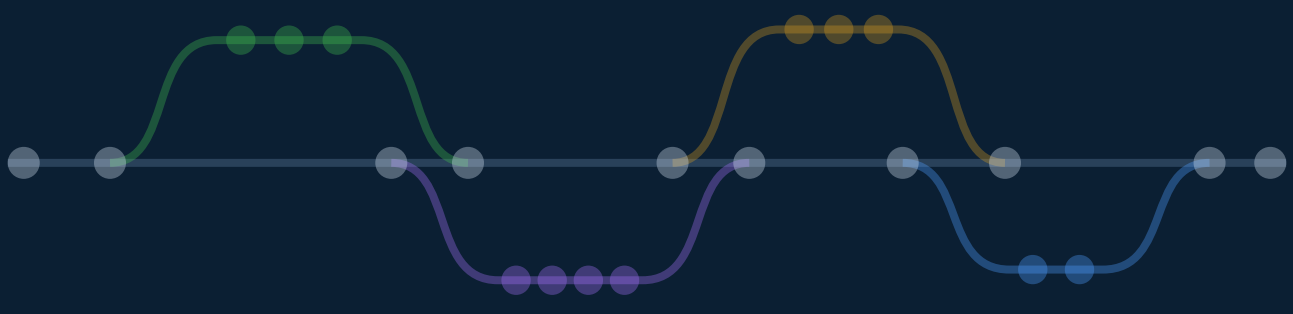
22 Plan the work with Issues and Projects

23 GitHub Actions: CI that guards main

24 Deploy from main

25 Automate code quality

26 Security for small teams



CHAPTER 22

Plan the work with Issues and Projects

Turn "let's build an app" into a board of small, assigned tasks.

Without a shared list of tasks, a friends' project drifts: two people build the same thing, nobody builds the boring parts, and nobody knows how close you are to done. GitHub Issues and Projects solve that, right next to the code.

Write issues someone can finish

AT A GLANCE **Issue**	
What is it?	An item in a repository's task list: one bug, feature or task, with a description, discussion, assignees, labels and a status (open or closed).
Why use it?	The whole team shares one visible list of work that lives right next to the code, so nobody builds the same thing twice and nothing is forgotten.
How does it work?	Each issue gets a number. Writing `#42` anywhere on GitHub links to issue 42. Pull requests that say `Closes #42` close it when they merge. Labels, milestones and assignees let you filter the list.
When to use it?	For every bug, feature, task or question that needs someone to act on it.
Example	Bob notices the footer overlaps the content on phones. Instead of mentioning it in chat (where it would be forgotten), Bob opens issue #27 with a screenshot.

A good issue has:

- a title that starts with a verb and names the result ("Add password reset by email");
- enough context that someone else could pick it up without asking you;
- **acceptance criteria**: a checklist that says exactly when it's done;
- a size of about two days of work or less. Bigger tasks get split into sub-issues.

```
Title: Add password reset by email

As a user who forgot my password, I want a reset link by email
so I can get back into my account.

Acceptance criteria
- [ ] "Forgot password?" link on the login page
- [ ] Reset email arrives within a minute; the link expires after 30 minutes
- [ ] Using a link twice shows "This link was already used"
- [ ] Tests cover token expiry and reuse

Design: link to the Figma frame
```

To create one: open the repository → **Issues** → **New issue**, pick a form (Chapter 8) and fill it in. From the terminal: `gh issue create --title "Add password reset by email" --label feature`, which then asks for the description.

Labels

Labels are colored tags on issues and PRs, used for filtering. Keep the set small, or nobody uses them consistently:

Label	For
`bug`, `feature`, `docs`, `chore`	Kind of work. Organizations can use issue types for this instead
`priority: high`, `priority: low`	What to pick first
`blocked`	Waiting on something. Say what in a comment
`good first issue`	Small, well-described tasks for someone new to the code
`needs discussion`	Not ready to start yet

```
gh label create "priority: high" --color D93F0B --description "Do this first"
gh label create blocked --color 000000
gh label list
gh label clone my-org/old-project
```

WHAT EACH LINE DOES

`gh label create "priority: high" --color D93F0B --description "..."`	Creates a label with a name, a color (as a hex code, without `#`) and a description shown on hover.
`gh label create blocked --color 000000`	A black `blocked` label. Quotes are only needed when a name contains spaces.
`gh label list`	Lists all labels in the repository.
`gh label clone my-org/old-project`	Copies every label from another repository into this one.

Milestones and sub-issues

AT A GLANCE Milestones and sub-issues	
What is it?	A milestone is a group of issues with a due date. A sub-issue is a smaller issue attached to a bigger parent issue.
Why use it?	Milestones show how close you are to a goal like "v1.0". Sub-issues break big work into pieces that one person can finish in a day or two.
How does it work?	Each issue can belong to one milestone; the milestone page counts open and closed issues to draw its progress bar. A parent issue lists its sub-issues and shows how many of them are closed.
When to use it?	Milestones for deadlines and versions; sub-issues for anything that's more than a couple of days of work.
Example	For demo day on 1 December, the team puts 20 issues into the milestone "Demo day". "User accounts" becomes a parent issue with four sub-issues, one for each screen.

Create a milestone, such as "MVP", "v1.0" or "Demo day", under **Issues** → **Milestones** → **New milestone**, or from the terminal:

```
gh api repos/my-org/my-app/milestones -f title="MVP" -f due_on="2026-12-01T00:00:00Z"
gh issue edit 42 --milestone "MVP"
```

WHAT EACH LINE DOES

`gh api repos/my-org/my-app/milestones -f title="MVP" -f due_on="..."`	Creates a milestone called MVP with a due date. The date format is year-month-day, then `T`, the time, and `Z` for UTC.
`gh issue edit 42 --milestone "MVP"`	Puts issue 42 into that milestone.

To add sub-issues, open the parent issue and use **Create sub-issue** or **Add existing issue**. The parent then shows how many of its children are done. Use this for anything bigger than a couple of days, like "User accounts" split into sign-up, login, password reset and profile page.

Projects: your team board

AT A GLANCE GitHub Projects	
What is it?	Boards, tables and roadmaps built on top of issues and pull requests, with extra fields such as status, priority and size.
Why use it?	Everyone sees who is doing what, what's next, and how much is left before the deadline, in one place.
How does it work?	A project holds items that point to issues and PRs, plus its own fields. Views filter, sort and group those items. Built-in workflows update fields automatically when something happens, for example setting status to Done when an issue closes or a PR merges.
When to use it?	As soon as more than a handful of issues are open, and for weekly planning.
Example	In Monday's call, the team drags issues into the "This week" column. All week, the board shows who is doing what without anyone asking.

Moving a card on the board changes the issue's status for everyone. To set one up:

1. In the organization, open **Projects** → **New project**, and start from the **Board** template.
2. The **Status** field becomes the columns. Use Todo, In progress, In review and Done.
3. Add fields that help you plan: **Priority** (single select), **Size** (S, M, L), **Iteration** (one or two weeks), and a target date.
4. Add views with the **+** next to the view tabs: **Board** for daily work, **Table** grouped by assignee for planning, **Roadmap** for dates.
5. Open the **…** menu → **Workflows** and turn on these automations: **Item added to project** (status becomes Todo), **Item closed** and **Pull request merged** (status becomes Done), and **Auto-add to project** with the filter `is:issue,pr is:open` for your repository, so new issues and PRs appear on the board by themselves.

From the terminal:

```
gh auth refresh -s project
gh project list --owner my-org
gh project item-add 3 --owner my-org --url https://github.com/my-org/my-app/issues/42
```

WHAT EACH LINE DOES

`gh auth refresh -s project`	Gives `gh` permission to work with Projects. You only need this once.
`gh project list --owner my-org`	Lists the organization's projects with their numbers.
`gh project item-add 3 --owner my-org --url ...`	Adds issue 42 to project number 3.

Discussions or Issues?

Issues are for work that can be finished and closed. Discussions (a forum tab you can turn on in the settings) are for ideas, questions, decisions and announcements. When a discussion produces work, open an issue and link the discussion from it. That keeps the board for things people can actually pick up.

A weekly rhythm that works for friends

- **Start of the week, 30 minutes on a call:** look at the board, choose the issues for the week, and have everyone assign themselves two to four.
- **Every day, two minutes, in writing:** a message in your chat or a pinned Discussion with "Done yesterday / Doing today / Blocked by".
- **During the week:** pull requests reviewed within a day.
- **End of the week, 30 minutes:** show what merged, close finished issues, move leftovers, and agree on one thing to do better.

CHAPTER 23

GitHub Actions: CI that guards main

Workflows that build and test every pull request automatically.

AT A GLANCE GitHub Actions and CI	
What is it?	GitHub's automation service. It runs the steps you describe in workflow files, such as installing, building and testing your project, whenever certain events happen.
Why use it?	Every change is checked the same way, automatically, on a clean machine. People can forget to run the tests; a workflow never does, and the ruleset can block merging until it passes.
How does it work?	When an event happens (a push, a new pull request), GitHub reads the workflow files in `.github/workflows/` of that commit. For each job it starts a fresh virtual machine (a runner), downloads your code, runs the steps in order, and reports the result as a status check on the commit and the PR.
When to use it?	In every repository that contains code: run tests, linters and builds on every pull request. Also for deploys and scheduled jobs.
Example	Chen opens a PR that accidentally breaks the login tests. The CI check turns red within three minutes, before anyone reviews or merges it.

"CI" (continuous integration) means a robot builds and tests your project every time someone pushes or opens a pull request, and shows a pass or fail result on the PR. Combined with the ruleset from Chapter 9, broken code can't reach `main`. GitHub's robot is called GitHub Actions.

How Actions fits together

Term	Meaning
Workflow	A YAML file in `.github/workflows/`. A repository can have as many as it needs
Event	What starts it: `push`, `pull_request`, `schedule`, `workflow_dispatch` (a manual button), `release` and more
Job	A list of steps that runs on one fresh virtual machine. Jobs run at the same time unless one `needs` another
Step	Either a terminal command (`run:`) or a ready-made action (`uses:`)
Action	A packaged step from the GitHub Marketplace or your own repository, such as `actions/checkout`, which downloads your code
Runner	The machine type: `ubuntu-latest` (Linux), `windows-latest` or `macos-latest`
Expression	`${{ ... }}`, which inserts a value such as `matrix.node` or `secrets.API_KEY`

YAML in two minutes

Workflows are written in YAML. Four rules cover almost everything you'll see:

```yaml
name: CI                  # key: value
on:                       # a key whose value is the indented block below it
  push:
    branches: [main]      # [a, b] is a list on one line
steps:
  - run: npm test         # "- " starts a list item
  - run: npm run build    # second list item
```

WHAT EACH LINE DOES

`key: value`	Each setting is a name, a colon, a space, then the value.
Indentation	Lines indented under a key belong to it. Use spaces, never tabs, and keep the indentation consistent: two spaces per level is usual. Wrong indentation is the most common YAML error.
`- item`	A dash and a space start one item in a list.
`[main]` and `# comment`	Square brackets are a short way to write a list. `#` starts a comment.

A CI workflow for a Node project

Create this file, commit it on a branch and open a PR; the workflow runs on that PR straight away.

`.github/workflows/ci.yml`

```yaml
name: CI

on:
  pull_request:
  push:
    branches: [main]

permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        node: [22, 24]
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v7
        with:
          node-version: ${{ matrix.node }}
          cache: npm
      - run: npm ci
      - run: npm run lint
```

continues on next page

```
      - run: npm test
      - run: npm run build

  ci-ok:
    if: always()
    needs: [test]
    runs-on: ubuntu-latest
    steps:
      - name: Fail if any required job failed
        if: contains(needs.*.result, 'failure') || contains(needs.*.result, 'cancelled')
        run: exit 1
```

WHAT EACH PART DOES

`name: CI`	The workflow's name, shown in the **Actions** tab and on PRs.
`on: pull_request` and `push: branches: [main]`	Run for every pull request, and for every push to `main`.
`permissions: contents: read`	The workflow's automatic access key (`GITHUB_TOKEN`) can only read the code, not change anything.
`concurrency: ...`	If you push again to the same PR while a run is still going, the older run is cancelled. Saves time and minutes.
`jobs: test:`	The first job, called `test`.
`runs-on: ubuntu-latest`	Run it on a fresh Linux machine.
`strategy: matrix: node: [22, 24]`	Run the whole job twice, once with Node 22 and once with Node 24. `fail-fast: false` lets both finish even if one fails.
`uses: actions/checkout@v7`	Download your repository's code onto the machine. `@v7` is the version of that action.
`uses: actions/setup-node@v7` with `node-version` and `cache: npm`	Install the Node version from the matrix, and reuse downloaded packages between runs to save time.
`run: npm ci`	Install the exact package versions listed in `package-lock.json`.
`run: npm run lint`, `npm test`, `npm run build`	Check code style, run the tests, and build the app. If any command fails, the job fails.
`ci-ok:` with `needs: [test]` and `if: always()`	A final job that waits for `test` and always runs.
The step with `if: contains(...)` and `run: exit 1`	Fails `ci-ok` if any job it waited for failed or was cancelled. Otherwise `ci-ok` passes.

Why the extra `ci-ok` job? The ruleset (Chapter 9) needs the *name* of a check to require. Matrix jobs get names like "test (22)" that change whenever you edit the matrix, which would leave the ruleset waiting forever for a check that no longer exists. Require `ci-ok` instead: its name never changes, and it only passes when everything else passed.

After you push, open the PR. Under the description you'll see the checks running, then a green tick or a red cross. Click **Details** next to a check to read its log.

The same for Python

AT A GLANCE **Matrix builds**	
What is it?	Running the same job several times with different settings, such as different language versions or operating systems.
Why use it?	To be sure the project works on every version your team or your users might have, not just the one on your laptop.
How does it work?	GitHub expands the `matrix` into one job per combination and runs them in parallel. Inside each job, `${{ matrix.<name> }}` holds that combination's value.
When to use it?	When your users run different versions or operating systems, and when you're upgrading and need to test old and new side by side.
Example	The team is moving from Node 22 to Node 24. A matrix of `[22, 24]` proves the app works on both before anyone switches.

`.github/workflows/ci.yml (the test job)`

```yaml
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python: ["3.12", "3.13", "3.14"]
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: ${{ matrix.python }}
          cache: pip
      - run: pip install -r requirements.txt
      - run: ruff check .
      - run: pytest
```

WHAT EACH LINE DOES

`python: ["3.12", "3.13", "3.14"]`	Run the job on three Python versions. Always put versions in quotes: unquoted, YAML reads `3.10` as the number 3.1.
`uses: actions/setup-python@v7`	Install Python, and cache downloaded packages (`cache: pip`).
`run: pip install -r requirements.txt`	Install your project's dependencies. `ruff` and `pytest` must be listed in that file.
`run: ruff check .`	Check the code for mistakes and style problems.
`run: pytest`	Run the tests.

Secrets and variables

AT A GLANCE **Actions secrets**	
What is it?	Passwords, API keys and other sensitive values stored in GitHub for workflows to use.
Why use it?	Workflows often need credentials, but anything written in a workflow file is visible to everyone who can read the repository.
How does it work?	Secrets are stored encrypted. They're decrypted and handed to a job only when that job runs and refers to them, and GitHub replaces their values with `***` in logs. Workflows triggered from forks don't receive them.
When to use it?	Whenever a workflow needs a password, token or API key.
Example	The deploy job needs the hosting provider's key. Alice stores it as the secret `DEPLOY_TOKEN`, so it never appears in the code or in the logs.

Some workflows need passwords or API keys, for example to talk to a test database. Never put them in the workflow file; store them as secrets.

```
gh secret set DATABASE_URL
gh secret set API_KEY --body "paste-value-here"
gh variable set APP_ENV --body staging
gh secret list
```

WHAT EACH LINE DOES

`gh secret set DATABASE_URL`	Creates or updates a secret. It asks you to paste the value, so it never appears in your terminal history.
`gh secret set API_KEY --body "..."`	The same, giving the value directly.
`gh variable set APP_ENV --body staging`	A variable is a setting that isn't secret; its value is visible in logs.
`gh secret list`	Lists secret names (never their values).

You can also manage them on the website under **Settings** → **Secrets and variables** → **Actions**. Use them in a workflow like this:

```
      - run: npm run test:e2e
        env:
          DATABASE_URL: ${{ secrets.DATABASE_URL }}
          APP_ENV: ${{ vars.APP_ENV }}
```

WHAT EACH LINE DOES

`env:`	Sets environment variables for this step, which your program reads like any other setting.
`${{ secrets.DATABASE_URL }}` / `${{ vars.APP_ENV }}`	Insert the secret's or variable's value.

- GitHub hides secret values in logs (shows `***`), but don't print them on purpose.
- Workflows started by pull requests from forks don't receive secrets, so strangers can't steal them.
- Don't work around that with the `pull_request_target` event. It runs with your secrets and write access, and if it checks out the PR's code, anyone who opens a PR can steal them.

Least privilege for GITHUB_TOKEN

AT A GLANCE **GITHUB_TOKEN**	
What is it?	A temporary access key that GitHub creates automatically for every workflow run.
Why use it?	It lets a workflow read your code, comment on PRs or publish releases without anyone pasting in a personal token.
How does it work?	GitHub creates the token when a job starts, with the permissions listed under `permissions:`, and it stops working when the job finishes. Keeping its permissions small limits the damage if a step ever runs malicious code.
When to use it?	It's used automatically in every workflow. Decide its permissions in every workflow file.
Example	A workflow that adds labels to new PRs gets `pull-requests: write`; the test workflow keeps only `contents: read`.

Set `permissions:` at the top of each workflow to the minimum it needs, usually `contents: read`, and give more only to the job that needs it, for example `pull-requests: write` for a job that posts a comment on PRs. If a workflow is ever tricked into running bad code, a read-only key limits the damage.

Pin third-party actions

AT A GLANCE **Action versions and pinning**	
What is it?	Choosing exactly which version of an action your workflow runs.
Why use it?	A workflow runs other people's code with access to your repository. If that project is hacked and a version tag is moved to bad code, your workflow would run it.
How does it work?	`@v7` refers to a tag, which its owner can move to a different commit. `@<40-character commit ID>` refers to one exact, unchangeable commit. Dependabot can keep pinned IDs up to date for you.
When to use it?	Every time you add an action that isn't maintained by GitHub.
Example	Bob adds a community action that posts test coverage on PRs. Bob pins it to its full commit ID, and Dependabot proposes updates later.

Actions from `actions/` and `github/` are maintained by GitHub, and a version tag like `@v7` is fine for them. For anyone else's action, use the full commit ID instead of a tag.

```
      - uses: some-org/some-action@8f4b7f84864484a7bf31766abe9204da3cbe65b3   # v2.1.0
```

The comment records which version that ID is. Dependabot (Chapter 26) keeps these IDs and comments up to date for you.

Other useful triggers

```yaml
on:
  workflow_dispatch:
    inputs:
      target:
        type: choice
        options: [staging, production]
  schedule:
    - cron: "0 6 * * 1"
  pull_request:
    paths-ignore:
      - "docs/**"
      - "**.md"
```

WHAT EACH LINE DOES

`workflow_dispatch:`	Adds a **Run workflow** button in the **Actions** tab, so you can start it by hand.
`inputs: target: type: choice`	When started by hand, it asks you to pick `staging` or `production`.
`schedule: - cron: "0 6 * * 1"`	Runs on a timer. The five fields are minute, hour, day of month, month, day of week: here 06:00 UTC every Monday.
`paths-ignore:`	Don't run for pull requests that only change files in `docs/` or Markdown files. `**` means "any folders".

Start a manual run from the terminal with `gh workflow run deploy.yml -f target=staging`. Scheduled workflows in public repositories are switched off automatically after 60 days without any activity in the repository.

> **Path filters and required checks don't mix**
>
> If a workflow is skipped because of `paths` or `paths-ignore`, its checks stay "Pending" forever, and a ruleset that requires them blocks the PR. Either don't require path-filtered workflows, or always run the workflow and skip the work inside it with an `if:` on the job. A job skipped by `if:` counts as passed.

When a run fails

```bash
gh run list --limit 5
gh run watch
gh run view <run-id> --log-failed
gh run rerun <run-id> --failed
```

WHAT EACH LINE DOES

`gh run list --limit 5`	The last five workflow runs, with their status and ID.
`gh run watch`	Pick a run that's in progress and follow it live.
`gh run view <run-id> --log-failed`	Shows only the log output of the steps that failed. Usually the error is in the last 20 lines.
`gh run rerun <run-id> --failed`	Runs only the failed jobs again, useful when a failure was a one-off network hiccup.

For more detail, re-run the job on the website with **Enable debug logging** ticked, or set a repository variable called `ACTIONS_STEP_DEBUG` to `true`. To test workflows on your own computer before pushing, try the open-source tool `act`, which needs Docker.

Reuse instead of copy-paste

AT A GLANCE **Composite actions and reusable workflows**	
What is it?	Ways to define a group of steps, or a whole workflow, once and use it from several workflows.
Why use it?	The same setup copied into five files gets out of sync. One shared definition means one place to fix and update.
How does it work?	A composite action is an `action.yml` file with a list of steps; any job can call it with `uses:` like a normal action. A reusable workflow has `on: workflow_call`; another workflow calls it as a whole job and can pass it inputs and secrets.
When to use it?	When the same steps appear in two or more workflows.
Example	The same setup steps were copied into `test.yml`, `lint.yml` and `deploy.yml`. After moving them into `.github/actions/setup`, upgrading Node means changing one line.

When several workflows repeat the same setup steps, move them into a "composite action" in your repository:

`.github/actions/setup/action.yml`

```yaml
name: Set up project
description: Install Node and dependencies
runs:
  using: composite
  steps:
    - uses: actions/setup-node@v7
      with:
        node-version: 24
        cache: npm
    - run: npm ci
      shell: bash
```

WHAT EACH LINE DOES

`runs: using: composite`	Says this action is just a list of steps.
`steps:`	The steps to run, written like in a workflow.
`shell: bash`	Composite actions must say which shell runs each `run:` command.

Use it in any workflow, after `actions/checkout`, with `- uses: ./.github/actions/setup`. To reuse an entire workflow, add `on: workflow_call` to it and call it from another workflow at the job level with `uses: ./.github/workflows/test.yml`.

Minutes and cost

Standard runners are free for public repositories. Private repositories get a monthly allowance: 2,000 minutes on the Free plan and 3,000 on Pro or Team, at the time of writing. Windows runners use minutes twice as fast and macOS runners ten times as fast. On a private repository, keep the matrix small, use `concurrency` to cancel outdated runs, and cache dependencies.

CHAPTER 24

Deploy from main

GitHub Pages for static sites, environments for everything else.

"Deploying" means putting your app online so people can use it. With GitHub Flow, `main` always works, so the simplest rule is: every time something merges into `main`, deploy it automatically.

GitHub Pages with Actions

AT A GLANCE **GitHub Pages**	
What is it?	Free hosting for static websites, served directly from a GitHub repository.
Why use it?	Your web project is online at a public address at no cost, and every merge to `main` updates it automatically.
How does it work?	The workflow builds the site, packs the finished files as an "artifact", and the `deploy-pages` action hands that artifact to GitHub's web servers, which publish it at `<owner>.github.io/<repo>`.
When to use it?	For static websites: portfolios, documentation, landing pages and frontend-only apps.
Example	The team's React app, built with Vite, is published at `team-pixel.github.io/app`, and every merge to `main` updates it.

GitHub Pages hosts static websites for free: plain HTML, CSS and JavaScript, or the build output of a React or Vue app made with Vite. (It can't run server code like Node, Python or a database.) In the repository, open **Settings** → **Pages** and set **Source** to **GitHub Actions**, then add this workflow:

`.github/workflows/pages.yml`

```yaml
name: Deploy site

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: false

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
```

continues on next page

```
      - uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm
      - run: npm ci
      - run: npm run build
      - uses: actions/configure-pages@v6
      - uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - id: deployment
        uses: actions/deploy-pages@v5
```

WHAT EACH PART DOES

`on: push: branches: [main]` and `workflow_dispatch:`	Deploy on every push to `main`, and allow manual runs.
`permissions: pages: write, id-token: write`	Allow this workflow to publish to Pages and prove its identity to the Pages service.
`concurrency: group: pages`	Only one deploy at a time; a new one waits for the current one rather than cancelling it.
`build` job: checkout, setup-node, `npm ci`, `npm run build`	Download the code, install Node and the packages, and build the site into the `dist` folder.
`uses: actions/configure-pages@v6`	Prepares the Pages settings for the deploy.
`uses: actions/upload-pages-artifact@v5` with `path: ./dist`	Packs the built `dist` folder for publishing. Change the path if your build writes elsewhere (for example `build`).
`deploy` job with `needs: build`	Runs only after `build` succeeds.
`environment: name: github-pages`	Records the deploy in the repository's deployment history, with a link to the live site.
`uses: actions/deploy-pages@v5`	Publishes the site.

The site appears at `https://my-org.github.io/my-app/`. Because it lives under `/my-app/`, a Vite project needs `base: "/my-app/"` in `vite.config.js`, or its CSS and JavaScript links point to the wrong place and you get a blank page. Single-page apps with client-side routing also need hash-based routing, or a copy of `index.html` saved as `404.html`, so that reloading a deep link works.

Environments and protection rules

AT A GLANCE **Environments**	
What is it?	Named deployment targets such as `staging` or `production`, each with its own secrets and protection rules.
Why use it?	To control who can deploy to production and when, and to keep production passwords away from everything else.
How does it work?	A job that names an environment waits until that environment's rules are satisfied (for example, a required reviewer clicks Approve). Only then does it start and receive the environment's secrets. GitHub records each deployment in the repository's history.
When to use it?	When you deploy to real users, need production-only secrets, or want someone to approve each deploy.
Example	Deploys to `staging` happen automatically after every merge. Deploys to `production` wait until Alice clicks Approve.

Adding `environment: production` to a job gives you three things: secrets that only that environment can read, a deployment history on the repository's front page, and protection rules. Protection rules can require someone to click **Approve** before the job runs, add a waiting time, or allow deploys only from `main`. Set them up under **Settings** → **Environments**. On private repositories, some protection rules need a paid plan.

```yaml
deploy:
  needs: test
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  environment:
    name: production
    url: https://my-app.example.com
  steps:
    - uses: actions/checkout@v7
    - run: ./scripts/deploy.sh
      env:
        DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}
```

WHAT EACH LINE DOES

`needs: test`	Only deploy after the `test` job passed.
`if: github.ref == 'refs/heads/main'`	Only deploy when the run is for the `main` branch, never for a pull request.
`environment: name: production`	Use the `production` environment, with its secrets and protection rules.
`run: ./scripts/deploy.sh`	Your own deploy script. What it does depends on where you host the app.
`DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}`	Pass the hosting provider's key to the script from the environment's secrets.

A preview for every pull request

AT A GLANCE **Preview deployments**	
What is it?	A temporary copy of your website, built from a pull request and published at its own address.
Why use it?	Reviewers can click through the actual change in a browser, which catches visual and usability problems that a diff never shows.
How does it work?	The hosting service is connected to your repository. When a PR is opened or updated, it builds that branch, publishes it at a unique URL, and posts the link on the PR. Depending on the service, the preview is deleted when the PR closes or kept for a while.
When to use it?	For web projects where reviewers need to see visual changes, not just code.
Example	Bob's PR changes the homepage colors. The PR comment links to a preview, and Chen opens it on a phone and spots text that's hard to read.

Vercel, Netlify, Cloudflare Pages and Render all offer preview deployments. Most have free tiers for small projects; check their current limits.

Deploy checklist

- Deploys happen only from `main` or from tags, and only after the tests passed (`needs:`).
- Passwords and keys live in environment secrets, never in code or in plain variables.
- Everyone knows how to roll back: revert the PR (Chapter 19), or re-run the deploy workflow from an older successful run.

CHAPTER 25

Automate code quality

Formatters, linters and Git hooks, so reviews can talk about logic instead of commas.

Decide once, automate forever

AT A GLANCE **Linters and formatters**	
What is it?	Programs that check code automatically. A formatter rewrites layout (spaces, quotes, line breaks); a linter warns about likely mistakes such as unused variables.
Why use it?	Style arguments disappear from reviews, everyone's code looks the same, and many bugs are caught before a human even looks.
How does it work?	Each tool reads a config file committed in the repository, so it applies the same rules for everyone. Formatters rewrite files in place; linters print problems and exit with an error code, which makes a hook or CI job fail.
When to use it?	From the first day of the project, in every language you use.
Example	Alice writes single quotes and Bob double quotes. Prettier rewrites both on save, and quotes are never mentioned in a review again.

Pick one formatter and one linter per language, commit their config files, and stop discussing style in reviews:

Language	Formatter	Linter
JavaScript, TypeScript	Prettier or Biome	ESLint or Biome
Python	Ruff (format) or Black	Ruff
Java	google-java-format or Spotless	Checkstyle, SpotBugs
C#	`dotnet format`	Roslyn analyzers
Go	`gofmt`	`go vet`, staticcheck

Run them in three places: when you save a file in your editor (turn on "format on save"), in a pre-commit hook for fast feedback, and in CI as the rule nobody can skip.

Git hooks

AT A GLANCE **Git hooks**	
What is it?	Scripts that Git runs automatically at certain moments, such as just before a commit is created.
Why use it?	To catch problems (formatting, secrets, broken YAML, bad commit messages) on your own computer, seconds before they'd be committed.
How does it work?	Before an action, Git looks in `.git/hooks` for a program named after the event, such as `pre-commit`, and runs it. If it exits with an error, Git cancels the action. Tools like pre-commit and Husky install those programs for you from a config file in the repository.
When to use it?	For quick checks you want before every commit or push: formatting, secrets, large files, commit messages.
Example	Chen accidentally stages a 300 MB screen recording. The pre-commit hook refuses the commit and names the file.

Common hooks are `pre-commit` (before a commit is created), `commit-msg` (after you write the message) and `pre-push` (before a push). Hooks live in `.git/hooks`, which is never committed, so teams share them through a tool.

pre-commit: works with any language (needs Python)

`.pre-commit-config.yaml`

```yaml
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v5.0.0          # pre-commit autoupdate bumps this to the latest
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-merge-conflict
      - id: check-added-large-files
        args: ["--maxkb=1000"]
      - id: detect-private-key
```

WHAT EACH HOOK CHECKS

`repo:` and `rev:`	Where the checks come from, and which version of them to use.
`trailing-whitespace`, `end-of-file-fixer`	Remove invisible spaces at line ends, and make every file end with a newline.
`check-yaml`	Refuse YAML files with syntax errors (such as broken workflow files).
`check-merge-conflict`	Refuse files that still contain conflict markers like `<<<<<<<`.
`check-added-large-files` with `--maxkb=1000`	Refuse files bigger than 1,000 KB, which usually shouldn't be in Git.
`detect-private-key`	Refuse files that look like private keys.

```
pip install pre-commit
pre-commit install
pre-commit run --all-files
pre-commit autoupdate
```

WHAT EACH LINE DOES

`pip install pre-commit`	Installs the tool (once per computer).
`pre-commit install`	Activates the hooks in this repository. Every teammate runs this once per clone.
`pre-commit run --all-files`	Runs every check on every file right now, not just on staged ones.
`pre-commit autoupdate`	Updates the `rev:` versions in the config to the newest releases.

From now on, every `git commit` runs the checks first. If one fails or fixes a file, look at the message, `git add` the fixed files, and commit again. Add your formatter's own hook too; Ruff, Black, Prettier and most others publish one, so copy the snippet from their documentation.

Husky and lint-staged: JavaScript projects

```
npm install --save-dev husky lint-staged
npx husky init
```

WHAT EACH LINE DOES

`npm install --save-dev husky lint-staged`	Installs two tools as development dependencies: Husky manages Git hooks; lint-staged runs checks on staged files only.
`npx husky init`	Creates `.husky/pre-commit` and adds a `prepare` script to `package.json`, so the hooks install themselves whenever a teammate runs `npm install`.

Replace the contents of `.husky/pre-commit` with the single line `npx lint-staged`, and tell lint-staged what to run in `package.json`:

`package.json (excerpt)`

```
{
  "lint-staged": {
    "*.{js,ts,tsx}": ["eslint --fix", "prettier --write"],
    "*.{css,md,json}": "prettier --write"
  }
}
```

WHAT EACH LINE DOES

`"*.{js,ts,tsx}": [...]`	For staged JavaScript and TypeScript files: fix lint problems ESLint can fix by itself, then format with Prettier.
`"*.{css,md,json}": "prettier --write"`	For CSS, Markdown and JSON files: just format them.

lint-staged only checks the files you're committing, so the hook stays fast even in a big project.

Check commit messages

```
npm install --save-dev @commitlint/cli @commitlint/config-conventional
echo "export default { extends: ['@commitlint/config-conventional'] };" > commitlint.config.js
echo "npx --no -- commitlint --edit \$1" > .husky/commit-msg
```

WHAT EACH LINE DOES

`npm install --save-dev @commitlint/cli @commitlint/config-conventional`	Installs commitlint and the Conventional Commits rules (Chapter 12).
`echo "..." > commitlint.config.js`	Creates the config file that tells commitlint to use those rules.
`echo "..." > .husky/commit-msg`	Creates a `commit-msg` hook that checks every commit message. A message like "fixed stuff" is rejected with an explanation.

With squash merges, what ends up on `main` is the pull request title, so check that in CI too. Several Marketplace actions check PR titles against Conventional Commits; pin whichever one you choose to a commit ID (Chapter 23).

> **Hooks are a convenience. CI is the rule.**
>
> Anyone can skip hooks with `git commit --no-verify`, and a fresh clone has no hooks until someone runs the install step. Run the same checks in CI and make them required in the ruleset.

Tests in CI

- Run unit tests on every PR. Keep the whole CI run under about five minutes, or people stop waiting for it.
- Fix "flaky" tests (tests that fail randomly) quickly, or skip them with an issue to fix them. A build that's red half the time teaches everyone to ignore red builds.
- If `main` goes red, fixing it comes before any other work.

CHAPTER 26

Security for small teams

Keep secrets out of Git, keep dependencies patched, and lock down accounts.

Never commit secrets

AT A GLANCE **Secret scanning and push protection**	
What is it?	GitHub features that look for passwords and API keys in the code you push.
Why use it?	A leaked key can be found and abused by bots within minutes. Stopping it before it's pushed is far easier than cleaning up afterwards.
How does it work?	When you push, GitHub checks the new commits against the known formats of keys from hundreds of providers. With push protection on, a push containing one is rejected and you're shown the file and line. Without it, an alert is created after the push.
When to use it?	Always, in every repository. It's free for public repositories.
Example	Bob pastes an API key into `config.js` and pushes. GitHub blocks the push and shows the exact line, and Bob moves the key into `.env` instead.

A "secret" is anything that grants access: passwords, API keys, database URLs with passwords in them, private keys. Once a secret is pushed to GitHub, assume someone has it.

- Put `.env` in `.gitignore` on day one. Commit a `.env.example` file with the same setting names but fake values, and a comment for each one, so teammates know what they need.
- CI secrets go in Actions secrets (Chapter 23). For local development, each person keeps their own `.env`. Share real values through a password manager's shared vault, never in chat.
- Turn on secret scanning and **push protection** in the repository's security settings (**Settings** → **Code security**; some accounts call it **Advanced Security**). Push protection rejects any push that contains a recognized key from GitHub, AWS, Stripe, Google, OpenAI and many other providers. It's free for public repositories.
- In your personal **Settings** → **Code security**, turn on **Push protection for yourself**. It protects every public repository you push to.

`.env.example`

```
# Copy this file to .env and fill in real values from the team vault.

# Connection to your local database
DATABASE_URL=postgres://user:password@localhost:5432/myapp

# API key from the provider's dashboard
API_KEY=replace-me
```

If a secret leaks

1. **Revoke or rotate it immediately** at the provider: create a new API key, change the database password. Assume it's already been copied; bots scan public GitHub pushes within minutes. This is the step that actually fixes the problem.
2. Put the new value in Actions secrets and in everyone's `.env`.
3. Remove the secret from the code in a normal commit.
4. Only if the file also contains something that can't be rotated (personal data, for example), rewrite history to remove it everywhere. Everyone must re-clone afterwards, and open PRs break.

```
pip install git-filter-repo
git clone git@github.com:my-org/my-app.git my-app-clean
cd my-app-clean
git filter-repo --invert-paths --path .env
git remote add origin git@github.com:my-org/my-app.git
git push --force --all origin
git push --force --tags origin
```

WHAT EACH LINE DOES

`pip install git-filter-repo`	Installs the history-rewriting tool (once).
`git clone ... my-app-clean`	Makes a fresh copy to work on; `git filter-repo` refuses to run on anything else, to protect you.
`git filter-repo --invert-paths --path .env`	Rewrites every commit in history without the file `.env`. ("invert" means "keep everything except this path".)
`git remote add origin ...`	filter-repo removes the `origin` remote on purpose, so you add it back.
`git push --force --all origin`	Replaces every branch on GitHub with the cleaned version. An admin must temporarily disable the ruleset first, because it blocks force pushes.
`git push --force --tags origin`	Does the same for tags.

GitHub can keep old copies in cached views and in pull request records; GitHub Support can purge those on request.

Dependabot

AT A GLANCE **Dependabot**	
What is it?	A GitHub bot that watches the libraries your project depends on.
Why use it?	Security holes in libraries are a common way in for attackers, and keeping dozens of packages up to date by hand gets forgotten.
How does it work?	GitHub reads your package files (like `package.json` and its lock file) to know what you use, and compares them with a database of known vulnerabilities to raise alerts. On the schedule you set, Dependabot checks for newer versions and opens pull requests that update the files; your CI then tests them.
When to use it?	In every project that uses outside libraries.
Example	A security hole is announced in a library the team uses. Dependabot opens a PR with the fixed version the same day; the tests pass and Alice merges it.

Turn on **Dependabot alerts** and **Dependabot security updates** in the security settings, then add a config file for regular version updates:

`.github/dependabot.yml`

```yaml
version: 2
updates:
  - package-ecosystem: npm
    directory: /
    schedule:
      interval: weekly
    groups:
      minor-and-patch:
        update-types: [minor, patch]
  - package-ecosystem: github-actions
    directory: /
    schedule:
      interval: weekly
```

WHAT EACH LINE DOES

`version: 2`	The config format version. Always 2.
`package-ecosystem: npm`	Watch npm packages (`package.json`). Other values include `pip`, `maven`, `gradle`, `nuget`, `gomod`, `cargo`, `composer` and `docker`.
`directory: /`	Where the `package.json` is; `/` means the repository's top folder.
`schedule: interval: weekly`	Check once a week.
`groups: minor-and-patch:`	Combine all small updates into one pull request instead of one PR per package.
`package-ecosystem: github-actions`	Also keep the actions in your workflows up to date.

Dependabot opens pull requests; CI tests them; a teammate reviews and merges. Grouping turns fifteen small PRs a week into one.

Code scanning

AT A GLANCE Code scanning (CodeQL)	
What is it?	Automatic analysis of your source code that looks for security bugs.
Why use it?	It finds mistakes like SQL injection or unsafe file paths before they reach users, and points to the exact lines.
How does it work?	CodeQL turns your code into a database that describes how data flows through it, then runs queries that look for dangerous patterns, such as user input reaching a database query unchecked. Results appear as alerts and as comments on pull requests.
When to use it?	In public repositories, and in anything that handles user input or personal data.
Example	CodeQL warns that the value from a search box goes straight into a SQL query. Chen switches to a parameterized query before the release.

Settings → **Code security** → **Code scanning** → **CodeQL analysis** → **Default** turns it on in one click. It's free for public repositories.

Accounts and access

- Everyone uses two-factor authentication, ideally a passkey or an authenticator app rather than SMS. Organization owners can require it for all members under the organization's **Authentication security** settings.
- Give most people Write and two people Admin (Chapter 8).
- If you need a personal access token (for a script or tool), create a **fine-grained** token limited to one repository, with an expiry date. Avoid classic tokens with the broad `repo` scope.
- When someone leaves the project, remove their access, and check **Deploy keys** and installed **GitHub Apps** in the repository settings.

A security policy

A `SECURITY.md` file tells people how to report a vulnerability privately. Turn on **Private vulnerability reporting** in the security settings, and reports arrive as private advisories instead of public issues that everyone (including attackers) can read. Worth doing for any public repository that other people might use.

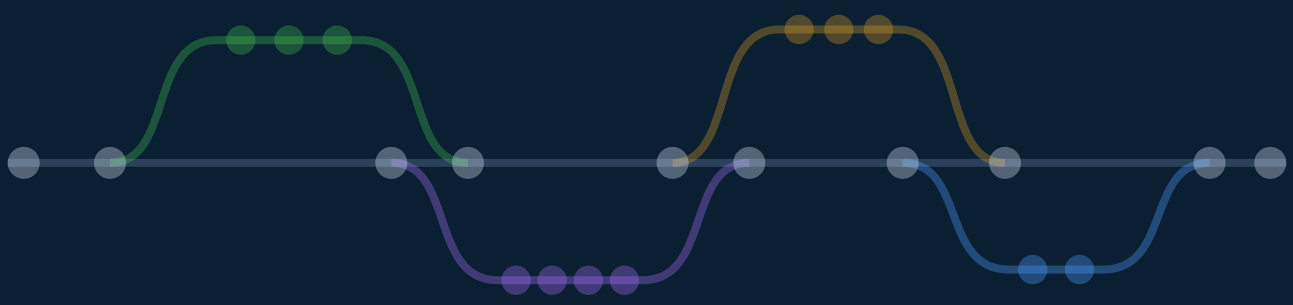
PART VII

Shipping

Version numbers, releases, and tools for repositories that grow.

27 Releases, tags and versions

28 Big files, big repos and shared environments



CHAPTER 27

Releases, tags and versions

Semantic versioning, annotated tags, release notes and hotfixes.

A release is a version of your project that you declare finished and give a number, like `v1.0.0`. It tells users (and your future selves) exactly which code was shipped, and lets you say "this bug was fixed in v1.0.1".

Semantic Versioning

AT A GLANCE Semantic Versioning	
What is it?	A set of rules for version numbers in the form MAJOR.MINOR.PATCH.
Why use it?	Anyone can tell from the number alone whether updating is safe: a patch only fixes things, a minor version adds things, a major version may break things.
How does it work?	You choose which part to increase based on the biggest kind of change since the last release, and reset the parts to its right to zero (for example, `1.4.2` plus a new feature becomes `1.5.0`).
When to use it?	Whenever you publish versions that other people use or install.
Example	The team adds dark mode (a new feature, nothing broken): 1.2.0 becomes 1.3.0. A week later they fix a typo: 1.3.0 becomes 1.3.1.

Version numbers look like `MAJOR.MINOR.PATCH`, for example `2.4.1`:

- **MAJOR** goes up for breaking changes, which force users to change something on their side (an API removed, a config format changed). `2.4.1` → `3.0.0`.
- **MINOR** goes up for new features that don't break anything. `2.4.1` → `2.5.0`.
- **PATCH** goes up for bug fixes. `2.4.1` → `2.4.2`.

Versions starting with `0.` mean "early development, anything may change". Pre-releases add a suffix: `1.0.0-beta.1`. If you write Conventional Commits (Chapter 12), the next version follows from the commit types since the last release: any `!` means major, any `feat` means minor, otherwise patch.

Tags

AT A GLANCE Tags	
What is it?	Names attached to specific commits that, unlike branches, don't move.
Why use it?	To mark released versions permanently, so `v1.0.0` always means exactly the same code.
How does it work?	A lightweight tag is just a file in `.git/refs/tags/` containing a commit ID. An annotated tag is a small object with its own message, author, date and optional signature, which points to the commit. Tags are pushed separately from branches.
When to use it?	For every release, and for moments you want to find again, like the version shown on demo day.
Example	Right before the demo, Alice tags `v1.0.0`. Months later, anyone can check out exactly the code that was shown.

Releases are built on tags.

```
git tag -a v1.0.0 -m "v1.0.0: first public release"
git push origin v1.0.0
git tag -l "v1.*"
git show v1.0.0
git describe --tags
```

WHAT EACH LINE DOES

`git tag -a v1.0.0 -m "..."`	Creates an *annotated* tag called `v1.0.0` on the current commit, with a message. Annotated tags also record who made them and when (and are signed if you set `tag.gpgsign` in Chapter 7).
`git push origin v1.0.0`	Uploads the tag to GitHub. Tags aren't uploaded by a normal `git push`.
`git tag -l "v1.*"`	Lists tags matching a pattern, here every `v1` version.
`git show v1.0.0`	Shows the tag's message and the commit it points at.
`git describe --tags`	Describes where you are relative to the last tag, for example `v1.0.0-14-g3e5a1f0` means "14 commits after v1.0.0, at commit 3e5a1f0". Handy for build numbers.

A tag without `-a` is a "lightweight" tag: just a name, with no author, date or message. Use annotated tags for releases; `push.followTags` and `git describe` work with them.

> **Never move a published tag**
>
> Once a tag is pushed, people and tools rely on it pointing at one specific commit. If a release turns out to be broken, release `v1.0.1`. The tag ruleset from Chapter 9 enforces this.

GitHub Releases

AT A GLANCE **GitHub Releases**	
What is it?	A page on GitHub attached to a tag, with release notes and optional downloadable files.
Why use it?	Users find each version in one place, can read what changed, and can download ready-made builds.
How does it work?	GitHub links the release to the tag. With `--generate-notes`, it lists the pull requests merged between the previous release's tag and this one, sorted using `.github/release.yml`. Uploaded files are stored with the release.
When to use it?	Whenever you ship a version that users should hear about or download.
Example	The team publishes v1.1.0 with automatically generated notes listing the 12 PRs merged since v1.0.0, plus a Windows installer to download.

Your users see releases on the repository's front page.

```
gh release create v1.0.0 --generate-notes
gh release create v1.1.0-beta.1 --prerelease --generate-notes
gh release create v1.0.0 ./dist/app-windows.zip ./dist/app-macos.zip --generate-notes
gh release create v1.0.0 --draft --notes-file notes.md
gh release list
```

WHAT EACH LINE DOES

`gh release create v1.0.0 --generate-notes`	Publishes release `v1.0.0`, with notes written automatically from the PRs merged since the previous release. If the tag doesn't exist yet, it's created from the latest `main`.
`--prerelease`	Marks it as a test version, not the recommended one.
`./dist/app-windows.zip ./dist/app-macos.zip`	Files listed after the tag are attached for download, for example built installers.
`--draft --notes-file notes.md`	Creates an unpublished draft using notes you wrote in `notes.md`, so you can check it on the website before publishing.
`gh release list`	Lists existing releases.

Sort the automatic notes into sections by label with this file:

`.github/release.yml`

```yaml
changelog:
  exclude:
    labels: [skip-changelog]
  categories:
    - title: New features
      labels: [feature]
    - title: Bug fixes
      labels: [bug]
    - title: Other changes
      labels: ["*"]
```

WHAT EACH LINE DOES

`exclude: labels: [skip-changelog]`	PRs with the `skip-changelog` label are left out of the notes.
`categories:`	The sections, in order. Each PR goes into the first section whose labels it has.
`labels: ["*"]`	`*` matches any PR, so everything not caught above lands in "Other changes".

CHANGELOG.md

Release notes live on GitHub; a changelog is a file in the repository, so it travels with the code. The Keep a Changelog format is the common one: newest version first, an "Unreleased" section at the top for things merged but not released yet, and changes grouped under Added, Changed, Fixed and Removed.

CHANGELOG.md

```
# Changelog

## [Unreleased]

## [1.1.0] - 2026-11-20
### Added
- Password reset by email (#42)
### Fixed
- Cart total rounding with tax (#57)

## [1.0.0] - 2026-11-02
- First release
```

Automate releases

Once your PR titles follow Conventional Commits, tools can do the release work for you:

- **release-please** keeps an open "release PR" that bumps the version number and updates the changelog as features merge. Merging that PR creates the tag and the GitHub release.
- **semantic-release** releases automatically on every merge to `main`.

Do v1.0 by hand so everyone understands the steps, then automate.

Hotfixes

AT A GLANCE **Hotfix and release branch**	
What is it?	A hotfix is an urgent fix to an already released version. A release branch is a branch that starts from a release tag, used to make patch releases of that version.
Why use it?	Sometimes `main` already contains unfinished or unreleased work, so you can't simply release `main` to fix a bug.
How does it work?	You create the release branch from the old tag, copy the fix onto it with `git cherry-pick`, and tag a new patch version from there. The fix is merged into `main` too, so it isn't lost in the next release.
When to use it?	When a released version needs an urgent fix, but `main` already contains work that isn't ready to ship.
Example	v1.0.0 is live, and `main` already has a half-finished chat feature. The login crash fix is merged into `main`, cherry-picked onto `release/1.0`, and shipped as v1.0.1.

With GitHub Flow, a hotfix is just a small PR into `main` that gets reviewed first, followed by a patch release (`v1.0.1`).

It gets trickier if `main` already contains unfinished or unreleased features that you don't want to ship yet. Then patch the old version on a release branch:

```
git switch -c release/1.0 v1.0.0
git cherry-pick -x 7c3d9e1
git push -u origin release/1.0
git tag -a v1.0.1 -m "v1.0.1"
git push origin v1.0.1
gh release create v1.0.1 --generate-notes
```

WHAT EACH LINE DOES

`git switch -c release/1.0 v1.0.0`	Creates a branch `release/1.0` starting exactly at the `v1.0.0` release (only once; reuse it for later patches).
`git cherry-pick -x 7c3d9e1`	Copies the fix, which was already merged into `main` as commit `7c3d9e1`, onto the release branch.
`git push -u origin release/1.0`	Uploads the release branch.
`git tag -a v1.0.1 -m "v1.0.1"`	Tags the fixed version.
`git push origin v1.0.1`	Uploads the tag.
`gh release create v1.0.1 --generate-notes`	Publishes the release.

Always fix `main` first and copy the fix to the release branch. If you only fix the release branch, the bug comes back in the next release.

CHAPTER 28

Big files, big repos and shared environments

Git LFS, submodules, monorepos, history surgery and dev containers.

Large files and Git LFS

AT A GLANCE **Git LFS (Large File Storage)**	
What is it?	A Git extension that stores large files outside the normal repository history.
Why use it?	Every version of every file in Git is downloaded by every clone forever. Storing big binaries in LFS keeps clones small and fast.
How does it work?	When you commit a tracked file, LFS replaces its contents in Git with a tiny text "pointer" (the file's hash and size) and uploads the real file to the LFS server when you push. When you check out, LFS downloads the real file and puts it in your folder in place of the pointer.
When to use it?	For large binary files that must be versioned: designs, game assets, audio, video, machine-learning models.
Example	A game team stores 2 GB of textures. With LFS, a fresh clone downloads only the textures for the current version, not every version ever committed.

GitHub warns about files over 50 MB and rejects files over 100 MB. Smaller binary files can hurt too, because every version of every file stays in every clone forever. Ten versions of a 20 MB image make every clone 200 MB bigger.

For design files, game assets, audio, video and datasets, use Git LFS. Git for Windows already includes it.

```
git lfs install
git lfs track "*.psd" "*.blend" "*.mp4"
git add .gitattributes
git add art/hero.psd
git commit -m "chore: track design files with LFS"
git lfs ls-files
```

WHAT EACH LINE DOES

`git lfs install`	Turns on LFS for your user account. Once per computer.
`git lfs track "*.psd" "*.blend" "*.mp4"`	From now on, Photoshop, Blender and MP4 files go to LFS. It writes these patterns into `.gitattributes`.
`git add .gitattributes`	Commit those rules so teammates' Git uses LFS for the same files.
`git add art/hero.psd` and `git commit`	Add and commit large files as usual; LFS handles them behind the scenes.
`git lfs ls-files`	Lists the files currently stored in LFS, so you can check.

LFS storage and download bandwidth have quotas on GitHub; check **Settings** → **Billing**. Files committed *before* you set up LFS stay in history as normal files. `git lfs migrate import --include="*.psd" --everything` converts them, but it rewrites history, so coordinate it like the filter-repo example in Chapter 26.

Better still, keep things out of Git that don't need to be there: build output, downloaded dependencies, and datasets anyone can download again from a URL.

Submodules, and why you might avoid them

AT A GLANCE **Submodule**	
What is it?	A Git repository placed inside another repository, pinned at one specific commit.
Why use it?	To reuse code from another repository while controlling exactly which version you use.
How does it work?	The outer repository records the inner one's address in `.gitmodules` and its commit ID as a special entry in its tree. `git submodule update` clones the inner repository and checks out that recorded commit. Updating means moving the inner repository and committing the new ID in the outer one.
When to use it?	Rarely: when you must include another repository at an exact version and can't publish it as a package.
Example	Two apps share a UI library kept in its own repository. Each app includes it as a submodule pinned at a version that has been tested with that app.

```
git submodule add git@github.com:my-org/shared-ui.git libs/shared-ui
git clone --recurse-submodules git@github.com:my-org/my-app.git
git submodule update --init --recursive
git submodule update --remote libs/shared-ui
git config --global submodule.recurse true
```

WHAT EACH LINE DOES

`git submodule add <url> libs/shared-ui`	Adds the `shared-ui` repository into the folder `libs/shared-ui`, pinned at its current commit.
`git clone --recurse-submodules <url>`	Clones a project and its submodules in one go.
`git submodule update --init --recursive`	Downloads the submodules after a normal clone or pull, at the pinned commits.
`git submodule update --remote libs/shared-ui`	Moves the submodule to the latest commit of its branch. Commit the change to pin the new version.
`git config --global submodule.recurse true`	Makes `pull` and `switch` update submodules automatically.

They're easy to get wrong. People forget to update or commit the pin, the submodule sits in detached HEAD, and you must push the submodule before the main repository or your teammates get a pin to a commit that doesn't exist yet. Simpler alternatives: publish the shared code as a package (npm, PyPI or GitHub Packages), or keep everything in one repository.

Monorepos

AT A GLANCE **Monorepo**	
What is it?	One repository that contains several apps or packages, each in its own folder.
Why use it?	Changes that span several parts (for example the API and the screen that uses it) can be made, reviewed and released in one pull request, and shared code needs no publishing.
How does it work?	Each folder has its own code and config. Tools such as workspaces, CODEOWNERS per folder and path-filtered workflows keep each part's reviews and tests focused.
When to use it?	When the parts of a project change together often, and the team is small.
Example	The team keeps `frontend/` and `backend/` in one repository, so adding a field to the API and to the form that uses it is a single PR.

Typical folders are `frontend/`, `backend/` and `mobile/`. To make a monorepo work:

- CODEOWNERS entries per folder, so the right people review each part.
- One workflow per app with path filters, keeping in mind the required-checks warning in Chapter 23.
- Workspace tools such as npm or pnpm workspaces, Turborepo, Nx, or uv workspaces for Python.

Very large repositories

AT A GLANCE **Partial clone and sparse checkout**	
What is it?	Two ways of working with only part of a big repository: a partial clone downloads file contents only when needed; sparse checkout shows only chosen folders.
Why use it?	Huge repositories become fast to clone and light on disk space.
How does it work?	With `--filter=blob:none`, Git downloads the commits and folder lists but fetches each file's contents from GitHub only when something needs it. Sparse checkout tells Git to write only the listed folders into your working tree.
When to use it?	For very large repositories, or when you only work in one folder of one.
Example	Bob only edits the documentation of a 5 GB repository. A partial clone plus `git sparse-checkout set docs` gives a small, fast folder.

If a repository is huge, you don't have to download all of it:

```
git clone --filter=blob:none git@github.com:my-org/big-repo.git
git sparse-checkout set frontend docs
git sparse-checkout disable
git maintenance start
```

WHAT EACH LINE DOES

`git clone --filter=blob:none <url>`	A "partial clone": downloads the full history but fetches file contents only when they're needed. Much faster for big repositories.
`git sparse-checkout set frontend docs`	Only the `frontend` and `docs` folders appear on your disk. The rest is still in the repository, just not checked out.
`git sparse-checkout disable`	Show all folders again.
`git maintenance start`	Lets Git tidy and optimize the repository in the background on a schedule.

Rewriting history safely with git filter-repo

AT A GLANCE **git filter-repo**	
What is it?	A tool that rewrites a repository's entire history according to rules you give it.
Why use it?	To remove a leaked secret or a huge file from every commit, or to restructure folders throughout history.
How does it work?	It reads every commit, applies your filters (drop this path, rename that folder, remove big files), and writes a new set of commits. From the first changed commit onwards, every commit gets a new ID, which is why everyone has to re-clone afterwards.
When to use it?	Rarely: when a secret or a huge file must disappear from every commit, or when you restructure a repository's history.
Example	Someone committed a database export with real customer emails. After rotating the passwords, the team removes the file from every commit with `git filter-repo`.

Besides removing a leaked file (Chapter 26), `git filter-repo` can strip big files or reorganize folders across the whole history:

```
git filter-repo --strip-blobs-bigger-than 50M
git filter-repo --path-rename old-folder/:new-folder/
```

WHAT EACH LINE DOES

`git filter-repo --strip-blobs-bigger-than 50M`	Removes every file larger than 50 MB from every commit in history.
`git filter-repo --path-rename old-folder/:new-folder/`	Renames a folder throughout history, as if it always had the new name.

The same rules apply every time: tell the team, merge or close open PRs first, work on a fresh clone, temporarily allow force pushes, push, and have everyone re-clone.

To fix how authors appear (for example, one person committed under two different emails) without rewriting anything, add a `.mailmap` file. `git log`, `git shortlog` and `git blame` use it to merge identities:

`.mailmap`

```
# Proper Name <proper email> <email used in old commits>
Alice Khan <1234567+alice@users.noreply.github.com> <alice@old-laptop.local>
```

Each line says: "commits made with the email on the right belong to the name and email on the left". GitHub itself doesn't read `.mailmap`, but Git's own commands do.

The same environment for everyone: dev containers and Codespaces

AT A GLANCE **Dev containers and Codespaces**	
What is it?	A dev container is a development environment described in a file in your repository and run in Docker. Codespaces runs the same thing on a cloud machine.
Why use it?	Everyone gets exactly the same tools and versions, whatever computer they use, and a new teammate is ready to code in minutes.
How does it work?	VS Code (or Codespaces) reads `.devcontainer/devcontainer.json`, builds a container from the listed image and features, runs the setup command, and connects the editor to the container. Your project files are shared into it, so you edit them as usual.
When to use it?	When teammates use different operating systems, when setup takes long, or when someone has to work from a borrowed or slow computer.
Example	Chen's laptop can't run the database the project needs. Opening a codespace gives the complete environment in the browser in two minutes.

When some friends use Windows and others macOS, "works on my machine" problems are guaranteed: different Node versions, missing tools, different paths. A dev container describes the tools and versions once, in a file in the repository. VS Code (with the Dev Containers extension and Docker installed) or GitHub Codespaces (a cloud computer in your browser) then builds exactly that environment for everyone.

`.devcontainer/devcontainer.json`

```
{
  "name": "my-app",
  "image": "mcr.microsoft.com/devcontainers/base:ubuntu",
  "features": {
    "ghcr.io/devcontainers/features/node:1": { "version": "24" },
    "ghcr.io/devcontainers/features/github-cli:1": {}
  },
  "postCreateCommand": "npm ci",
  "forwardPorts": [5173],
  "customizations": {
    "vscode": {
      "extensions": ["dbaeumer.vscode-eslint", "esbenp.prettier-vscode"]
    }
  }
}
```

WHAT EACH LINE DOES

`"image"`	The base system to start from: a standard Ubuntu Linux image made for dev containers.
`"features"`	Extra tools to install on top: Node 24 and the GitHub CLI.
`"postCreateCommand": "npm ci"`	A command run once after the container is built: install the project's packages.
`"forwardPorts": [5173]`	Makes the app's development server (Vite uses port 5173) reachable from your browser.
`"extensions"`	VS Code extensions installed automatically inside the container, so everyone has the same linter and formatter.

To use it locally, open the folder in VS Code and choose **Reopen in Container** when it asks. For Codespaces, open the repository on GitHub and click **Code** → **Codespaces** → **Create codespace**. Personal accounts get a free monthly Codespaces allowance (120 core-hours and 15 GB of storage on the Free plan at the time of writing). Stop codespaces when you're done; they use your allowance while running, and they stop by themselves after 30 idle minutes by default.

PART VIII

More of GitHub

Notifications, profiles, repository management, discussions, packages, tools, AI and integrations.

---

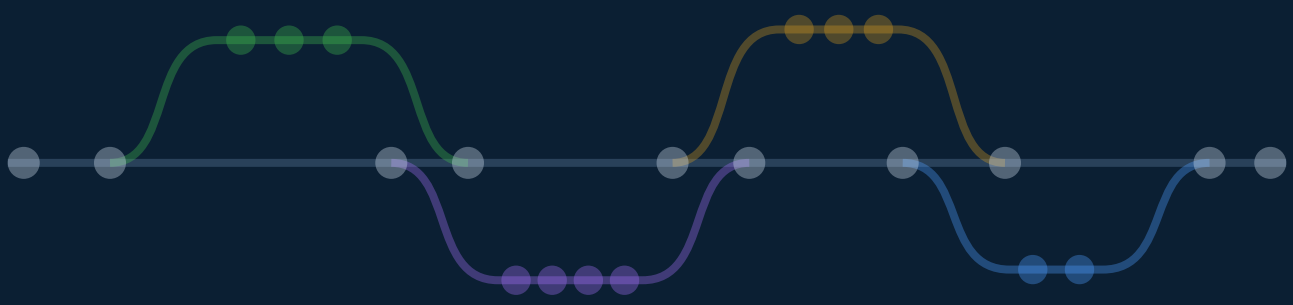
29 Notifications, profiles and the social side

30 Manage your repository

31 Discussions, wikis, gists and packages

32 Tools: GitHub Desktop, VS Code, mobile and Copilot

33 Connect GitHub to other tools: API, webhooks and apps



CHAPTER 29

Notifications, profiles and the social side

Stay on top of what matters, and make your work visible.

Notifications

AT A GLANCE **Notifications**	
What is it?	Messages from GitHub telling you that something happened that involves you: a review request, a mention, a comment on your issue, a failed workflow, a new release.
Why use it?	In a team, the next step often waits for you: a review, an answer, a fix. Notifications make sure you hear about it without checking every repository by hand.
How does it work?	GitHub sends a notification when you're participating (you opened, commented on, were assigned to, or were mentioned in something), when a review is requested from you, or when something happens in a repository you watch. They arrive in your inbox at github.com/notifications (the bell icon) and, depending on your settings, by email and on the mobile app.
When to use it?	Check the inbox at least once a day while a project is active, and tune the settings in the first week so you only get what you need.
Example	Bob requests Alice's review. Alice's inbox shows it with the reason "Review requested"; after reviewing, Alice clicks **Done** and the inbox is empty again.

Work through your inbox

- Each notification shows its reason: review requested, mention, assigned, author, comment, state change, CI activity.
- Click **Done** (or press `e`) when you've handled one. **Save** keeps one for later. **Unsubscribe** stops notifications from that issue or PR.
- The filters on the left narrow the list: by repository, or by reason, for example only "Review requested" or "Mentioned".
- On any issue or pull request, the **Subscribe / Unsubscribe** button on the right decides whether you hear about that one thread.

Choose what you hear about

Go to your picture → **Settings** → **Notifications**:

- Choose **on GitHub**, **email** or both for things you participate in and things you watch.
- Turn on notifications for **failed workflows only**, so you hear about broken builds but not every green one.
- In an organization, **Custom routing** sends that organization's notifications to a different email address, for example your school account.

On each repository, the **Watch** button sets how much you hear: **Participating and @mentions** (only threads you're involved in), **All Activity** (everything), **Ignore**, or **Custom** (only issues, pull requests, releases, discussions or security alerts). Repositories you get push access to may be watched with All Activity automatically; the **Automatically watch repositories** option in your notification settings controls that. For your team's repository, Custom with pull requests and releases is a good start.

Mentions, reactions and saved replies

- `@alice` in any comment notifies Alice; `@my-org/backend` notifies everyone in that team. Use them to ask a specific person, not to notify everybody.
- Use **reactions** (the smiley icon on a comment) instead of writing "+1" or "thanks!" comments that notify everyone.
- **Saved replies** (Settings → Saved replies) store answers you type often, like "Thanks! Could you add steps to reproduce?". Insert them from the arrow icon in any comment box.

Your profile

AT A GLANCE **GitHub profile**	
What is it?	Your public page at `github.com/your-name`, showing who you are and what you've built and contributed.
Why use it?	It's your developer portfolio. Teammates, open-source maintainers and employers look at it to see what you work on.
How does it work?	It shows your picture, name, bio and links, up to six pinned repositories, your profile README (Chapter 5), a contribution graph of the last year, your organizations and achievements. Everything comes from your account settings and your activity on GitHub.
When to use it?	Set it up early, and update your pinned repositories whenever you finish a project you're proud of.
Example	When the team ships v1.0, Alice pins the team repository to Alice's profile and adds a line about it in the profile README. Visitors see the project first.

- **Edit profile** (on your profile page): picture, name, bio, pronouns (optional), company or school, location, website and social links.
- **Customize your pins:** choose up to six repositories (yours or ones you contributed to) to show first.
- **Set status:** a short message with an emoji, such as "On holiday until Monday" or "Focusing on exams".

The contribution graph

The green squares count one day each. A day turns green when you, for example, push commits to the default branch of a repository (not a fork), open an issue or pull request, or review a pull request. Commits only count if their email is linked to your account (Chapter 7). If most of your work is in private repositories, turn on **Include private contributions on my profile** in the contribution settings above the graph: others see the count, but not the details.

Stars, lists and discovering projects

- Star repositories you want to find again; they're listed under **Stars** on your profile. Use **Lists** to sort them, such as "Learning", "UI libraries" or "Tools".
- **Explore** (github.com/explore) and **Trending** (github.com/trending) show popular and fast-growing repositories, and **Topics** group repositories by subject, like `github.com/topics/react`.
- **GitHub Sponsors** lets you support open-source developers with money, and lets developers receive it.

CHAPTER 30

Manage your repository

The About box, community files, licenses, templates, renaming, archiving and insights.

The About box and topics

Click the gear next to **About** on the repository page to set a description, a website link (for example your GitHub Pages site) and **topics**, such as `react`, `school-project` or `hackathon`. Topics make the repository easier to find in search. Under **Settings** → **General** → **Social preview** you can upload an image that's shown when the repository link is shared in chat apps and social media.

Community health files

GitHub recognizes a set of standard files and links to them in the right places:

File	Purpose
`README.md`	What the project is and how to start (Chapter 5)
`LICENSE`	What others are allowed to do with the code
`CONTRIBUTING.md`	How to contribute; linked when someone opens an issue or PR (Chapter 34)
`CODE_OF_CONDUCT.md`	Expected behavior in the community
`SECURITY.md`	How to report vulnerabilities privately (Chapter 26)
`SUPPORT.md`	Where to get help
`.github/ISSUE_TEMPLATE/`, `pull_request_template.md`	Forms and templates (Chapter 8)

Insights → **Community standards** shows a checklist of which of these the repository has, with buttons to add the missing ones. An organization can set defaults for all its repositories by creating a public repository called `.github` and putting these files in it.

Choosing a license

License	In short
MIT	Anyone can use, copy and change the code, even in closed products, as long as they keep your copyright notice. Simple and very common
Apache-2.0	Like MIT, plus an explicit patent grant. Common in larger projects
GPL-3.0	Anyone can use and change it, but if they distribute a changed version, it must also be GPL and open source
No license	Nobody may legally copy, change or redistribute the code, even if the repository is public

choosealicense.com, made by GitHub, explains the options in plain language. For a school project, ask your teacher whether a license is expected.

Template repositories

AT A GLANCE **Template repository**	
What is it?	A repository marked as a template, which anyone can use to start a new repository with the same files and folders, but without its history.
Why use it?	Every new project starts with the same setup (README, .gitignore, CI workflow, issue forms, rulesets explained in CONTRIBUTING) instead of copying files by hand.
How does it work?	Tick **Template repository** under **Settings** → **General**. A **Use this template** button appears on the repository page; it creates a new repository containing a copy of the current files as a single fresh commit.
When to use it?	When your team or class starts several similar projects, such as one repository per assignment or per hackathon.
Example	The team keeps `team-pixel/starter` with their CI, templates and editor settings. For the next hackathon, Chen clicks **Use this template** and the new repository is ready in ten seconds.

From the terminal: `gh repo create team-pixel/new-app --private --template team-pixel/starter --clone`.

Rename, transfer, change visibility, archive or delete

These live at the bottom of **Settings** → **General**, in the **Danger Zone**:

- **Change repository visibility:** public to private or the other way round. Making a repository private removes stars and watchers, so think before switching back and forth.
- **Rename:** GitHub redirects the old URL and old clone addresses to the new name, but update your remotes anyway with `git remote set-url origin <new address>`.
- **Transfer ownership:** move the repository to another account or organization.
- **Archive this repository:** makes it read-only. Use it for finished projects.
- **Delete this repository:** you must type the name to confirm. Deleted repositories can sometimes be restored within 90 days from the owner's settings, but don't count on it.

AT A GLANCE **Archiving a repository**	
What is it?	Marking a repository as read-only. It stays visible, but nobody can push, open issues or pull requests, or comment.
Why use it?	It tells everyone the project is finished and no longer maintained, while keeping it available for reference and in your portfolio.
How does it work?	GitHub shows an "archived" banner and blocks all changes. Clones and forks still work. You can unarchive it from the same settings page if you start working on it again.
When to use it?	When a school project is graded, a hackathon is over, or a project is replaced by a new one.
Example	After the semester ends, the team archives `team-pixel/exam-planner`. Visitors see it's finished, and nobody opens issues expecting fixes.

Rename the default branch

Older repositories often use `master`. To rename it, go to **Settings** → **General** → **Default branch**, click the pencil, and enter `main`. GitHub updates open pull requests and redirects links. Then everyone updates their own copy once:

```
git branch -m master main
git fetch origin
git branch -u origin/main main
git remote set-head origin -a
```

WHAT EACH LINE DOES

`git branch -m master main`	Renames your local branch.
`git fetch origin`	Downloads the renamed branch from GitHub.
`git branch -u origin/main main`	Links your `main` to GitHub's `main`.
`git remote set-head origin -a`	Updates which branch your computer considers GitHub's default.

Insights: what's happening in the repository

AT A GLANCE **Repository insights**	
What is it?	Charts and statistics about a repository: who contributes, how much activity there is, and how many people visit.
Why use it?	To see the health and progress of a project at a glance, and to show your contributions.
How does it work?	The **Insights** tab has several pages. **Pulse** summarizes the last week or month (merged PRs, opened and closed issues). **Contributors** charts commits per person. **Traffic** shows views, clones and where visitors came from over the last 14 days (visible to people with push access). **Commits** and **Code frequency** show activity over time, **Dependency graph** lists the libraries you use, and **Network** draws all branches and forks.
When to use it?	In weekly reviews and retrospectives, and before demos or job applications when you want numbers.
Example	In the retrospective, the team opens Pulse: 23 pull requests merged and 31 issues closed this month, and Contributors shows the work was shared evenly.

CHAPTER 31

Discussions, wikis, gists and packages

Places on GitHub for conversations, documentation, snippets and published software.

GitHub Discussions

AT A GLANCE GitHub Discussions	
What is it?	A forum inside a repository, with categories, threads, answers and polls.
Why use it?	Questions, ideas and announcements don't belong in the issue list, which should hold only actionable work. Discussions keep conversations searchable and next to the code.
How does it work?	Turn it on under **Settings** → **General** → **Features** → **Discussions**. A Discussions tab appears with categories such as Announcements, General, Ideas, Q&A (where an answer can be marked as accepted), Show and tell, and Polls. Any issue can be turned into a discussion with **Convert to discussion** on its page.
When to use it?	For open questions, design ideas that need input, weekly updates and announcements; especially useful when people outside the team use your project.
Example	Bob isn't sure whether to use a database or simple JSON files. Bob starts a discussion in Ideas, everyone weighs in, and the decision and reasons stay written down for later.

Wikis

AT A GLANCE GitHub Wiki	
What is it?	A set of documentation pages attached to a repository, edited in the browser and written in Markdown.
Why use it?	Longer documentation (setup guides, architecture notes, meeting notes) gets its own space with a sidebar and page links, editable by anyone with access.
How does it work?	Turn it on under **Settings** → **General** → **Features** → **Wikis**. The wiki is stored as its own small Git repository, so you can also clone it with `git clone https://github.com/my-org/my-app.wiki.git` and edit pages locally. Wiki changes don't go through pull requests. Wikis are free for public repositories and need a paid plan for private ones.
When to use it?	For documentation that changes independently of the code and doesn't need review. For documentation that should be reviewed with the code, use a `docs/` folder instead.
Example	The team keeps meeting notes and a "How we deploy" page in the wiki, but the API documentation lives in `docs/`, so it's updated in the same pull requests as the code.

Gists

AT A GLANCE **Gist**	
What is it?	A quick way to share a single file or a few files, like a code snippet, a config or notes, without creating a full repository.
Why use it?	Sharing a snippet in chat loses formatting and history. A gist has a link, syntax highlighting, comments and versions.
How does it work?	Create one at gist.github.com or with `gh gist create`. A **public** gist appears on your profile and in search. A **secret** gist is unlisted, not private: anyone with the link can see it. Every gist is a small Git repository, so it has history and can be cloned and forked.
When to use it?	For snippets, examples, small scripts and notes you want to share by link or reuse later.
Example	Chen writes a useful script that cleans up merged branches and shares it with the team as a gist instead of pasting it into chat.

```
gh gist create cleanup.sh --desc "Delete merged branches"
gh gist create notes.md --public
gh gist list
```

WHAT EACH LINE DOES

`gh gist create cleanup.sh --desc "..."`	Creates a secret gist from the file, with a description, and prints its link.
`gh gist create notes.md --public`	Creates a public gist.
`gh gist list`	Lists your gists.

Never put passwords or keys in a gist, secret or not.

GitHub Packages and the container registry

AT A GLANCE **GitHub Packages**	
What is it?	GitHub's hosting for software packages that your project publishes: Docker container images, npm packages, Maven and Gradle packages, NuGet packages and Ruby gems.
Why use it?	Your build output lives next to its source code, with the same permissions, and workflows can publish to it using the automatic `GITHUB_TOKEN`.
How does it work?	Each package is published to a registry (for containers it's `ghcr.io`) under your account or organization and can be linked to a repository. Access follows the repository's or package's visibility settings. GitHub Actions can publish packages with `packages: write` permission.
When to use it?	When your project produces something others install or run, like a Docker image of your backend or a shared library used by several of your apps.
Example	The team's backend workflow builds a Docker image on every release and pushes it to `ghcr.io/team-pixel/backend`. The hosting server pulls the new image to update itself.

A workflow that publishes a container image on every release:

.github/workflows/publish-image.yml

```yaml
name: Publish image

on:
  release:
    types: [published]

permissions:
  contents: read
  packages: write

jobs:
  image:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - name: Sign in to ghcr.io
        env:
          TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: echo "$TOKEN" | docker login ghcr.io -u ${{ github.actor }} --password-stdin
      - run: docker build -t ghcr.io/team-pixel/backend:${{ github.event.release.tag_name }} .
      - run: docker push ghcr.io/team-pixel/backend:${{ github.event.release.tag_name }}
```

WHAT EACH LINE DOES

`on: release: types: [published]`	Runs whenever a GitHub release is published (Chapter 27).
`permissions: packages: write`	Allows the workflow's token to publish packages.
`env: TOKEN: ...` and `docker login ghcr.io ...`	Passes the automatic token to the step as the variable `TOKEN`, and signs in to GitHub's container registry with it.
`docker build -t ghcr.io/team-pixel/backend:<tag> .`	Builds the image from the `Dockerfile` in the repository and names it after the release tag, for example `v1.2.0`. Image names must be lowercase.
`docker push ...`	Uploads the image. It appears under **Packages** on the organization page. To show it on the repository page too, add the line `LABEL org.opencontainers.image.source=https://github.com/team-pixel/backend` to the `Dockerfile`.

CHAPTER 32

Tools: GitHub Desktop, VS Code, mobile and Copilot

Ways to use GitHub without the terminal, and with an AI assistant.

The terminal is the most powerful way to use Git, and this guide uses it throughout. But you don't have to do everything there. These tools cover everyday work with buttons and panels, and everyone on the team can use whichever they prefer, because they all work on the same repository.

GitHub Desktop

AT A GLANCE GitHub Desktop	
What is it?	GitHub's free desktop app for Windows and macOS that does everyday Git work with buttons instead of commands.
Why use it?	Seeing your changes, picking what to commit and switching branches is easier to learn visually, and the app makes mistakes less likely for beginners.
How does it work?	Sign in with your GitHub account, then clone repositories from a list. The **Changes** tab shows every changed file with a diff; the checkboxes next to files and lines are the staging area. You type a summary, click **Commit to <branch>**, then **Push origin**. The branch menu creates and switches branches, and **Create Pull Request** opens the PR page on GitHub.
When to use it?	When learning, for quick everyday commits, and for teammates who prefer not to use the terminal. For advanced operations such as `rebase --onto` or the reflog, use the terminal.
Example	Chen is new to Git. With GitHub Desktop, Chen sees exactly which lines changed, unticks a debug line, commits the rest, and pushes, all on the first day.

Download it from desktop.github.com. It uses the same repository as the terminal, so you can switch between the two at any time.

VS Code's Source Control panel

AT A GLANCE VS Code Source Control	
What is it?	The Git tools built into Visual Studio Code, plus GitHub's official extension for pull requests and issues.
Why use it?	You can stage, commit, push, switch branches, resolve conflicts and even review pull requests without leaving your editor.
How does it work?	Open the **Source Control** panel (`Ctrl+Shift+G`). Changed files are listed; click **+** to stage a file (or select lines in the diff and stage just those), type a message and click **Commit**, then **Sync Changes** to pull and push. The branch name at the bottom left switches or creates branches. Conflicts open in a three-way merge editor. The **GitHub Pull Requests** extension lists PRs and issues and lets you check out and review a PR, commenting on lines inside the editor.
When to use it?	If VS Code is your editor, for most everyday Git work. Keep the terminal for anything the panel doesn't offer.
Example	Alice reviews Bob's pull request inside VS Code: checks it out from the GitHub Pull Requests panel, runs the app, and leaves two comments on specific lines without opening a browser.

GitHub Mobile

AT A GLANCE GitHub Mobile	
What is it?	GitHub's official app for iOS and Android.
Why use it?	Reviews and questions often block teammates. The app lets you unblock them from anywhere.
How does it work?	Sign in, and your notifications arrive as push notifications. You can read and comment on issues and pull requests, review code changes, approve, merge, and approve two-factor sign-in requests.
When to use it?	For quick reviews, answering questions and merging small, approved PRs while you're away from your computer.
Example	On the bus, Bob gets a notification that a typo fix is waiting for review, reads the one-line change, approves it, and Alice's PR merges before Bob gets home.

GitHub Copilot

AT A GLANCE GitHub Copilot	
What is it?	GitHub's AI coding assistant. It suggests code as you type, answers questions in a chat panel, can review pull requests, and on some plans can work on an issue by itself and open a pull request.
Why use it?	It speeds up routine code, explains unfamiliar code and error messages, and gives a quick first review.
How does it work?	Copilot runs as an extension in editors such as VS Code and JetBrains IDEs, and inside github.com. It sends the relevant code context to an AI model and shows its suggestions, which you accept, change or ignore. Plans and limits change over time: there is a free tier with monthly limits, and verified students can get more through GitHub Education. Check github.com/features/copilot for the current details.
When to use it?	For boilerplate, tests, explaining code and drafting documentation. Not as a replacement for understanding your own code or for a human review.
Example	Alice asks Copilot Chat to explain a regular expression found in the code and to suggest test cases for it, then reads and runs every test before committing them.

Rules for AI-written code

Whoever opens the pull request is responsible for every line, whoever or whatever wrote it. Run it, test it, understand it, and mention in the description which parts were generated (Chapter 17).

CHAPTER 33

Connect GitHub to other tools: API, webhooks and apps

Automate your own workflows, and plug GitHub into chat and hosting services.

The GitHub API

AT A GLANCE GitHub API	
What is it?	GitHub's programming interface. Everything you can do on the website (reading repositories, creating issues, merging PRs) can be done by a program sending web requests.
Why use it?	To automate repetitive work and build your own tools and reports, such as a script that creates the same labels in every new repository.
How does it work?	There is a REST API (one web address per kind of thing, like `/repos/my-org/my-app/issues`) and a GraphQL API (one address, flexible queries). Requests are authenticated with a token. Signed-in REST requests are limited to about 5,000 per hour. `gh api` sends requests using your `gh` login, so you don't need to handle tokens.
When to use it?	When something you do by hand on GitHub is repeated often, or when you want data that the website doesn't show the way you need it.
Example	Before each weekly meeting, Bob runs one `gh api` command that lists all open issues without an assignee, and pastes the list into the meeting notes.

```
gh api user
gh api repos/my-org/my-app/issues --paginate --jq '.[].title'
gh api repos/my-org/my-app/issues -f title="Write tests for login" -f body="See #42"
```

WHAT EACH LINE DOES

`gh api user`	Shows information about your own account, as JSON (structured text that programs read).
`gh api repos/.../issues --paginate --jq '.[].title'`	Gets all open issues (`--paginate` follows every page of results) and prints only their titles. `--jq` filters JSON output. GitHub's API counts pull requests as issues too, so open PRs appear in this list as well.
`gh api repos/.../issues -f title="..." -f body="..."`	Creates a new issue. Adding `-f` fields makes `gh api` send a POST request, which means "create".

For scripts that run without your `gh` login, such as in a scheduled workflow, use the automatic `GITHUB_TOKEN` or a fine-grained personal access token limited to one repository (Chapter 26).

Webhooks

AT A GLANCE **Webhook**	
What is it?	A message GitHub sends to a web address you choose whenever something happens in a repository: a push, a new issue, a merged pull request, a release.
Why use it?	Other services can react instantly, for example posting updates in your team chat or starting a deployment, without checking GitHub over and over.
How does it work?	Under **Settings** → **Webhooks** → **Add webhook**, you enter the receiving address (the "payload URL"), the content type, an optional secret used to sign each message, and which events to send. When an event happens, GitHub sends an HTTP request with the details as JSON. The **Recent Deliveries** tab shows each message and its response, and can resend one.
When to use it?	When a service you use offers a webhook address, or when you build your own bot or deploy tool.
Example	The team wants GitHub activity in their Discord server. A Discord webhook address with `/github` added at the end receives GitHub's messages directly, and new pull requests and releases appear in the `#dev` channel.

To connect a Discord channel:

1. In Discord, open the channel settings → **Integrations** → **Webhooks** → **New Webhook**, and copy its URL.
2. On GitHub, go to **Settings** → **Webhooks** → **Add webhook**.
3. Paste the URL as the **Payload URL** and add `/github` to the end of it.
4. Set **Content type** to `application/json`.
5. Choose **Let me select individual events** and tick, for example, Pull requests, Issues and Releases. Click **Add webhook**.

GitHub Apps and the Marketplace

AT A GLANCE GitHub Apps and the Marketplace	
What is it?	Apps are services that connect to your GitHub account or organization with specific permissions, such as hosting providers, chat bots or code-quality tools. The Marketplace at github.com/marketplace lists apps and Actions.
Why use it?	You can add features without building them: preview deployments, test coverage reports, chat notifications, project management integrations.
How does it work?	Installing an app shows exactly which permissions it asks for and which repositories it may access. You can limit it to selected repositories. Installed apps are listed under **Settings** → **Applications** (personal) or the organization's **GitHub Apps** settings, where you can change or remove them. "Sign in with GitHub" on other websites uses OAuth apps, which are listed under **Settings** → **Applications** → **Authorized OAuth Apps**.
When to use it?	When a service you rely on offers a GitHub integration. Review permissions before installing, and remove apps you no longer use.
Example	The team installs a hosting provider's app on the repository only. Every pull request now gets a preview link, and the app can't see any of the team's other repositories.

gh extensions

The GitHub CLI can be extended with commands written by GitHub and the community:

```
gh extension browse
gh extension install dlvhdr/gh-dash
gh extension list
gh extension upgrade --all
```

WHAT EACH LINE DOES

`gh extension browse`	Opens an interactive list of available extensions to read about and install.
`gh extension install dlvhdr/gh-dash`	Installs an extension from its repository. This one, a popular example, adds `gh dash`, a terminal dashboard of your pull requests and issues.
`gh extension list`	Shows installed extensions.
`gh extension upgrade --all`	Updates all installed extensions.

Extensions run code on your computer with your GitHub login, so install only ones you trust, just as you would any other program.

PART IX

Playbook and reference

Run the project from start to finish, fix common errors fast, and practice before it counts.

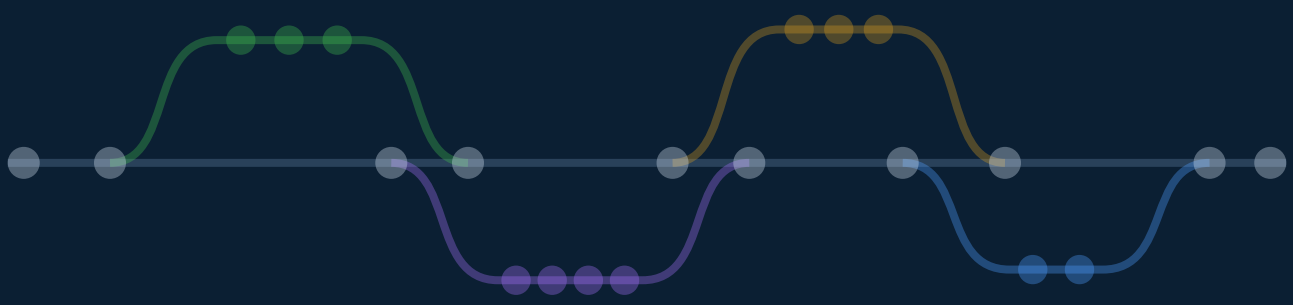
---

34 Team project playbook: idea to v1.0

35 Troubleshooting common errors

36 Cheat sheet

37 Practice drills



CHAPTER 34

Team project playbook: idea to v1.0

The whole guide as a schedule, with checklists for every phase.

This chapter turns the rest of the guide into a plan you can follow with your friends. Each phase says what to do, who does it, and which chapter explains the details. Tick the boxes as you go.

Roles

Role	Responsibilities
Maintainer (one or two people)	Creates the organization, repository, rulesets and automatic tests. Keeps the board tidy. Makes the final call when a technical discussion gets stuck. Publishes releases
Developer (everyone)	Picks issues, works on branches, opens PRs, keeps their own PRs moving, fixes test failures on their own PRs
Reviewer (everyone)	Reviews within a day. Every PR needs at least one approval. Rotate reviewers so everyone learns every part of the code

You can rotate the maintainer role for each release, so more than one person learns the setup.

Phase 0: Kickoff (one call, about an hour)

Agree on these together and write the answers down in the README and CONTRIBUTING.md:

- ☐ What you're building, in one paragraph, and what's explicitly *not* part of v1.0
- ☐ Tech stack and versions: language, framework, database, hosting
- ☐ The deadline or demo date. It becomes the v1.0 milestone
- ☐ Who the maintainer is
- ☐ The working agreement (the template at the end of this chapter)
- ☐ Where you talk. One chat channel for quick questions; decisions get written into an issue or a Discussion so you can find them later

Phase 1: Repository setup (maintainer, one to two hours)

- ☐ Free organization created, friends invited, team `core` with Write access (Chapter 8)
- ☐ Public or private decided, knowing the plan limits (Chapter 8)
- ☐ Repository created with README, LICENSE and .gitignore (Chapter 8)
- ☐ `.gitattributes`, `.editorconfig`, `.env.example`, CODEOWNERS, PR template and issue forms added (Chapters 7, 8 and 26)
- ☐ App skeleton created so it runs with one command, and the README's "Getting started" tested on a second computer
- ☐ Test workflow with the `ci-ok` job merged, and seen running (Chapter 23)
- ☐ Ruleset on main: PR required, one approval, `ci-ok` required, no force pushes (Chapter 9)

- [ ] Merge settings: squash only, auto-delete branches, auto-merge allowed (Chapter 8)
- [ ] Dependabot and secret scanning push protection turned on (Chapter 26)
- [ ] Labels, a "v1.0" milestone and a project board with workflows (Chapter 22)

Phase 2: Everyone's first day (each person, about 30 minutes)

- [ ] Accept the organization invitation (check your email, or your GitHub notifications)
- [ ] Git config, SSH key, signing key and `gh auth login` done (Chapter 7)
- [ ] Repository cloned with `git clone git@github.com:my-org/my-app.git`, and app and tests running by following the README
- [ ] Hooks installed (`pre-commit install`, or `npm install` for Husky) (Chapter 25)
- [ ] First PR opened and merged: add your name to `CONTRIBUTORS.md`, using the full loop from Chapter 11

Phase 3: Conflict drill (together, 30 minutes)

Everyone edits the *same line* of `CONTRIBUTORS.md` at the same time and opens a PR. The first PR merges cleanly. Everyone else now has a conflict, and resolves it one at a time by rebasing on `main` (Chapter 15):

```
git fetch
git rebase origin/main
git add CONTRIBUTORS.md
git rebase --continue
git push --force-with-lease
```

WHAT EACH LINE DOES

`git fetch`	Download the new `main`, which now contains the first person's change.
`git rebase origin/main`	Replay your commit on top of it. Git stops with a conflict in `CONTRIBUTORS.md`.
(edit the file)	Open it, keep both names, delete the marker lines.
`git add CONTRIBUTORS.md`	Mark the conflict as resolved.
`git rebase --continue`	Finish the rebase.
`git push --force-with-lease`	Upload your updated branch. The PR can now merge.

Half an hour of practice with something harmless means nobody panics when it happens in real code.

Phase 4: Plan the backlog (together, about an hour)

- [ ] Every user-facing feature written as an issue with acceptance criteria (Chapter 22)
- [ ] Every issue sized at two days of work or less; bigger ones split into sub-issues
- [ ] Technical tasks added too: deploy pipeline, error pages, README, sample data
- [ ] Labels, sizes and priorities set; everything for v1.0 put in the milestone
- [ ] First week chosen: each person takes two or three issues, in different parts of the code to start with, to avoid early conflicts

Phase 5: Build (repeat every week)

Each person, for each issue, runs the loop from Chapter 11:

```
gh issue develop 42 --checkout --name feat/42-login-form
git add -p
git commit
git push
gh pr create --draft --fill
git fetch && git rebase origin/main && git push --force-with-lease
gh pr ready && gh pr edit --add-reviewer alice
gh pr merge --auto --squash --delete-branch
```

WHAT EACH LINE DOES

`gh issue develop 42 --checkout --name ...`	Create a branch for issue 42 from the latest `main` and switch to it.
`git add -p` and `git commit`	Save your work in small, reviewed commits. Repeat as often as needed.
`git push`	Upload the branch.
`gh pr create --draft --fill`	Open a draft PR, so the team sees your work and the tests run.
`git fetch && git rebase origin/main && git push --force-with-lease`	Every day: bring in your friends' merged work and update your branch.
`gh pr ready && gh pr edit --add-reviewer alice`	When finished: mark the PR ready and ask for a review.
`gh pr merge --auto --squash --delete-branch`	Merge automatically once it's approved and the tests pass, then clean up.

Each week: plan on Monday, post short daily written updates, show and review what merged on Friday (Chapter 22).

Signs that things are drifting, and what to do:

Signal	Healthy	Act if
Age of open PRs	Under 2 days	Over 3 days: review it today, or split it
PR size	Under 400 lines	Over 800 lines: split it before review
Tests on main	Green	Red for more than an hour: fixing it is everyone's top priority
Branch age	Under 3 days	Over a week: rebase, merge what's done, or close it
Issues in progress per person	1 or 2	More than 3: finish before starting anything new

Phase 6: Stabilize (the last week before release)

- ☐ Feature freeze: from now on only bug fixes merge
- ☐ Bug bash: everyone uses the app for an hour, trying to break it, and files issues with the bug form
- ☐ Fix by priority; anything that isn't critical moves to the next milestone
- ☐ README finished: setup steps, screenshots, live URL, team credits
- ☐ Production deployed from `main` and checked by hand (Chapter 24)

Phase 7: Release v1.0

First, bump the version in a normal PR, for example titled `chore: release v1.0.0`, which updates the version in `package.json` (or `pyproject.toml`) and adds a v1.0.0 section to CHANGELOG.md. With npm, `npm version 1.0.0 --no-git-tag-version` changes the version number in `package.json` without creating a tag. After that PR merges:

```
git switch main
git pull
git tag -a v1.0.0 -m "v1.0.0"
git push origin v1.0.0
gh release create v1.0.0 --generate-notes
```

WHAT EACH LINE DOES

`git switch main` and `git pull`	Get onto the latest `main`, which now includes the version bump.
`git tag -a v1.0.0 -m "v1.0.0"`	Put the permanent `v1.0.0` label on that commit (Chapter 27).
`git push origin v1.0.0`	Upload the tag.
`gh release create v1.0.0 --generate-notes`	Publish the release page with notes listing every merged PR.

- ☐ Release notes read and edited where needed
- ☐ Announced: a short demo video or screenshots; pin the repository on everyone's GitHub profile
- ☐ Milestone closed

Phase 8: Retrospective, and after the release

Hold a 30-minute call. What went well? What slowed you down? Pick one or two changes and write them into CONTRIBUTING.md. Then keep the project healthy: merge Dependabot PRs every week, fix bugs with the hotfix process (Chapter 27), and open the next milestone.

Template: CONTRIBUTING.md with your working agreement

Copy this into the repository's top folder as `CONTRIBUTING.md` and adjust it to what your team agreed. GitHub links to it whenever someone opens an issue or PR.

CONTRIBUTING.md

```
# Contributing to my-app

## How we work
- Every change goes through a pull request into `main`. No direct pushes.
- Branch names: `<type>/<issue>-<short-name>`, e.g. `feat/42-login-form`.
- PR titles follow Conventional Commits: `feat: ...`, `fix: ...`, `docs: ...`.
- PRs stay under about 400 changed lines. Bigger work is split or stacked.
- One approval and passing tests are required. We squash-merge.
- Reviews happen within one day. If you can't, say so in the PR.
- Rebase your branch on `main` at least daily. Use `--force-with-lease`.
- Don't push to someone else's branch without asking first.
- Never commit secrets. Copy `.env.example` to `.env`.
```

continues on next page

```
## Getting started
1. `git clone git@github.com:my-org/my-app.git && cd my-app`
2. `npm install` (also installs the Git hooks)
3. `cp .env.example .env`, then fill in values from the team vault
4. `npm run dev`

## Definition of done
- [ ] The issue's acceptance criteria are met
- [ ] Tests added, CI green
- [ ] Reviewed and approved
- [ ] Merged, deployed, and the issue is closed

## Decisions
Decisions go in a Discussion or in the related issue, not only in chat.
```

CHAPTER 35

Troubleshooting common errors

The message, the cause and the fix for the problems teams hit most.

Each entry starts with the message as Git or GitHub prints it, then explains what it means and how to fix it. If your message isn't here, copy its first line into a search engine: you're almost never the first person to see it.

`! [rejected] main -> main (fetch first)`

The full message continues "Updates were rejected because the remote contains work that you do not have locally." **Cause:** someone pushed to that branch since you last downloaded it, so GitHub won't let your push overwrite their work. **Fix:** run `git pull` (which, with the Chapter 7 config, puts your commits on top of theirs), then `git push`.

`! [rejected] feat/x -> feat/x (non-fast-forward)` after a rebase

Cause: expected. A rebase gives your commits new IDs, so GitHub's copy of the branch can't simply be extended. **Fix:** on your own branch, `git push --force-with-lease`.

`fatal: The current branch feat/x has no upstream branch`

Cause: the branch doesn't exist on GitHub yet, and Git doesn't know where to push it. **Fix:** `git push -u origin HEAD` this once (`HEAD` means "the branch I'm on"). Set `push.autoSetupRemote true` (Chapter 7) and you won't see it again.

`Your branch and 'origin/feat/x' have diverged`

Cause: your copy and GitHub's copy both have commits the other doesn't. **Fix:** first see which ones with `git log --oneline --left-right HEAD...@{u}`. Lines marked `<` are yours, `>` are GitHub's. If you rebased on purpose, `git push --force-with-lease`. If a teammate pushed to your branch, `git pull --rebase` to put your commits on top of theirs.

`You are in 'detached HEAD' state`

Cause: you checked out a commit or a tag instead of a branch, so new commits won't belong to any branch. **Fix:** to keep work you did there, create a branch right away with `git switch -c <new-branch>`. Otherwise just `git switch main`. If you already switched away and lost commits, they're in `git reflog` (Chapter 19).

`error: Your local changes to the following files would be overwritten by checkout`

The same happens with merge and pull. **Cause:** you have uncommitted changes in files that the other branch also changes, and Git won't throw them away for you. **Fix:** commit them, or put them aside with `git stash push -u`, switch, and later `git stash pop` once you're back.

`CONFLICT (content): Merge conflict in src/app.js`

Cause: both sides changed the same lines. **Fix:** see Chapter 15.

`Permission denied (publickey)`

Cause: Git couldn't prove who you are over SSH. **Fix:** check, in order: `ssh -T git@github.com` (does GitHub recognize you?), `ssh-add -l` (is your key loaded?), `git remote -v` (is the address an SSH one, starting with `git@`?), and whether the key appears under **Settings** → **SSH and GPG keys** on GitHub. On Windows, see the `core.sshCommand` setup in Chapter 7.

`remote: Permission to my-org/my-app.git denied to bob`

Often followed by "The requested URL returned error: 403". **Cause:** either you don't have access yet (has the invitation been accepted?), or Git is using a different GitHub account saved on your computer. **Fix:** check your access on the repository page. With HTTPS on Windows, open **Credential Manager** from the Start menu, remove the `git:https://github.com` entry under **Windows Credentials**, and push again to sign in fresh. Or run `gh auth status` and then `gh auth setup-git`, so Git uses the account `gh` is signed in with.

`GH013: Repository rule violations found for refs/heads/main`

Also seen as `GH006: Protected branch update failed`. **Cause:** the ruleset is doing its job: you tried to push straight to `main`. **Fix:** push a branch and open a PR. If you committed on `main` by accident, move the commits to a branch (Chapter 19).

`this exceeds GitHub's file size limit of 100.00 MB`

The full message is like "remote: error: File data/dump.sql is 142.00 MB; this exceeds GitHub's file size limit of 100.00 MB". **Cause:** the file is inside one of the commits you're pushing, so deleting it in a new commit doesn't help: the old commit still contains it. **Fix,** if those commits haven't been pushed yet:

```
git reset --soft origin/main
git restore --staged data/dump.sql
echo "data/dump.sql" >> .gitignore
git commit -m "feat: import script (without the dump)"
git push
```

WHAT EACH LINE DOES

`git reset --soft origin/main`	Undo your local, unpushed commits, but keep all their changes staged. (On a feature branch that already exists on GitHub, use `git reset --soft @{u}` instead.)
`git restore --staged data/dump.sql`	Take the big file out of the staging area. It stays on your disk.
`echo "data/dump.sql" >> .gitignore`	Add the file to `.gitignore` so it isn't added again by accident.
`git commit -m "..."`	Commit everything else again, without the big file.
`git push`	Now the push succeeds.

If the file is in older commits that are already on GitHub, remove it with `git filter-repo` or move it to LFS (Chapter 28).

`fatal: refusing to merge unrelated histories`

Cause: you ran `git init` on your computer *and* created the repository on GitHub with a README, so there are two separate histories with nothing in common. **Fix:** merge them once with `git pull origin main --allow-unrelated-histories`, resolve any conflicts, and push. Next time, create the GitHub repository empty, or clone it first and work inside the clone.

`warning: in the working copy of 'x', LF will be replaced by CRLF`

Cause: Git is converting line endings (Chapter 7). It's a warning, not an error. **Fix:** add the `.gitattributes` file from Chapter 7 and renormalize.

Every line of a file shows as changed

Cause: usually line endings, which `.gitattributes` fixes. Run `git diff` and look at the top of a file's diff: if it says "old mode 100644 / new mode 100755", a file permission flag changed instead (common when mixing Windows and WSL). **Fix:** for the permission case, `git config core.fileMode false` tells Git to ignore permission changes.

`error: The branch 'feat/x' is not fully merged`

Cause: normal after a squash merge, because Git can't see that the squash commit contains your branch's work. **Fix:** if the PR shows "Merged", delete the branch with `git branch -D feat/x`.

The PR shows dozens of commits and files I didn't change

Cause: the PR's base branch is wrong, or your branch was started from another feature branch instead of `main`. **Fix:** change the base with the **Edit** button next to the PR title, or move your commits with `git rebase --onto origin/main <old-base> <your-branch>` (Chapter 14).

A required check says "Expected: Waiting for status to be reported"

Cause: the check name in the ruleset doesn't match any job that ran. Maybe a job was renamed, the matrix changed, or the workflow was skipped by a path filter. **Fix:** update the ruleset, or require the single `ci-ok` job from Chapter 23.

This branch is out-of-date with the base branch

Cause: `main` moved on since you last updated, and the ruleset requires branches to be up to date. **Fix:** click **Update branch** on the PR, or run `git fetch`, `git rebase origin/main` and `git push --force-with-lease`.

`error: cannot lock ref 'refs/remotes/origin/...'`

Cause: your computer's records of GitHub branches are stale or clash with each other (for example, GitHub now has a branch called `feat` while you still have a record of `origin/feat/x`). **Fix:** `git fetch --prune`.

My commits don't show my avatar or count on my profile

Cause: the email in those commits isn't one of your GitHub account's verified emails. **Fix:** check with `git config user.email` and correct it (Chapter 7). This fixes future commits; old ones keep the old email.

A hook blocks my commit

Cause: a pre-commit or commit-msg check found a problem (Chapter 25). **Fix:** read the message, which usually names the exact file and rule. Fix it, `git add` again and commit again. Use `--no-verify` only in a real emergency, and remember that CI runs the same check.

I committed a secret

Rotate it first, right now: generate a new key or password at the provider. Then follow Chapter 26.

CHAPTER 36

Cheat sheet

The commands from this guide, grouped by job.

Every command here is explained in full in the chapter shown in brackets after each group name.

First steps (Chapters 2 and 3)

Command	Does
`git --version`	Check that Git is installed
`git config --global user.name "Your Name"`	Set the name written into your commits
`git clone <address>`	Download a repository from GitHub
`git init`	Turn the current folder into a new repository
`git status`	See what's changed and what's staged
`git add <file>` / `git add .`	Stage one file / everything
`git commit -m "message"`	Save a snapshot
`git push` / `git pull`	Upload your commits / download others'
`git log --oneline`	Short history
`git diff`	See your unstaged changes
`git switch -c <branch>` / `git switch <branch>`	Create a branch / change branch
`git merge <branch>`	Bring another branch's work into this one

Daily work (Chapter 11)

Command	Does
`git switch main && git pull`	Go to main and download the latest work
`git switch -c feat/42-name`	Create a new branch from where you are and switch to it
`gh issue develop 42 --checkout`	Branch linked to issue 42, from the latest main
`git status`	What's changed, what's staged, which branch you're on
`git add -p`	Stage changes one block at a time, after looking at each
`git commit -v`	Commit, with the diff shown in the editor
`git push`	Upload your branch (creates it on GitHub the first time, with Chapter 7's config)
`gh pr create --draft --fill`	Open a draft pull request
`git fetch && git rebase origin/main`	Bring your branch up to date with main
`git push --force-with-lease`	Upload a rebased branch safely
`gh pr ready`	Draft becomes ready for review
`gh pr merge --auto --squash --delete-branch`	Merge once tests and approvals pass
`git fetch --prune`	Forget branches that were deleted on GitHub

Looking around (Chapters 6 and 21)

Command	Does
`git status -sb`	Short status with branch info
`git log --oneline --graph --all`	The whole commit graph
`git log main..HEAD`	Your commits that aren't on main yet
`git diff`	Changes you haven't staged
`git diff --staged`	What you're about to commit
`git diff main...HEAD`	Everything your branch changes
`git show <sha>`	One commit and its changes
`git log -S "text"`	Commits that added or removed some text
`git blame -w -C -C -C <file>`	Who last changed each line, ignoring moves and spacing
`git bisect start` / `run <cmd>`	Find the commit that introduced a bug
`git reflog`	Everywhere HEAD has been: your safety net

Rewriting your own branch (Chapters 12 and 14)

Command	Does
`git commit --amend --no-edit`	Add staged changes to the last commit
`git commit --fixup <sha>`	Create a fix aimed at an earlier commit
`git rebase -i origin/main`	Edit, combine, reorder or drop commits on your branch
`git rebase --onto origin/main <old-base> <branch>`	Move a branch to a new starting point
`git rebase -i --exec "npm test" origin/main`	Run the tests at every commit
`git rebase --continue` / `--abort`	Carry on after a conflict / give up and go back

Undo (Chapter 19)

Command	Does
`git restore <file>`	Throw away unstaged edits to a file (permanent)
`git restore --staged <file>`	Unstage a file, keep the edits
`git restore --source=<sha> <file>`	Bring back an old version of a file
`git reset --soft HEAD~1`	Undo the last commit, keep its changes staged
`git reset --hard origin/<branch>`	Match GitHub exactly (throws away local work)
`git revert <sha>`	Undo a commit with a new commit (safe on shared branches)
`git reset --hard ORIG_HEAD`	Undo the rebase or reset you just did
`git branch <name> <sha>`	Bring back a deleted branch

Juggling (Chapter 20)

Command	Does
`git stash push -u -m "msg"`	Put all changes aside, including new files
`git stash pop`	Bring the newest stash back
`git cherry-pick -x <sha>`	Copy one commit onto this branch
`git worktree add <path> <branch>`	Check out a second branch in another folder
`gh pr checkout 57`	Check out a teammate's pull request

Releases (Chapter 27)

Command	Does
`git tag -a v1.2.0 -m "v1.2.0"`	Create an annotated version tag
`git push origin v1.2.0`	Upload the tag
`gh release create v1.2.0 --generate-notes`	Publish a release with notes from merged PRs
`git describe --tags`	Which release you're on, and how far past it

GitHub CLI

Command	Does
`gh repo create my-org/app --public --clone`	Create a repository and download it
`gh repo fork owner/repo --clone`	Fork and download, with `upstream` set up
`gh issue list --assignee @me`	Your issues
`gh issue create --title "..." --label bug`	New issue
`gh pr status`	Your PRs and your review requests
`gh pr review 57 --approve`	Approve a pull request
`gh pr checks --watch`	Follow the tests on the current PR
`gh run view <id> --log-failed`	Logs of failed workflow steps
`gh secret set NAME`	Add an Actions secret
`gh workflow run deploy.yml`	Start a manual workflow
`gh browse`	Open the repository in the browser

CHAPTER 37

Practice drills

Exercises to run with your team before the real project.

Make a throwaway repository in your organization, called something like `git-dojo`, with the same ruleset and test setup as the real one. Run these drills there. Each takes 10 to 30 minutes, and each teaches something that's far less stressful to learn on a repository nobody cares about. Every drill lists its steps and a "done when" check, so you know you've got it.

Drill 1: Conflict on purpose

Goal: resolve a conflict during a rebase without panicking.

1. Everyone clones `git-dojo` and creates a branch: `git switch -c drill/<your-name>`.
2. Everyone changes the *same line* of `README.md` to something different, then runs `git commit -am "docs: my version"` and `git push`, and opens a PR with `gh pr create --fill`.
3. Merge the first PR.
4. Everyone else runs `git fetch` and `git rebase origin/main`. Git stops with a conflict.
5. Open `README.md`, combine the versions, delete the marker lines, then `git add README.md` and `git rebase --continue`.
6. `git push --force-with-lease`, and merge. The next person repeats steps 4 to 6.

Done when everyone has resolved at least one conflict during a rebase, and can explain why "ours" and "theirs" swap during one (Chapter 15).

Drill 2: Clean up a messy branch

Goal: use interactive rebase to turn messy commits into clean ones.

1. On a new branch, make six small commits, including ones called "wip", "typo" and "oops". `git commit --allow-empty -m "wip"` makes a quick empty commit if you don't want to edit files each time.
2. Run `git rebase -i origin/main`. Use `fixup`, `squash` and `reword` to end up with two commits with good messages.
3. Make one more change, and commit it with `git commit --fixup <id of the first commit>`.
4. Run `git rebase -i origin/main` again and check that autosquash has moved it into place. Save and close.

Done when `git log --oneline origin/main..` shows exactly two commits, and each one passes the tests.

Drill 3: Break things, then recover

Goal: trust the reflog.

1. Make three commits, then run `git reset --hard HEAD~3`. They seem gone.
2. Run `git reflog`, find the line from just before the reset, and run `git reset --hard HEAD@{1}` (or use the ID you found). They're back.
3. Create a branch with one commit on it, switch away, and delete it with `git branch -D <name>`. Note the "(was abc1234)" in Git's message.
4. Recreate it with `git branch <name> abc1234`.

Done when you've recovered both without help, and you can say which kind of lost work the reflog *can't* recover (uncommitted changes).

Drill 4: Bisect race

Goal: find the commit that introduced a bug, by hand and automatically.

This script builds a repository with 50 commits and a bug hidden at a random commit. Run it in Git Bash in an empty folder. One person then hunts by hand with `git bisect good` and `git bisect bad`; another uses `git bisect run`.

```
mkdir bisect-drill && cd bisect-drill && git init -q
git config core.autocrlf false
bad=$((RANDOM % 49 + 2))
for i in $(seq 1 50); do
  if [ "$i" -ge "$bad" ]; then echo "status=broken" > app.txt
  else echo "status=ok" > app.txt; fi
  echo "$i" > build.txt
  git add . && git commit -qm "Change $i"
done
git bisect start HEAD "$(git rev-list --max-parents=0 HEAD)"
git bisect run grep -q "status=ok" app.txt
git bisect reset
```

WHAT EACH LINE DOES

`mkdir bisect-drill && cd bisect-drill && git init -q`	Make a new folder, go into it, and turn it into an empty Git repository (`-q` means quiet).
`git config core.autocrlf false`	Stops line-ending warnings on Windows for this throwaway repository.
`bad=$((RANDOM % 49 + 2))`	Picks a random number from 2 to 50: the commit where the bug appears. It's stored in a variable called `bad`.
`for i in $(seq 1 50); do ... done`	Repeats the lines inside 50 times, with `i` counting 1, 2, 3 ... 50.
`if [ "$i" -ge "$bad" ]; then ... else ...; fi`	From commit number `bad` onwards, write `status=broken` into `app.txt`; before that, write `status=ok`.
`echo "$i" > build.txt`	Changes another file every time, so every commit has something in it.
`git add . && git commit -qm "Change $i"`	Commits everything with the message "Change 1", "Change 2" and so on.
`git bisect start HEAD "$(git rev-list --max-parents=0 HEAD)"`	Starts bisect with the newest commit as bad and the very first commit (the one with no parents) as good.
`git bisect run grep -q "status=ok" app.txt`	At each step, `grep` succeeds (good) if the file says `status=ok` and fails (bad) if it doesn't. Git finishes by printing the first bad commit.
`git bisect reset`	Ends the search and goes back to the newest commit.

Done when `git bisect` names the first bad commit, and its message "Change N" matches the hidden number (`echo $bad` shows it).

Drill 5: Stacked PRs

Goal: practice dependent pull requests and `--onto`.

1. `git switch -c feat/a origin/main`, make a commit, push, and open PR A into `main`.
2. `git switch -c feat/b feat/a`, make a commit, push, and open PR B with `gh pr create --base feat/a --fill`.
3. Squash-merge PR A, with branch deletion on. Watch GitHub change PR B's base to `main`.
4. Run `git fetch` and `git rebase --onto origin/main feat/a feat/b`, then `git push --force-with-lease`.

Done when PR B's "Files changed" shows only B's changes, and it merges cleanly.

Drill 6: Red CI

Goal: see the ruleset and automatic tests protect `main`.

1. Open a PR that adds a deliberately failing test.
2. Wait for the checks to go red, and try to merge. The button is blocked.
3. Read the failure with `gh run view <run-id> --log-failed`.
4. Fix the test, push, and watch the PR turn green.

Done when the PR merges only after the fix.

Drill 7: Fork round trip

Goal: contribute to a repository you can't push to.

1. Fork a teammate's *personal* repository with `gh repo fork <their-name>/<repo> --clone`.
2. Create a branch, fix something small, push to your fork, and open a PR into their repository.
3. After they merge it, sync your fork: `git fetch upstream`, `git switch main`, `git merge --ff-only upstream/main`, `git push origin main`.

Done when your fork's `main` matches upstream and has no commits of its own.

Drill 8: Release and hotfix

Goal: publish a release, then patch it without shipping unfinished work.

1. Tag `v0.1.0` and run `gh release create v0.1.0 --generate-notes`.
2. Merge one more feature into `main`.
3. Fix a "bug" in a PR into `main`, and merge it. Note the commit ID of the fix.
4. Create `release/0.1` from the tag with `git switch -c release/0.1 v0.1.0`, copy the fix with `git cherry-pick -x <id>`, push the branch, tag `v0.1.1`, push the tag, and create the release.

Done when `v0.1.1` contains the fix but not the new feature. Check with `git log --oneline v0.1.0..v0.1.1`, which should list only the fix.

Drill 9: Committed on main

Goal: rescue commits made on the wrong branch.

1. On your local `main`, make a commit and try to push it. The ruleset refuses.
2. Run `git branch rescue/my-work`, then `git reset --hard origin/main`, then `git switch rescue/my-work`.
3. Push the branch and open a PR.

Done when your local `main` matches `origin/main` and the commit is in a PR.

Where to go next

- **Pro Git** by Scott Chacon and Ben Straub, free at git-scm.com/book. Chapters 3 (branching), 7 (Git tools) and 10 (internals) go deeper than this guide.
- **GitHub Docs** at docs.github.com, especially the sections on pull requests, rulesets and Actions.
- **Learn Git Branching** at learngitbranching.js.org: interactive, visual exercises for rebase, cherry-pick and more. Excellent for beginners.
- **GitHub Skills** at skills.github.com: short hands-on courses that run inside your own repositories.
- **Oh Shit, Git!?!** at ohshitgit.com: plain-language recovery recipes.
- `git help <command>` opens the full manual for any command; `git <command> -h` prints a short summary.